

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Proyecto ReportaYA

Reporte de Diseño del Software Diseño orientado a objetos SPRINT 2

Versión 1.0

Autores:

Ordoñez Flores Enzo Fabrizio
Bellido Mera Eros Aaron Ricardo
Montenegro Carrillo Sebastian
Yance Valenzuela Anthony
Cabezas Diaz Renzo Edgar
Riveros Janampa Gustavo Adolfo

Grupo: 6

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Historial de Revisiones

Fecha	Versión	Descripción	Autor
22/11/2025	Versión 1.0	Primera versión con los apartados y contenidos básicos	• Bellido Mera Eros Aaron Ricardo • Cabezas Diaz Renzo Edgar • Montenegro Carrillo Sebastian • Ordoñez Flores Enzo Fabrizio • Riveros Janampa Gustavo Adolfo • Yance Valenzuela Anthony

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Tabla de Contenidos

Índice

1. Introducción	5
1.1 <i>Propósito de este documento</i>	5
1.2 <i>Visión general del sistema</i>	5
1.3 <i>Objetivos de diseño</i>	5
1.4 <i>Referencias</i>	5
1.5 <i>Definiciones, siglas y abreviaturas</i>	5
2. Consideraciones Éticas y el Impacto Global de la Solución	6
2.1 <i>Responsabilidades Éticas y Profesionales</i>	6
2.2 <i>Impacto Social de la Solución</i>	6
2.3 <i>Impacto Económico de la Solución</i>	6
2.4 <i>Impacto Ambiental de la Solución</i>	6
2.5 <i>Impacto Global de la Solución</i>	6
2.6 <i>Estrategias de Mitigación y Diseño Consciente</i>	6
3. Visión general del diseño	7
3.1 <i>Introducción</i>	7
3.2 <i>Vista lógica de la arquitectura del Software</i>	7
3.3 <i>Prácticas y metodología de Desarrollo</i>	7
3.4 <i>Diseño Integral</i>	7
4. Diseño Estructural	7
4.1 <i>Explicación y justificación del diseño</i>	7
4.2 <i>Diagrama de clases de diseño</i>	8
4.3 <i>patrón de Diseño</i>	9
5. Modelo dinámico	9
5.1 <i>Historia de Usuario [nombre]</i>	9
5.1.1 <i>Diagrama de Secuencia</i>	9
5.1.2 <i>Código del diagrama de secuencia</i>	10

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

6.	Interfaces de usuario y modelo de datos	10
6.1	<i>Interfaces del sistema</i>	10
6.1.1	Interfaz buscar un empleado	10
6.1.2	Interfaz del registrar un empleado	11
6.2	<i>Modelo de Base de Datos</i>	11
6.3	<i>Script De La Base De Datos</i>	12
7.	Prueba de Software	12
7.1	<i>Prueba de Caja Blanca</i>	12
7.2	<i>Prueba de Caja Negra</i>	12
7.3	<i>pruebas Unitaria</i>	13
8.	Código en GitHub	13
7. 1	<i>URL del repositorio</i>	13
7.2	<i>Código Del Sprint</i>	13
7.3	<i>Contribución por participante</i>	13

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Reporte de Diseño de Software del Sprint

1. Introducción

Este documento presenta el diseño detallado del software correspondiente al Sprint 2 del proyecto ReportaYA. Esta etapa se centra en la expansión de las funcionalidades del sistema para cubrir el ciclo de vida completo de las incidencias urbanas, desde su visualización geoespacial y seguimiento por parte del ciudadano, hasta su validación, asignación, resolución y cierre por parte de los actores municipales (Administrador, Operador y Técnico de Campo).

El reporte está dirigido a la audiencia prevista del proyecto, incluyendo al equipo de desarrollo, evaluadores académicos y stakeholders. Se describen las decisiones arquitectónicas y de diseño técnico necesarias para soportar nuevos flujos de trabajo complejos, gestión de roles jerárquicos y manipulación de evidencias multimedia, apoyados en diagramas UML (clases, secuencia) y modelos de datos actualizados.

1.1 Propósito de este documento

El propósito de este documento es presentar el diseño detallado del software ReportaYA, una aplicación móvil desarrollada para mejorar la gestión municipal de incidencias urbanas en ciudades del Perú. Este documento está dirigido a:

- El equipo de desarrollo responsable de la implementación.
- Docentes y evaluadores académicos.
- Stakeholders del proyecto (municipalidades y ciudadanía).
- Profesionales que requieran comprender la arquitectura técnica del sistema.

El documento proporciona una especificación completa de la estructura del software siguiendo principios de diseño orientado a objetos, incluyendo diagramas UML, modelos de datos, descripciones de clases y justificaciones de las decisiones tecnológicas adoptadas. Sirve como referencia tanto para el desarrollo actual como para futuras mejoras y mantenimiento del sistema.

1.2 Visión general del sistema

ReportaYA es una aplicación móvil que permite a los ciudadanos reportar incidencias urbanas (como acumulación de basura, veredas dañadas o alumbrado público defectuoso) y hacer seguimiento del estado de atención por parte de las municipalidades.

El sistema también proporciona a las entidades municipales una plataforma para validar, clasificar, asignar y supervisar los reportes recibidos, mejorando la eficiencia y

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

transparencia en la gestión de incidencias urbanas.El proyecto se alinea con los siguientes Objetivos de Desarrollo Sostenible (ODS):

ODS 9: Industria, Innovación e Infraestructura

El proyecto ReportaYA contribuye significativamente a la meta de modernizar la infraestructura y reconvertir las industrias para que sean sostenibles, utilizando los recursos con mayor eficacia y promoviendo la adopción de tecnologías y procesos industriales limpios y ambientalmente racionales. En el contexto de la gestión municipal, la "industria" de servicios públicos a menudo se ve obstaculizada por procesos manuales, burocracia en papel y falta de datos en tiempo real, lo que resulta en una infraestructura urbana descuidada y una respuesta ineficiente.

ReportaYA impulsa la innovación digital al reemplazar estos sistemas obsoletos con una arquitectura de software moderna basada en la nube, integración de geolocalización precisa y procesamiento de datos en tiempo real. Al implementar una solución tecnológica que permite la **desmaterialización de procesos** (eliminando formularios físicos) y la optimización de rutas para los técnicos de campo, el sistema no solo mejora la eficiencia operativa, sino que también sienta las bases para una infraestructura digital municipal resiliente.

Esta digitalización permite a las municipalidades recopilar y analizar datos históricos sobre fallos en la infraestructura (como patrones en roturas de tuberías o fallas de alumbrado), facilitando pasar de un modelo de mantenimiento reactivo a uno preventivo. De esta manera, el software actúa como un catalizador para la innovación en la gestión pública, demostrando cómo la tecnología móvil y los sistemas de información robustos son herramientas esenciales para construir instituciones más eficaces, transparentes y capaces de sostener el crecimiento urbano acelerado del Perú.

ODS 11: Ciudades y comunidades sostenibles

El ODS 11 busca lograr que las ciudades y los asentamientos humanos sean inclusivos, seguros, resilientes y sostenibles. ReportaYA ataca directamente las problemáticas que degradan la calidad de vida urbana, como la acumulación de residuos sólidos, el deterioro de veredas y la falta de alumbrado público, factores que no solo afectan la estética de la ciudad, sino que representan riesgos directos para la salud pública y la seguridad ciudadana.

La plataforma fomenta la sostenibilidad al empoderar a los ciudadanos, convirtiéndolos en agentes activos de cambio (sensores humanos) que participan directamente en el mantenimiento de su entorno. Al reducir las barreras para reportar incidencias mediante una interfaz móvil accesible e inclusiva, ReportaYA democratiza el acceso a los servicios municipales, asegurando que las zonas periféricas o históricamente desatendidas tengan la misma visibilidad y capacidad de respuesta que los centros urbanos consolidados.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Además, el sistema de triaje y priorización (BAJA, MEDIA, ALTA) integrado en el software asegura que las incidencias críticas que amenazan la seguridad de la comunidad (como cables expuestos o focos infecciosos) sean atendidas con urgencia. Al mejorar la velocidad de respuesta y la transparencia en la resolución de problemas, se fortalece la confianza institucional y se promueve una cultura de cuidado compartido del espacio público. En última instancia, ReportaYA contribuye a crear entornos urbanos más limpios y seguros, donde la tecnología sirve como puente para mejorar el bienestar social y la convivencia armónica en las ciudades.

1.3 Objetivos de diseño

El objetivo principal del diseño de **ReportaYA** es proporcionar una solución tecnológica robusta, escalable y mantenible que digitalice y optimice el ciclo de vida completo de la gestión de incidencias urbanas. El diseño aborda la necesidad de conectar eficientemente a la ciudadanía con las autoridades municipales, eliminando barreras burocráticas y tiempos muertos mediante una arquitectura orientada a objetos.

El alcance del diseño abarca dos frentes críticos:

1. **La experiencia del ciudadano:** Garantizar un sistema de captura de datos intuitivo y preciso. Para lograr esto, el diseño implementa validaciones automáticas de geolocalización y formularios dinámicos que aseguran la integridad de la información (coordenadas GPS) antes de que llegue al servidor, minimizando reportes falsos o incompletos.
2. **La eficiencia operativa municipal:** Proveer herramientas de gestión avanzadas. El diseño incorpora patrones como **State** (implícito en el manejo de enums y flujos) y **Service Layer** para administrar transiciones de estado complejas (traje, asignación, resolución y auditoría). Esto asegura que cada incidencia siga un flujo de trabajo estricto y auditable, permitiendo la asignación inteligente de técnicos basada en disponibilidad y especialidad.

Técnicamente, el diseño busca el desacoplamiento mediante principios **SOLID**, facilitando la futura incorporación de nuevos canales de notificación o cambios en el motor de base de datos sin afectar la lógica de negocio, asegurando así la longevidad del software.

1.4 Referencias

El presente documento se elaboró con base en la información y artefactos generados durante el Sprint 2 del proyecto ReportaYA, siguiendo el marco de trabajo Scrum y las plantillas oficiales del curso Ingeniería de Software II de la Universidad de Lima.

Las principales referencias utilizadas fueron:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- **Informe del Sprint 2 de Scrum – Grupo 6**, que documenta la planificación, ejecución y resultados del primer sprint del proyecto ReportaYA.
- **Historias de usuario y criterios de aceptación** descritos en el sprint, los cuales sirvieron como base para el diseño técnico y la elaboración de los diagramas UML.
- **Diseño estructural y principios SOLID** definidos en este mismo documento, que orientan la implementación del backend mediante una arquitectura en capas.
- **Documentación técnica de Spring Boot, Spring Data JPA y PostgreSQL**, tecnologías utilizadas en la construcción del sistema.
- **Herramientas de modelado UML**, como *Draw.io*, empleadas para representar gráficamente la estructura y comportamiento del software.

Estas referencias reflejan el trabajo realizado en el Sprint 2 y respaldan las decisiones de diseño y arquitectura aplicadas en el desarrollo del sistema.

1.5 Definiciones, siglas y abreviaturas

Término / Sigla	Definición
ACID	<i>Atomicity, Consistency, Isolation, Durability</i> . Conjunto de propiedades que garantizan que las transacciones en la base de datos se procesen de manera confiable (vital para el proceso de asignación y cierre).
API	<i>Application Programming Interface</i> . Conjunto de reglas que permite que la aplicación móvil se comuniquen con el backend.
Controller	Componente de la capa de presentación que recibe las peticiones HTTP y coordina la respuesta hacia el cliente móvil.
CRUD	<i>Create, Read, Update, Delete</i> . Acrónimo que refiere a las operaciones básicas de almacenamiento de datos.
DTO	<i>Data Transfer Object</i> . Objeto utilizado para transportar datos entre procesos, reduciendo la cantidad de llamadas y ocultando la estructura interna de la base de datos.
Entity	Clase Java que representa una tabla en la base de datos relacional (mapeo ORM).

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Factory Pattern	Patrón de diseño creacional utilizado para instanciar diferentes tipos de cuentas (Ciudadano, Técnico, Operador) sin acoplar el código a clases concretas.
FCM	<i>Firestore Cloud Messaging</i> . Solución multiplataforma de mensajería utilizada para el envío de notificaciones push a los dispositivos móviles.
JPA	<i>Java Persistence API</i> . Especificación de Java para acceder, persistir y gestionar datos entre objetos Java y una base de datos relacional.
JWT	<i>JSON Web Token</i> . Estándar para la creación de tokens de acceso que permiten la autenticación y autorización segura entre el cliente y el servidor.
Lazy Loading	Estrategia de carga diferida donde los datos relacionados (ej. lista de reportes de un usuario) no se cargan de la base de datos hasta que son explícitamente solicitados.
ODS	<i>Objetivos de Desarrollo Sostenible</i> . Metas globales adoptadas por las Naciones Unidas; el proyecto se alinea con los ODS 9 y 11.
ORM	<i>Object-Relational Mapping</i> . Técnica que permite convertir datos entre el sistema de tipos utilizado en el lenguaje orientado a objetos y la base de datos relacional.
RBAC	<i>Role-Based Access Control</i> . Mecanismo de control de acceso que restringe la disponibilidad de los recursos del sistema según el rol del usuario (Operador vs. Técnico).
Repository	Patrón que abstrae la lógica de acceso a datos, permitiendo que la capa de negocio permanezca agnóstica al tipo de base de datos.
REST	<i>Representational State Transfer</i> . Estilo de arquitectura de software utilizado para el diseño de servicios web y APIs.
Service	Componente de la capa de aplicación que encapsula la lógica de negocio, validaciones complejas y reglas del flujo de trabajo.
SOLID	Acrónimo de los cinco principios básicos de la programación orientada a objetos y diseño de software (Single responsibility, Open-closed, Liskov substitution, Interface segregation, Dependency inversion).

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Transaction	Secuencia de operaciones realizadas como una sola unidad lógica de trabajo. Si una operación falla, todas fallan (Rollback), asegurando la integridad de los datos.
Triaje	Proceso de negocio mediante el cual un Operador Municipal evalúa, clasifica y asigna un reporte a un técnico especializado.
Workflow	Flujo de trabajo que representa el ciclo de vida de una incidencia a través de sus distintos estados (Pendiente, Revisión, Proceso, Resuelta, Cerrada).

2. Consideraciones Éticas y el Impacto Global de la Solución

2.1 Responsabilidades Éticas y Profesionales

• Identificación de Dilemas Éticos

Durante el desarrollo de ReportaYA surgieron dilemas éticos relacionados con el manejo de información sensible. El uso de geolocalización, fotografías y datos personales plantea riesgos de privacidad, especialmente si la información es almacenada sin criterios claros o accedida por personal no autorizado. También existe la posibilidad de que la plataforma sea usada para reportes falsos o malintencionados, o que ciertas zonas reciban mayor atención debido al volumen de reportes en lugar de la gravedad real de las incidencias. Estos dilemas obligan a evaluar constantemente qué datos son necesarios, cómo se protegen y cómo se previenen usos indebidos.

• Adherencia a Códigos de Conducta Profesional

Las decisiones tomadas en el proyecto se guiaron por principios de ingeniería profesional y códigos como ACM/IEEE, priorizando integridad, transparencia y protección de la privacidad. Esto implica comunicar claramente los fines del uso de datos, evitar sesgos en los flujos de priorización, garantizar la calidad del software y mantener una postura honesta frente a las limitaciones del sistema. Asimismo, se aplicaron criterios de “no causar daño”, respeto por la dignidad humana y promoción de soluciones inclusivas.

• Compromiso con la Responsabilidad Profesional

La responsabilidad profesional va más allá de que el sistema funcione: significa asegurar que los ciudadanos interactúen con la app de manera segura, comprensible y confiable. Eso incluye validar cuidadosamente los formularios, prevenir errores comunes, diseñar interfaces claras y mantener el sistema actualizado. También implica responder

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

oportunamente ante fallas, garantizar soporte para el personal municipal y comunicar resultados o estadísticas sin manipular ni exagerar su impacto.

2.2 Impacto Social de la Solución

• Inclusión y Accesibilidad

ReportaYA está diseñada para que cualquier vecino —independientemente de su nivel de alfabetización digital— pueda reportar una incidencia sin barreras técnicas. Para ello se priorizaron pantallas intuitivas, iconografía clara y textos comprensibles. También se consideraron principios de accesibilidad, como compatibilidad con lectores de pantalla, contraste adecuado y tamaños de letra legibles. La aplicación complementa otros canales tradicionales para no excluir a quienes no manejan smartphones.

• Equidad y Justicia Social

El sistema promueve la participación equitativa, permitiendo visibilizar problemas en zonas históricamente desatendidas. Sin embargo, se reconoce el riesgo de que los sectores con mayor acceso tecnológico generen más reportes, aumentando la disparidad. Para evitarlo, las municipalidades deben priorizar incidencias según severidad, impacto y urgencia, no solo según la frecuencia de reportes. Esto ayuda a que todas las zonas reciban atención justa y proporcional.

• Gestión de la Privacidad y Seguridad de Datos

El manejo ético de la información es esencial. Se aplica el principio de minimización de datos, almacenando solo lo indispensable. La información sensible (GPS, fotos, datos personales) se protege con cifrado, controles de acceso y políticas de retención limitada. Se comunica al ciudadano qué datos se recolectan, por qué y por cuánto tiempo, reforzando la confianza en el sistema.

• Influencia en Comportamientos y Cultura

ReportaYA puede fortalecer la cultura de participación ciudadana y responsabilidad compartida, promoviendo el cuidado del entorno urbano. No obstante, también puede motivar denuncias impulsivas o la exposición innecesaria de terceros. Por ello, es recomendable acompañar la implementación con campañas de uso responsable, buenas prácticas y pautas de convivencia ciudadana.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

2.3 Impacto Económico de la Solución

• Análisis de Costos y Beneficios

El proyecto requiere inversión inicial (desarrollo, pruebas, servidores, capacitación y difusión) y costos de mantenimiento (soporte, hosting, actualizaciones). Sin embargo, los beneficios superan estos gastos: reducción de tiempos de atención, eliminación de registros manuales, planificación más eficiente y mejor asignación de recursos municipales. A mediano plazo, esto puede generar ahorros significativos y mejorar la gestión pública.

• Efectos en el Empleo y el Mercado

ReportaYA no busca reemplazar personal, sino potenciar su trabajo. Automatizar tareas repetitivas libera tiempo para actividades de mayor valor, incrementando la productividad. A su vez, genera nuevas oportunidades laborales en áreas como soporte técnico, análisis de datos y gestión digital municipal. La solución puede impulsar el ecosistema de innovación cívica y motivar alianzas con empresas tecnológicas o startups.

• Viabilidad Comercial y Sostenibilidad Financiera

La arquitectura escalable permite adaptar la solución a distintos distritos sin rehacer el sistema. Esto facilita la expansión y reduce costos de implementación. Además, el uso de componentes de software libre y servicios eficientes puede disminuir licenciamientos costosos. La alineación con los Objetivos de Desarrollo Sostenible (ODS) también abre puertas a fondos internacionales y programas de innovación pública.

2.4 Impacto Ambiental de la Solución

• Eficiencia Energética y Consumo de Recursos

Aunque la solución es digital, implica consumo energético tanto en servidores como en dispositivos móviles. Por ello, se optimizó el uso de datos, el procesamiento local y la comunicación con el backend para minimizar el impacto. También se pueden elegir proveedores de nube con políticas de eficiencia energética y energías renovables.

• Minimización de la Huella Ecológica

La digitalización reduce el uso de papel, elimina trámites presenciales innecesarios y permite planificar rutas más eficientes para cuadrillas municipales, reduciendo combustible y emisiones. Además, al mejorar la gestión de residuos, puntos críticos o fallas urbanas, contribuye directamente a un entorno más limpio y saludable.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

• Consideraciones sobre Obsolescencia Tecnológica

El diseño busca evitar que el ciudadano necesite dispositivos costosos o recientes. La app funciona en versiones moderadas de Android y iOS, y su arquitectura modular facilita actualizaciones sin reconstruir el sistema ni forzar cambios de dispositivos. Esto reduce la obsolescencia y hace la solución más inclusiva.

2.5 Impacto Global de la Solución

• Cumplimiento de Regulaciones y Normativas Internacionales

Aunque la implementación es local, se adoptan buenas prácticas globales en protección de datos (GDPR como referencia), accesibilidad (WCAG) y gobierno digital. Estas guías fortalecen la legitimidad del proyecto y permiten que la solución se escale o integre con plataformas regionales orientadas a ciudades inteligentes.

• Adaptabilidad Cultural y Diversidad

ReportaYA es adaptable a distintos contextos culturales, lingüísticos y sociales. Las categorías de incidencias, mensajes y flujos pueden configurarse según la realidad de cada distrito. Incorporar procesos de co-diseño con vecinos ayuda a evitar sesgos, respetar la diversidad y generar mayor aceptación.

• Aplicabilidad en Contextos de Recursos Limitados

El sistema contempla escenarios de baja conectividad, permitiendo guardar reportes offline y enviarlos cuando el dispositivo recupere señal. También funciona en dispositivos de gama media y puede integrarse con canales alternativos para ciudadanos sin acceso a smartphones. Esto amplía su alcance más allá de grandes ciudades.

2.6 Estrategias de Mitigación y Diseño Consciente

• Propuestas para Mitigar Impactos Negativos

Entre las estrategias recomendadas están:

- políticas de retención limitada de datos,
- cifrado de extremo a extremo,

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- revisión de reportes sospechosos,
- difuminado automático de rostros o placas,
- validación básica de cuentas para prevenir abuso,
- monitoreo de indicadores de equidad en la atención.

Estas medidas reducen riesgos asociados a privacidad, discriminación o uso malintencionado del sistema.

• **Decisiones de Diseño Orientadas a la Ética y Sostenibilidad**

Las decisiones técnicas se basaron en principios de privacidad desde el diseño, seguridad desde el diseño y diseño universal. Esto implica no pedir información innecesaria, garantizar interfaces claras, mostrar mensajes transparentes y priorizar tecnologías de bajo consumo de recursos. Así, la solución se mantiene responsable y sostenible en el tiempo.

• **Argumentación y Justificación de Enfoques Adoptados**

Los enfoques adoptados se justifican porque fortalecen la confianza ciudadana, protegen derechos fundamentales y aumentan las probabilidades de éxito y continuidad del proyecto. Optar por prácticas éticas asegura un servicio responsable, transparente y respetuoso. Además, la sostenibilidad técnica y la escalabilidad permiten que ReportaYA evolucione sin comprometer la seguridad ni la calidad del sistema, alineándose con los ODS 9 y 11.

3. Visión general del diseño

3.1 Introducción

El diseño del sistema ReportaYA sigue un enfoque orientado a objetos, utilizando una arquitectura cliente-servidor. La aplicación móvil (cliente) se comunica con un backend RESTful, el cual gestiona la lógica de negocio y la persistencia de datos. Para el modelado de clases y diagramas UML, se emplearon herramientas como Draw.io. El diseño abarca diagramas estructurales (clases, componentes) y dinámicos (secuencia), además de un modelo de base de datos relacional.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

3.2 Vista lógica de la arquitectura del Software

Arquitectura del Sprint 1: Gestión de Usuarios y Autenticación

En la fase inicial, la arquitectura se centró en establecer los cimientos del sistema. El diseño lógico se basó en una comunicación Cliente-Servidor simple.

- Cliente: Aplicación móvil (React Native) encargada de la interfaz de login y registro.
- Backend: Un monolito modular construido con Spring Boot que expone una API REST.
- Módulos Activos: El foco principal fue el AuthService y CuentaService. Se implementó el patrón Abstract Factory para la creación de cuentas, permitiendo diferenciar desde el inicio las entidades Ciudadano, Técnico y Operador Municipal sin acoplar el sistema.
- Base de Datos: Modelo relacional inicial centrado en las tablas cuentas y personas.

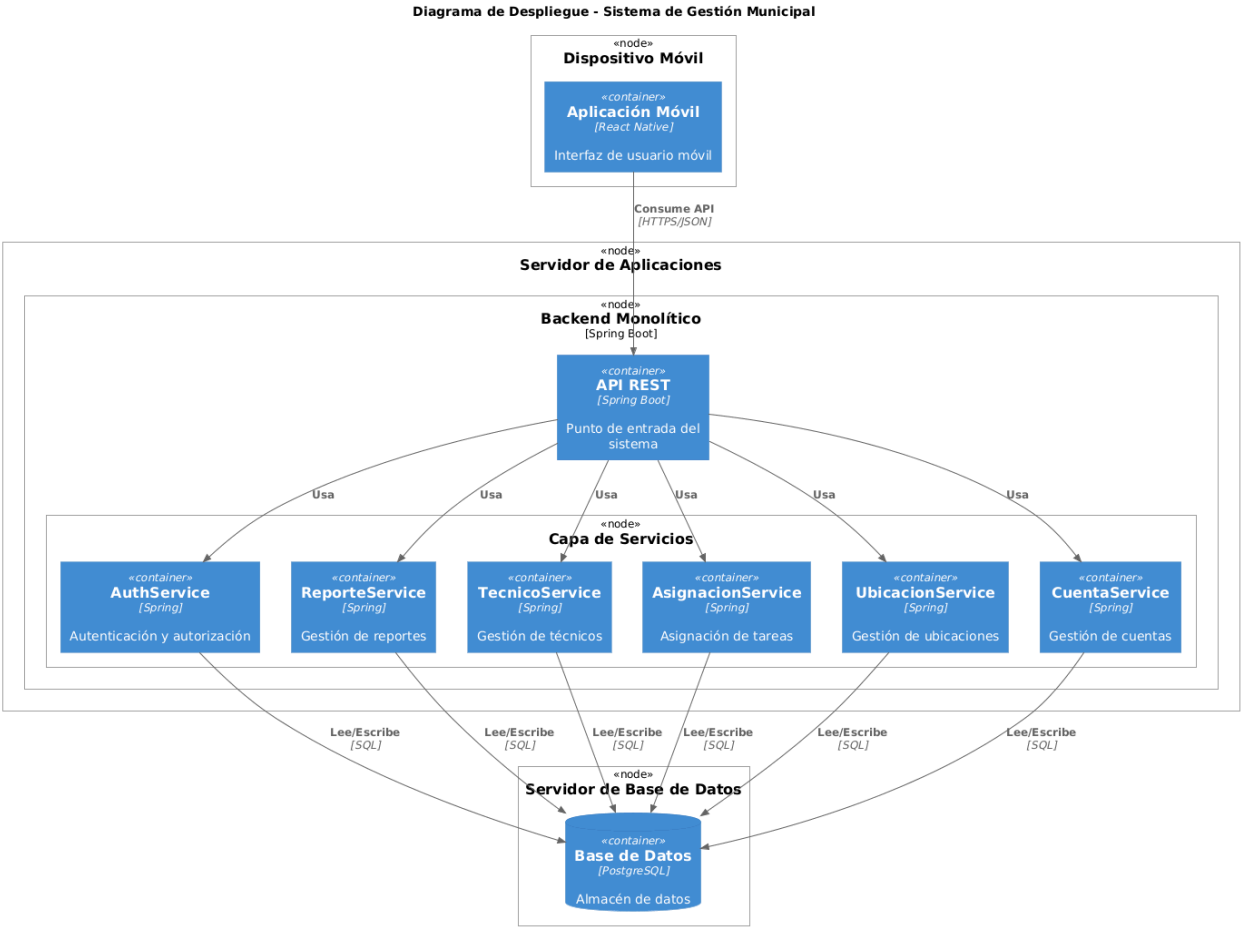
Arquitectura del Sprint 2: Gestión de Reportes, Triage e Integraciones

Para el Sprint 2, la arquitectura evolucionó significativamente para soportar la lógica de negocio central. El diagrama de despliegue y componentes refleja una Arquitectura en Capas (Layered Architecture) estricta, diseñada para manejar flujos complejos de estados y geolocalización.

- Evolución de Componentes:
 - Se integraron nuevos controladores y servicios (ReporteController, AsignacionController, UbicacionService) que orquestan el flujo de trabajo del reporte (PENDIENTE → REVISIÓN → PROCESO → RESUELTA).
 - Se implementó el patrón Repository para abstraer la persistencia de datos complejos, incluyendo consultas con JOIN FETCH para optimizar la carga de ubicaciones y evidencias.
- Integración de Servicios Externos: La arquitectura ahora incorpora adaptadores para comunicarse con servicios de terceros:
 - Google Maps API / Leaflet: Para la georreferenciación y validación de coordenadas en el módulo de ubicación.
 - Firebase Cloud Messaging (FCM): Se añadió una capa de notificaciones push desacoplada (NotificationService) para alertar a los usuarios sobre cambios de estado en tiempo real.

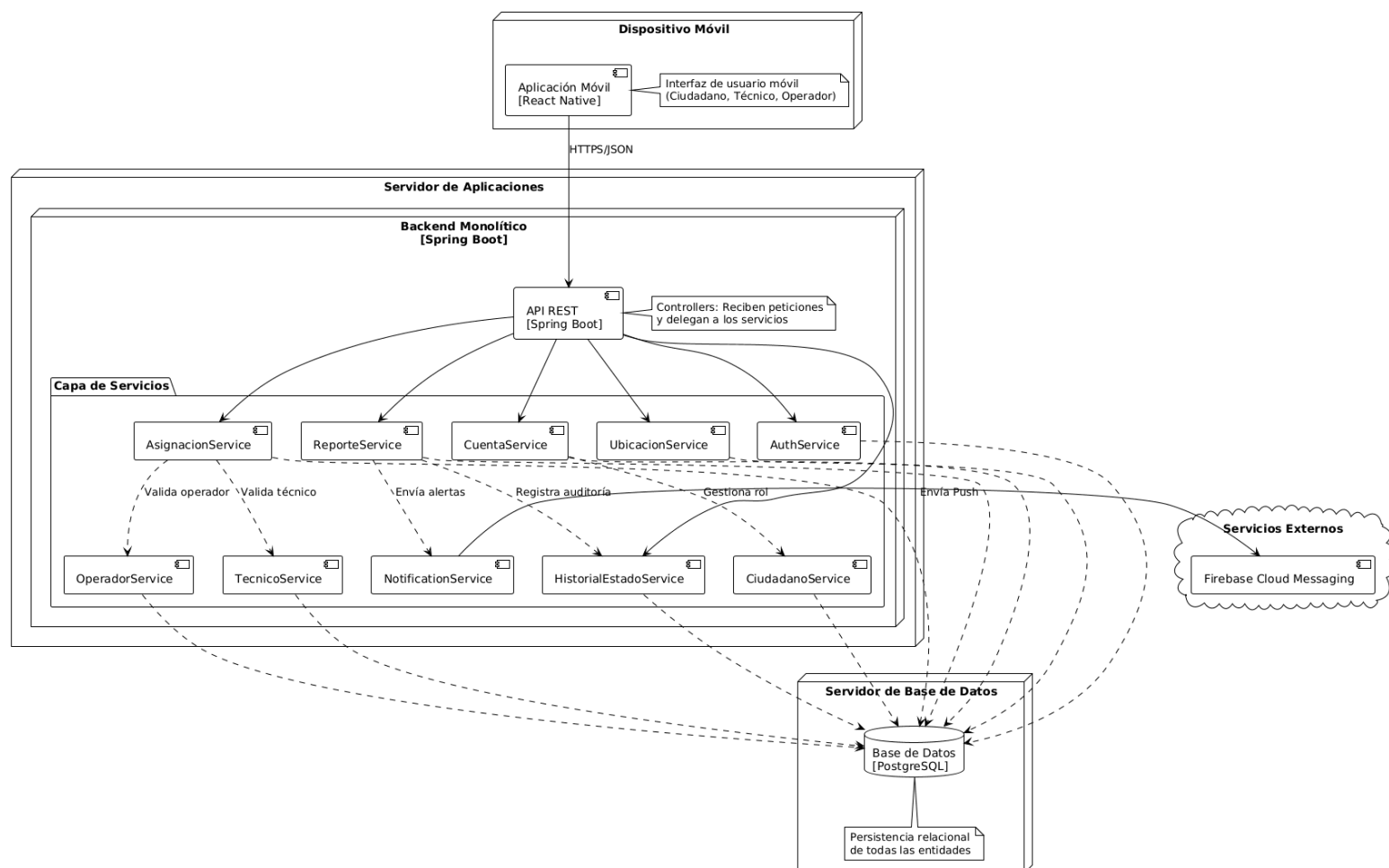
Arquitectura de software del Sprint 1:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Arquitectura de software del Sprint 2:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



3.3 Prácticas y metodología de Desarrollo

El desarrollo del sistema ReportaYA se basa en principios y prácticas de ingeniería de software que garantizan calidad, mantenibilidad y escalabilidad del código. En este segundo sprint, se ha puesto especial énfasis en la integridad de los datos y la cohesión de los nuevos flujos de negocio.

3.3.1 Principios SOLID Aplicados

El diseño del sistema está fundamentado en los cinco principios SOLID, reforzando su aplicación en los nuevos módulos de gestión:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Single Responsibility Principle (SRP): Cada clase tiene una única razón para cambiar. Los Controllers solo manejan HTTP, los Repositories gestionan persistencia y los DTOs transportan datos. En la Capa de Aplicación, se ha separado la lógica de triaje en AsignacionService y la de alertas en NotificationService, evitando que ReporteService se convierta en una clase monolítica con múltiples responsabilidades.
- Open/Closed Principle (OCP): El sistema está abierto a extensión pero cerrado a modificación. La jerarquía de Cuenta permite agregar nuevos roles sin modificar código existente. Asimismo, la implementación de servicios de notificación permite integrar nuevos canales (como SMS o Email) en el futuro sin alterar la lógica central de los reportes.
- Liskov Substitution Principle (LSP): Las subclases Ciudadano, Tecnico y OperadorMunicipal pueden sustituir a su clase base Cuenta sin romper la funcionalidad del sistema. El polimorfismo funciona correctamente en todas las operaciones de autenticación y asociación de datos.
- Interface Segregation Principle (ISP): Se utilizan interfaces específicas por contexto (ICiudadanoService, ITecnicoService, IOperadorService) en lugar de una única interfaz monolítica. De esta forma, los controladores de la API móvil del técnico no dependen de métodos administrativos que solo utiliza el operador.
- Dependency Inversion Principle (DIP): Las capas de alto nivel (Controllers, Services) dependen de abstracciones (interfaces) en lugar de implementaciones concretas. Spring Framework gestiona la inyección de dependencias automáticamente, facilitando el desacoplamiento entre la lógica de negocio y la persistencia de datos.

3.3.2 Patrones de Diseño Implementados

Además de los patrones base, se han incorporado patrones estructurales y de comportamiento para soportar la complejidad del Sprint 2:

- Repository Pattern: Abstrae el acceso a datos mediante interfaces Spring Data JPA, eliminando código repetitivo y facilitando el testing mediante mocks.
- Abstract Factory Pattern: Centraliza la creación de objetos de la jerarquía Cuenta mediante factories especializadas, permitiendo extensibilidad sin modificar código cliente.
- Service Layer Pattern: Encapsula la lógica de negocio y define la frontera de las transacciones. Coordina la respuesta de múltiples repositorios (ej. actualizar reporte, crear asignación, guardar historial) en una sola operación atómica.
- Transactional Pattern: Implementado mediante la anotación @Transactional. Garantiza las propiedades ACID en operaciones críticas como el triaje y el cierre de reportes, asegurando que no queden datos inconsistentes si ocurre un error a mitad del proceso.
- Data Transfer Object (DTO): Utilizado extensivamente para desacoplar la capa de presentación del modelo de dominio, especialmente crítico en este sprint para manejar la transferencia de evidencias fotográficas (Base64) y datos anidados de ubicación sin exponer la estructura de la base de datos.

3.3.3 Arquitectura en Capas

El sistema utiliza una arquitectura en capas con separación clara de responsabilidades, integrando ahora servicios externos:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Capa de Presentación: Controllers REST que exponen endpoints HTTP, validan la entrada de datos y gestionan la seguridad por roles.
- Capa de Aplicación: Services que implementan la lógica de negocio, validaciones de estado (workflow del reporte) y orquestación de transacciones.
- Capa de Dominio: Entidades JPA que representan el modelo de negocio y sus relaciones (Herencia, Polimorfismo).
- Capa de Persistencia: Repositories que abstraen el acceso a la base de datos PostgreSQL.
- Capa de Integración: Componentes encargados de la comunicación con servicios externos, específicamente el adaptador para Firebase Cloud Messaging (Notificaciones).
- Capa de Transferencia: DTOs que transportan datos optimizados entre el cliente móvil y el servidor.

3.4 Diseño Integral

3.4.1 Salud Pública y Seguridad Ciudadana

El sistema está diseñado para priorizar y acelerar la resolución de incidencias que amenazan la salud y seguridad de la comunidad.

- **Mecanismo de Triage** : La arquitectura permite la clasificación de reportes por niveles de prioridad (BAJA, MEDIA, ALTA). Esto asegura que situaciones críticas, como focos infecciosos por acumulación de residuos (Riesgo de Salud) o cables eléctricos expuestos (Riesgo de Seguridad), sean atendidas con urgencia sobre incidencias estéticas.
- **Georreferenciación Precisa**: La integración de mapas y ubicación GPS garantiza que los equipos técnicos lleguen al punto exacto del problema sin demoras, optimizando el tiempo de respuesta ante emergencias urbanas.

3.4.2 Seguridad del Sistema y Privacidad de Datos

El diseño incorpora medidas robustas para proteger la integridad de la información y la privacidad de los usuarios, conforme el sistema escala en complejidad.

- **Control de Acceso Basado en Roles (RBAC)**: Se ha implementado una estricta segregación de interfaces y endpoints. El diseño garantiza que datos sensibles del ciudadano solo sean visibles para el personal municipal autorizado, y que los técnicos de campo solo accedan a la información operativa necesaria para su labor, minimizando vectores de exposición de datos.
- **Integridad de Evidencias**: El almacenamiento de fotografías mediante codificación segura y su asociación inmutable a los reportes (transaccionalidad ACID) previene la manipulación de pruebas, garantizando la transparencia en la gestión pública.

3.4.3 Factores Sociales y Culturales

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

El software se ha diseñado considerando la diversidad de actores y la necesidad de restaurar la confianza institucional.

- **Inclusión Digital:** Las interfaces móviles (tanto para el ciudadano como para el técnico) priorizan la simplicidad y el uso de elementos visuales claros, facilitando su uso por personas con diferentes niveles de alfabetización digital.
- **Transparencia y Auditoría:** El módulo de auditoría implementado en el Sprint 2, donde un operador debe validar la solución del técnico, introduce un mecanismo de control de calidad. Esto fomenta una cultura de responsabilidad profesional y asegura al ciudadano que su reporte no será cerrado arbitrariamente sin una solución real.
- **Comunicación Proactiva:** El sistema de notificaciones (NotificationService) mantiene al ciudadano informado en cada etapa, reduciendo la incertidumbre y fortaleciendo el vínculo de confianza entre la comunidad y el municipio.

3.4.4 Impacto Ambiental

El diseño promueve prácticas sostenibles mediante la digitalización y optimización de procesos.

- **Desmaterialización de Procesos:** La transición de formularios y partes de trabajo en papel a un flujo 100% digital (evidencias fotográficas y asignaciones en aplicativo móvil) reduce significativamente el consumo de papel y la generación de residuos administrativos.
- **Eficiencia Operativa:** Al permitir que los técnicos reciban asignaciones directamente en sus dispositivos móviles sin necesidad de retornar a la central, el sistema contribuye a la reducción de desplazamientos innecesarios, disminuyendo indirectamente la huella de carbono de la flota municipal.

3.4.5 Factores Económicos

La arquitectura modular y escalable responde a la necesidad de eficiencia en el gasto público.

- **Optimización de Recursos:** El algoritmo de asignación permite distribuir la carga de trabajo de manera equitativa y eficiente entre los técnicos disponibles, maximizando la productividad de la fuerza laboral municipal.
- **Prevención de Costos Mayores:** Al facilitar la detección temprana y el seguimiento de incidencias menores, el sistema ayuda a prevenir el deterioro grave de la infraestructura urbana, lo que a largo plazo representa un ahorro significativo en costos de reparación mayor para la administración pública.

4. Diseño Estructural

4.1 Explicación y justificación del diseño

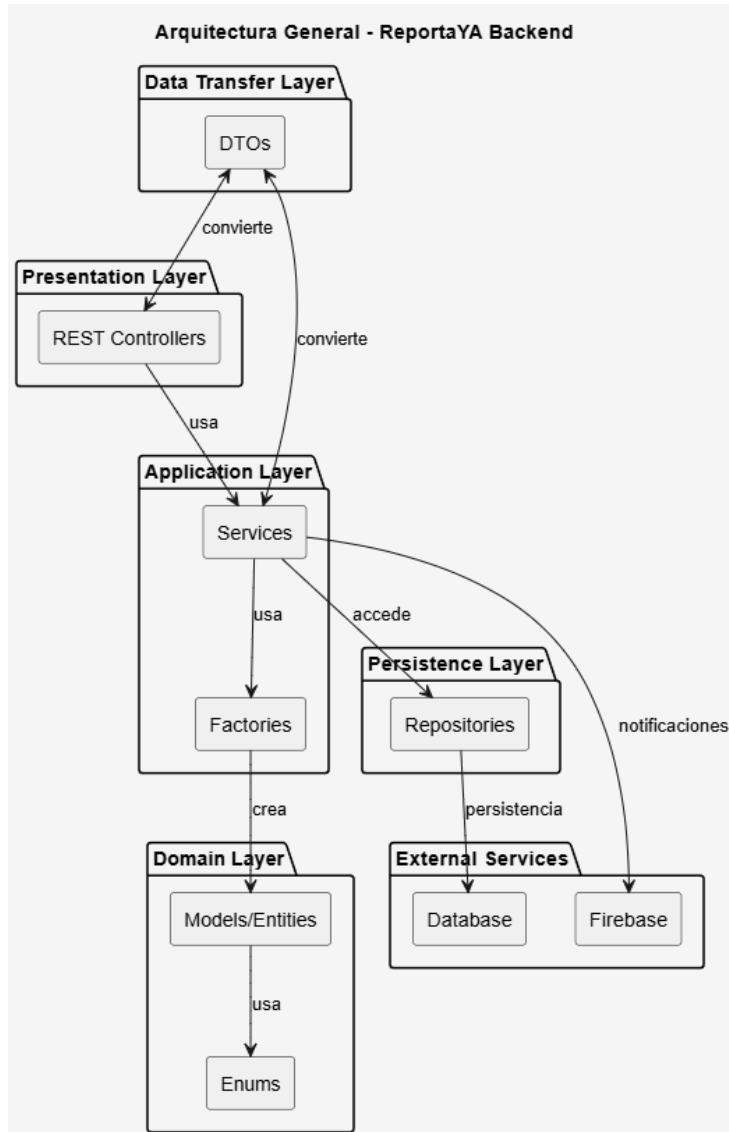
El backend de ReportaYA implementa una arquitectura en capas (Layered Architecture) con separación clara de responsabilidades siguiendo los principios SOLID. Esta estructura garantiza:

- **Mantenibilidad:** Cambios en una capa no afectan otras
- **Testabilidad:** Cada componente puede probarse independientemente

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Escalabilidad: Fácil integración de nuevas funcionalidades
- Cohesión: Responsabilidades bien definidas

Diagrama General de Arquitectura



Justificación de Decisiones por Sprint

Sprint 1: Autenticación y Gestión de Cuentas

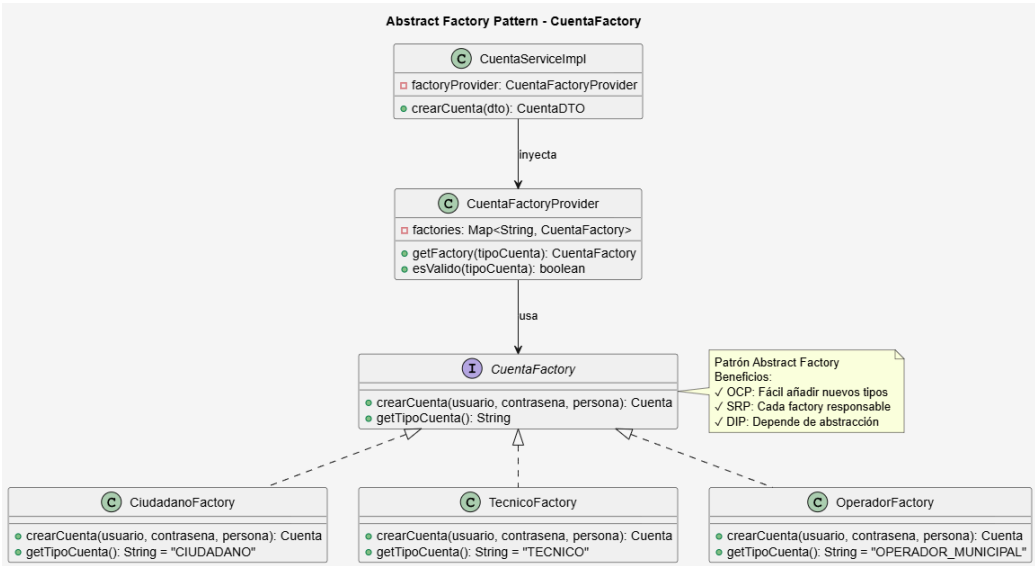
Decisión: Abstract Factory Pattern para Creación de Cuentas

Problema Identificado: Necesitábamos crear 3 tipos de cuentas (Ciudadano, Técnico, OperadorMunicipal) con diferentes comportamientos, pero evitando condicionales complejos en el código.

Solución Implementada:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Diagrama de Autenticación y Gestión de Cuentas



Justificación SOLID:

Principio	Cómo lo cumple	Beneficio
SRP	Cada factory solo crea un tipo de cuenta	Responsabilidad única y clara
OCP	Nuevos tipos sin modificar código existente	Fácil extensión futura
LSP	Todas las factories implementan la interfaz	Intercambiables
ISP	Interface CuentaFactory pequeña y enfocada	Solo lo necesario
DIP	CuentaServiceImpl depende de CuentaFactory (abstracción)	Desacoplado

Alternativas Rechazadas:

- Constructor con switch/if-else: Frágil, viola OCP
- Constructores específicos: Menos flexible, difícil mantener
- Reflexión: Overhead de performance

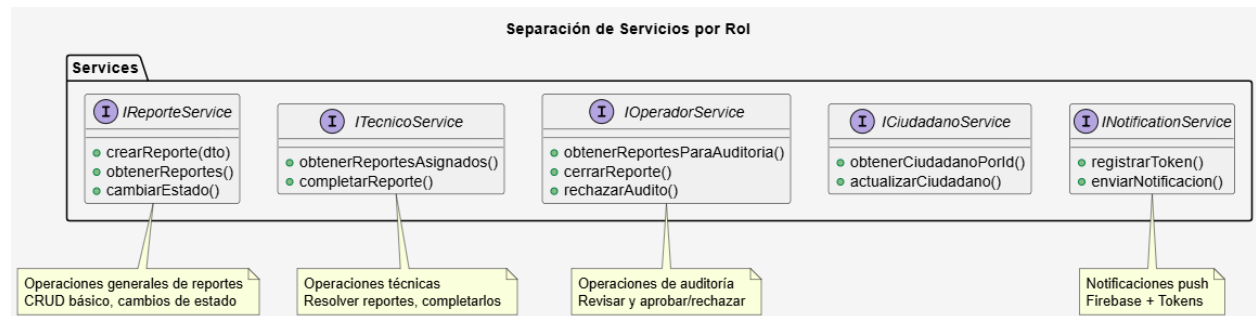
Sprint 2: Flujo de Reportes (Ciudadano → Técnico → Operador)

Decisión: Separación de Responsabilidades por Rol

Problema: Un único servicio de reportes con todas las operaciones sería difícil de mantener (SRP violado).

Solución:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Justificación:

- SRP: Cada servicio una responsabilidad clara
- ISP: Interfaces pequeñas con métodos relevantes
- DIP: Controllers inyectan interfaces, no implementaciones
- Testabilidad: Fácil crear mocks de cada servicio

Ejemplo: CuentaController

```

@RestController
@RequestMapping("/api/cuenta")
public class CuentaController {
    private final ICuentaService cuentaService;

    @Autowired
    public CuentaController(ICuentaService cuentaService) {
        this.cuentaService = cuentaService; // ✅ DIP
    }

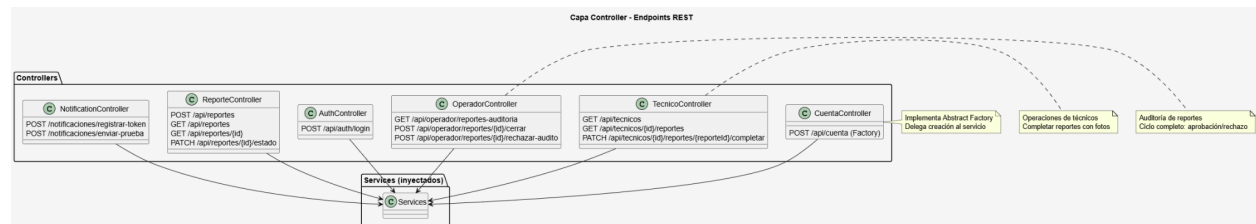
    @PostMapping
    public ResponseEntity<> crearCuenta(@Valid @RequestBody CuentaDTO cuentaDTO) {
        CuentaDTO resultado = cuentaService.crearCuenta(cuentaDTO);
        return ResponseEntity.status(HttpStatus.CREATED).body(resultado);
    }
}
  
```

Arquitectura por Capas

4.1.1 Capa Controller (Presentación)

Responsabilidad: Recibir solicitudes HTTP, validar entrada, delegar a servicios, retornar respuestas.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

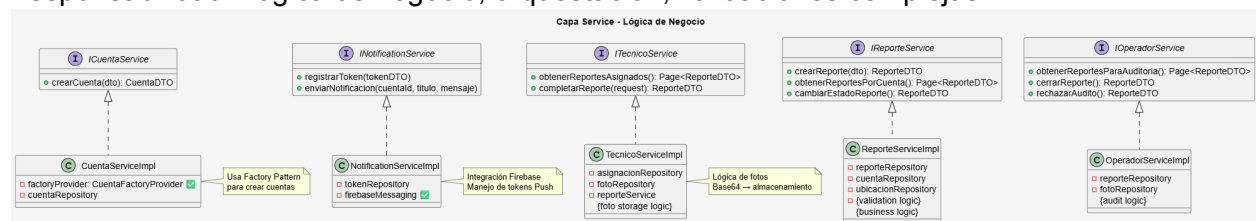


Patrones Utilizados:

- Inyección de Dependencias (Constructor)
- Validación de entrada (@Valid)
- CORS habilitado para móvil/frontend
- Manejo de excepciones con ResponseEntity

4.1.2 Capa Service (Aplicación)

Responsabilidad: Lógica de negocio, orquestación, validaciones complejas.

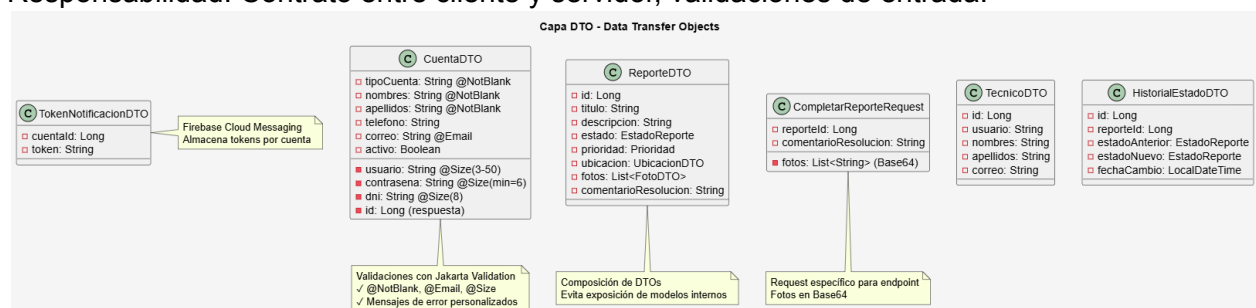


Decisiones Clave:

1. Transaccionalidad: @Transactional en operaciones que modifican datos
2. Validaciones: Lógica de negocio (no solo anotaciones)
3. Separación: Cada servicio una responsabilidad específica
4. DIP: Inyección de repositories (abstracciones)

4.1.3 Capa DTO (Transferencia de Datos)

Responsabilidad: Contrato entre cliente y servidor, validaciones de entrada.



Justificación:

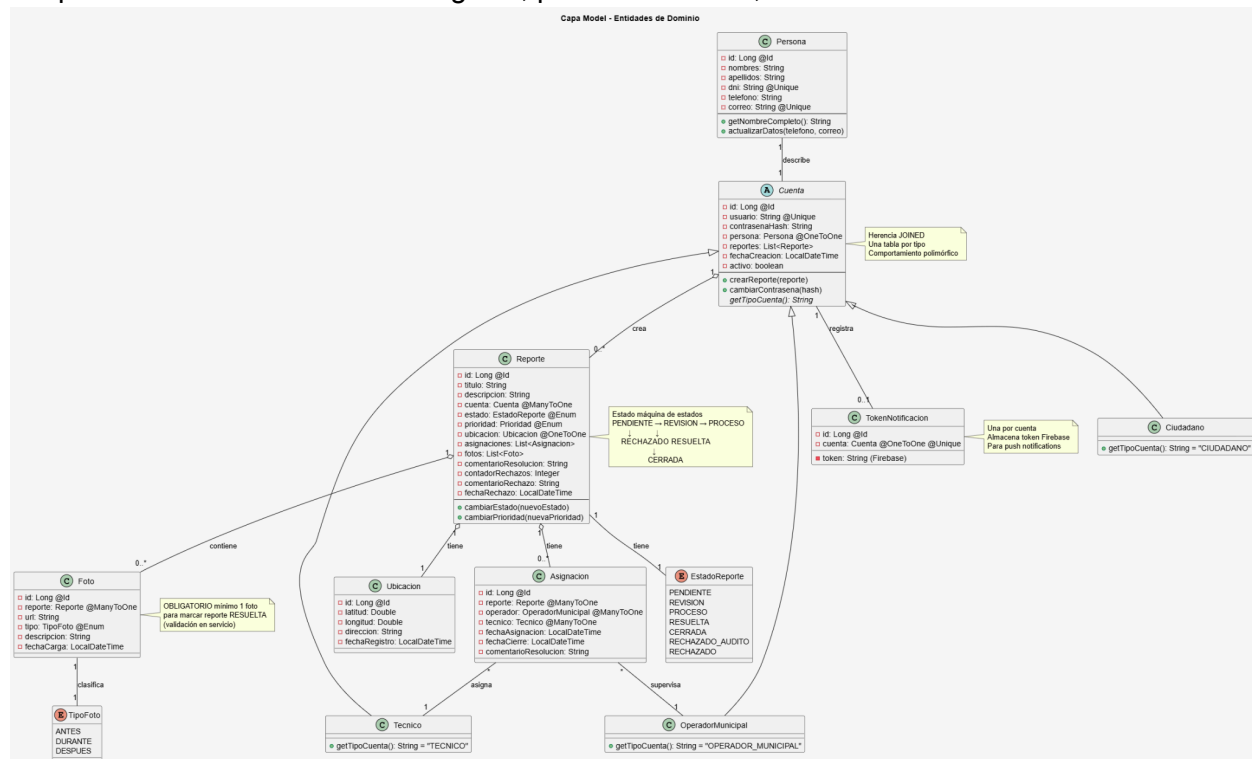
Aspecto	Razón
Separar DTO del Model	ISP: Cliente solo recibe datos necesarios

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Validaciones	SRP: Entrada validada en controller
Composición	DDD: DTOs anidados vs. todo plano
Request específicos	ISP: Cada endpoint su contrato

4.1.4 Capa Model (Dominio)

Responsabilidad: Entidades de negocio, persistencia JPA, relaciones.



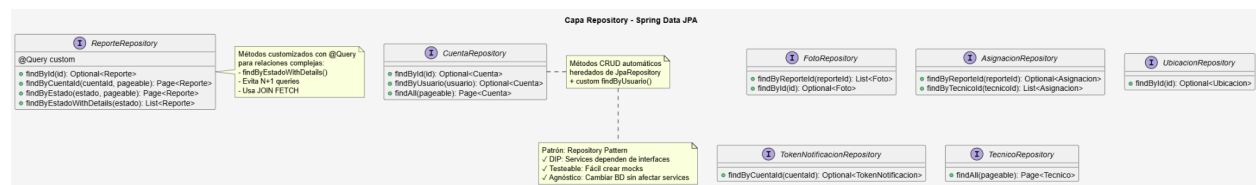
Decisiones Arquitectónicas:

- Herencia JOINED: Cada tipo de cuenta su tabla, con tabla padre Cuenta
 - Relaciones polimórficas (Reporte → Cuenta)
 - Eficiente en queries
 - Algo complejo en JPA
- Relaciones Lazy: fetch = FetchType.LAZY por defecto
 - Mejor performance
 - Cuidado con lazy initialization fuera de transacción
- Cascada: cascade = CascadeType.ALL en relaciones uno-a-muchos
 - Eliminar padre elimina hijos
 - Usar con cuidado

4.1.5 Capa Repository (Persistencia)

Responsabilidad: Acceso a datos, queries, abstraer JPA.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Métodos Clave:

```
// CuentaRepository
Optional<Cuenta> findByIdUsuario(String usuario);

// ReporteRepository
@Query("SELECT r FROM Reporte r " +
      "JOIN FETCH r.ubicacion " +
      "JOIN FETCH r.cuenta " +
      "WHERE r.estado = ?1")
List<Reporte> findByEstadoWithDetails(EstadoReporte estado);

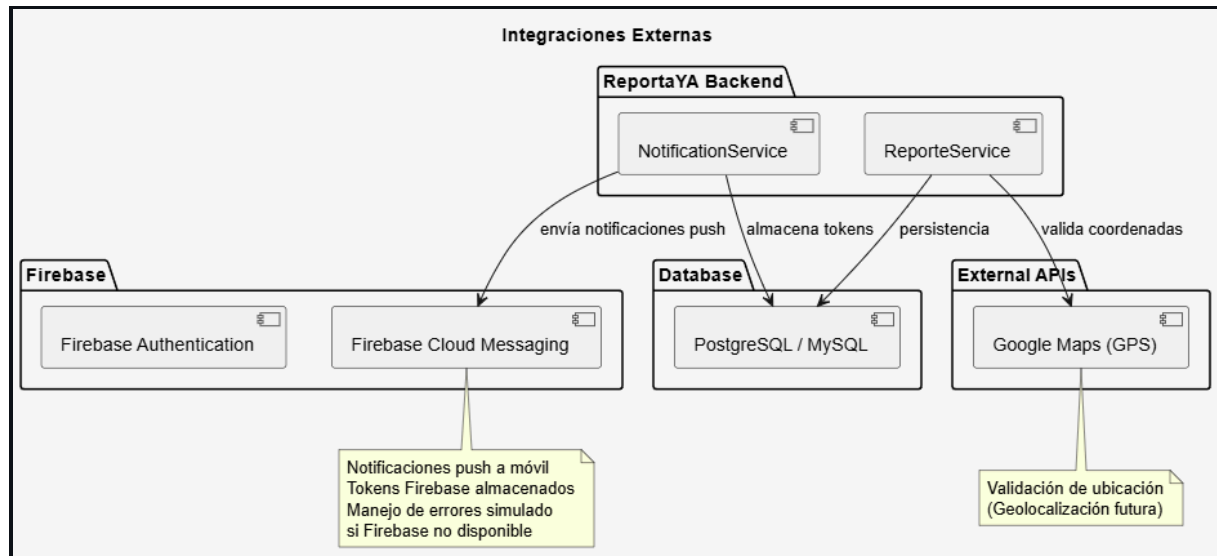
// TokenNotificacionRepository
Optional<TokenNotificacion> findByIdCuentaId(Long cuentaId);
```

Justificación:

- Repository Pattern: Abstracción de persistencia
- DIP: Services no conocen JPA
- JOIN FETCH: Evita problema N+1
- Testable: Mocks fáciles

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

4.1.6 Integraciones Externas



Patrones de Diseño Implementados

Abstract Factory Pattern

Ubicación: `com.ulima.incidenciaurbana.factory`

Implementación:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```
// Interfaz
public interface CuentaFactory {
    Cuenta crearCuenta(String usuario, String contrasenaHash, Persona persona);
    String getTipoCuenta();
}

// Implementaciones
public class CiudadanoFactory implements CuentaFactory {
    @Override
    public Cuenta crearCuenta(String usuario, String contrasenaHash, Persona persona) {
        return new Ciudadano(usuario, contrasenaHash, persona);
    }

    @Override
    public String getTipoCuenta() {
        return "CIUDADANO";
    }
}

// Provider
@Component
public class CuentaFactoryProvider {
    private final Map<String, CuentaFactory> factories = new HashMap<>();

    public CuentaFactory getFactory(String tipoCuenta) {
        return factories.get(tipoCuenta);
    }
}
```

Por qué:

- Cumple OCP: Nueva factory sin modificar código existente
- Cumple SRP: Cada factory su responsabilidad
- Desacoplamiento: Service no conoce implementaciones concretas

Repository Pattern

Ubicación: com.ulima.incidenciaurbana.repository

Beneficios:

- DIP: Servicios dependen de interfaces Repository
- Testeable: Fácil mockear repositories
- Agnóstico: Cambiar JPA por otro ORM sin afectar servicios

DTO Pattern

Ubicación: com.ulima.incidenciaurbana.dto

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Ventajas:

- ISP: Cliente solo recibe datos necesarios (no entidades complejas)
- Seguridad: No expone relaciones internas
- Validación: Anotaciones Jakarta Validation

Service Layer Pattern

Ubicación: com.ulima.incidenciaurbana.service

Propósito:

- Centralizar lógica de negocio
- Separar de controllers y repositories
- Reutilizable entre múltiples endpoints

Dependency Injection Pattern

Framework: Spring Boot (Constructor Injection)

```
@Service
public class TecnicoServiceImpl implements ITecnicoService {
    private final AsignacionRepository asignacionRepository;
    private final ReporteRepository reporteRepository;
    private final IReporteService reporteService;

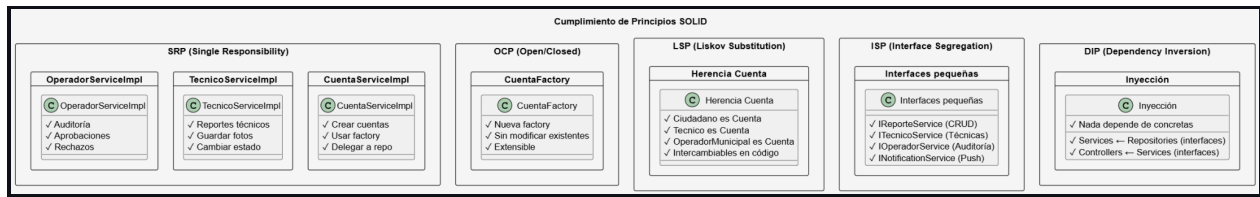
    @Autowired // ← Inyección explícita
    public TecnicoServiceImpl(
        AsignacionRepository asignacionRepository,
        ReporteRepository reporteRepository,
        IReporteService reporteService) {
        this.asignacionRepository = asignacionRepository;
        this.reporteRepository = reporteRepository;
        this.reporteService = reporteService;
    }
}
```

Beneficios SOLID:

- DIP: Inyecta abstracciones, no implementaciones
- SRP: Constructor declara dependencias explícitamente
- Testeable: Fácil pasar mocks

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Cumplimiento SOLID por Componente



Decisiones de Diseño Rechazadas

Monolitito Service (Todo en un servicio)

Por qué NO: Violaría SRP, difícil de mantener, sin separación de responsabilidades.

Constructor con Switch/If-else para Factory

```
// ✖ MAL
public Cuenta crearCuenta(String tipo, ...) {
    if ("CIUDADANO".equals(tipo)) {
        return new Ciudadano(...);
    } else if ("TECNICO".equals(tipo)) {
        return new Tecnico(...);
    }
    // Violá OCP: Cambio futuro requiere modificar esta clase
}
```

DTOs = Modelos (Exponer entidades JPA directamente)

Por qué NO:

- Cliente ve estructura interna
- Relaciones lazy crean problemas
- Seguridad: expone campos sensibles

Relaciones EAGER en Models

```
// ✖ MAL
@ManyToOne(fetch = FetchType.EAGER)
private Cuenta cuenta;
```

Por qué NO: Queries n+1, performance terrible, carga innecesaria.

sin Transactions (@Transactional)

Por qué NO: Inconsistencia de datos si falla guardado parcial.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

4.2 Diagrama de clases de diseño

Sprint 1

Diagrama de Clases del Sprint 1:

El diagrama de clases completo es extenso, por lo que se ha dividido en subsistemas lógicos funcionales porque Muestra flujo vertical completo (de HTTP a BD) y Facilita trazabilidad con casos de uso:

Subsistema 1: Gestión de Usuarios y Autenticación

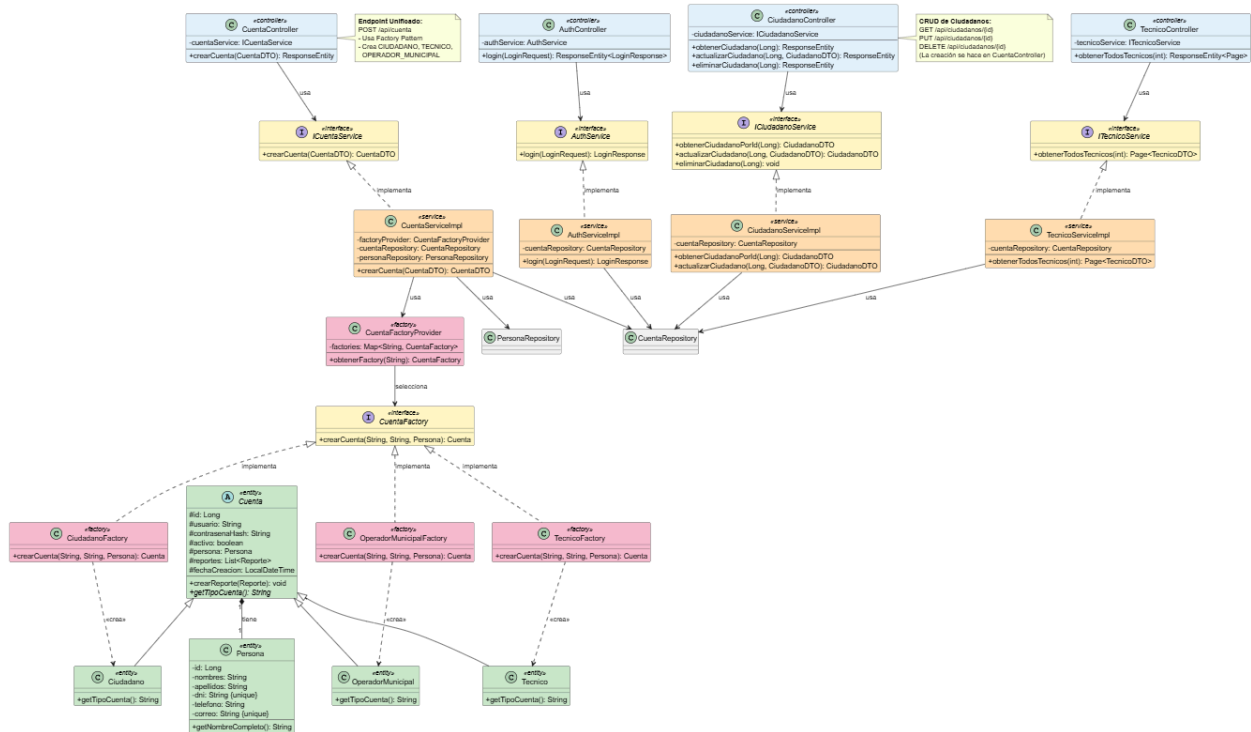
Subsistema 2: Gestión de Reportes

Subsistema 3: Gestión de Asignaciones (Triage)

Subsistema de Gestión de Usuarios

Figura 2: Diagrama de Clases - Gestión de Usuarios

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Explicación del Subsistema de Usuarios

Responsabilidades:

1. Autenticación de usuarios: Login con validación de credenciales (AuthController + AuthService)
2. Creación unificada de cuentas: Registro de cualquier tipo de cuenta mediante endpoint centralizado (CuentaController + Factory Pattern)
3. Gestión CRUD específica por rol:
 - Ciudadanos: Consulta, actualización y eliminación (CiudadanoController)
 - Técnicos: Listado paginado para asignaciones (TecnicoController)
4. Polimorfismo de cuentas: Manejo transparente de Ciudadano, Tecnico, OperadorMunicipal como subtipos de Cuenta

Componentes Principales:

Componente	Tipo	Responsabilidad
AuthController	Controller	Endpoint de login (POST /api/auth/login)
CuentaController	Controller	Endpoint unificado de creación (POST /api/cuenta)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

CiudadanoController	Controller	CRUD de ciudadanos (GET, PUT, DELETE)
TecnicoController	Controller	Listar técnicos disponibles (GET /api/tecnicos?page=n)
AuthService	Service	Validación de credenciales, generación de respuesta de login
ICuentaService	Service	Orquesta Factory Pattern, valida duplicados, persiste cuenta + persona
ICiudadanoService	Service	Lógica de negocio específica de ciudadanos
ITecnicoService	Service	Lógica de paginación y consulta de técnicos
CuentaFactoryProvider	Factory (Provider)	Selecciona factory según tipo de cuenta (CIUDADANO, TECNICO, OPERADOR_MUNICIPAL)
CuentaFactory	Interface	Define contrato de creación de cuentas
CiudadanoFactory, TecnicoFactory, OperadorMunicipalFactory	Factories	Implementaciones concretas que crean subtipos de Cuenta

Decisiones de Diseño:

1. Endpoint Unificado de Creación (CuentaController + Factory Pattern)

Decisión: Un solo endpoint POST /api/cuenta crea cualquier tipo de cuenta usando Abstract Factory.

Justificación:

- Single Responsibility: CuentaController solo coordina creación, no conoce detalles de cada tipo

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Open/Closed: Agregar nuevo rol (ej: "SUPERVISOR") solo requiere crear nueva factory, sin modificar controller
 - DRY (Don't Repeat Yourself): Validaciones comunes (usuario único, DNI único) en un solo lugar
 - Seguridad futura: Facilita implementar autenticación diferenciada (ciudadanos públicos, técnicos/operadores protegidos)
- Flujo de creación:

1. CuentaController recibe CuentaDTO con tipoCuenta ("CIUDADANO", "TECNICO", etc.)
2. CuentaServiceImpl usa CuentaFactoryProvider.obtenerFactory(tipoCuenta)
3. Factory apropiada crea instancia concreta (Ciudadano, Tecnico, OperadorMunicipal)
4. Service valida duplicados (usuario, DNI, correo) y persiste Cuenta + Persona

2. CRUD Segregado por Rol (CiudadanoController, TecnicoController)

Decisión: Controllers separados para operaciones específicas de cada rol.

Justificación:

- Interface Segregation Principle: Cada controller expone solo métodos relevantes para su contexto
- Separación de concerns: Creación (CuentaController) vs. Gestión (CiudadanoController)
- Facilita autorización futura: Diferentes permisos por endpoint (ej: solo ciudadanos pueden editar su perfil)

Ejemplo:

- GET /api/ciudadanos/{id}: Consultar perfil de ciudadano
- PUT /api/ciudadanos/{id}: Actualizar datos personales
- DELETE /api/ciudadanos/{id}: Dar de baja cuenta
- GET /api/tecnicos?page=0: Listar técnicos disponibles (para operadores en el triaje)

3. Herencia JOINED para Jerarquía de Cuenta

Decisión: Usar @Inheritance(strategy = InheritanceType.JOINED) en JPA.

Ventajas:

- Sin duplicación: usuario, contrasena_hash, persona_id solo en tabla cuentas
- Normalización: Base de datos limpia y eficiente
- Consultas polimórficas: SELECT * FROM cuentas retorna todos los tipos
- Extensibilidad: Agregar campos específicos (ej: especialidad en tecnicos) sin afectar otras tablas

Alternativas descartadas:

- SINGLE_TABLE: Muchas columnas NULL (desperdicio de espacio), discriminador necesario
- TABLE_PER_CLASS: Duplicación de columnas, consultas polimórficas lentas (UNION ALL)

4. Relación 1:1 Obligatoria entre Cuenta y Persona

Decisión: Cada Cuenta tiene exactamente una Persona, y viceversa.

Justificación:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Separación de concerns: Cuenta maneja autenticación, Persona maneja datos personales
- Normalización: Evita duplicar nombres, apellidos, dni en cada subtipo de cuenta
- Cascada: Al eliminar cuenta, se elimina persona automáticamente (CascadeType.ALL)
- Constraint UNIQUE: Garantiza 1:1 a nivel de base de datos

Subsistema 2: Gestión de Reportes e Incidencias

Figura 3: Diagrama de Clases - Gestión de Reportes e Incidencias

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

ReporteServiceImpl	Service	Lógica de negocio: validaciones de workflow, coordinación con repositorios
IUbicacionService	Service Interface	Define contrato de operaciones de ubicación
UbicacionServiceImpl	Service	Lógica de negocio: gestión de ubicaciones GPS
Reporte	Entity	Entidad raíz del agregado, contiene lógica de cambio de estado
Ubicacion	Entity (Value Object)	Coordenadas GPS y dirección textual
EstadoReporte	Enum	Estados posibles del workflow
Prioridad	Enum	Niveles de prioridad (BAJA, MEDIA, ALTA)

Decisiones de Diseño:

1. Workflow de Estados:

Decisión: Implementar máquina de estados con transiciones válidas definidas

Justificación:

- Integridad de datos: Previene transiciones inválidas (ej: PENDIENTE → FINALIZADO directamente)
- Trazabilidad: Cada cambio de estado actualiza fechaActualizacion automáticamente
- Lógica encapsulada: Método cambiarEstado() en entidad Reporte mantiene cohesión
- Extensibilidad: Fácil agregar nuevos estados (ej: EN_ESPERA_APROBACION) sin romper código existente

2. Relación 1:1 Obligatoria con Ubicación (Composición)

Decisión: Cada Reporte tiene exactamente una Ubicación, con cascada completa.

Justificación:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Ubicación como Value Object: Ubicación no tiene identidad propia fuera de Reporte
- Cascada ALL: Al eliminar reporte se elimina ubicación automáticamente (sin huérfanos)
- Obligatoriedad: optional = false garantiza que todo reporte tiene ubicación GPS
- Simplicidad: No se necesita UbicaciónRepository para operaciones básicas (aunque existe para consultas especializadas)
- Constraint UNIQUE: Garantiza que una ubicación no pueda compartirse entre reportes

3. Gestión de Prioridades con Valor por Defecto

Decisión: Toda incidencia inicia con prioridad MEDIA, modificable por operador.

Justificación:

- Triaje eficiente: Operador puede repriorizar según análisis inicial
- Valor sensato por defecto: MEDIA evita sesgos iniciales
- Método dedicado: cambiarPrioridad() actualiza timestamp automáticamente
- Constraint CHECK: Base de datos valida que solo valores válidos sean almacenados

4. Timestamps Automáticos de Auditoría

Decisión: Mantener fechaCreacion y fechaActualizacion para trazabilidad completa.

Justificación:

- Auditoría: Rastrear cuándo se creó y cuándo fue la última modificación
- Callback automático: @PreUpdate de JPA actualiza timestamp en cada UPDATE
- Inmutabilidad de creación: updatable = false previene modificar fechaCreacion
- Soporte a SLA: Calcular tiempo de resolución (fechaCreacion → FINALIZADO)

5. Agregación Bidireccional con Asignaciones

Decisión: Reporte mantiene lista de asignaciones con método helper para consistencia.

Justificación:

- Historial completo: Lista mantiene todas las asignaciones (incluso cerradas)
- Consistencia bidireccional: Método helper previene inconsistencias
- Cascada ALL: Al eliminar reporte se eliminan sus asignaciones
- Método de cierre: cerrarAsignacionActiva() cierra asignaciones pendientes antes de FINALIZADO

6. Consultas Optimizadas con Paginación

Decisión: Endpoints devuelven Page<ReporteDTO> en lugar de listas completas.

Justificación:

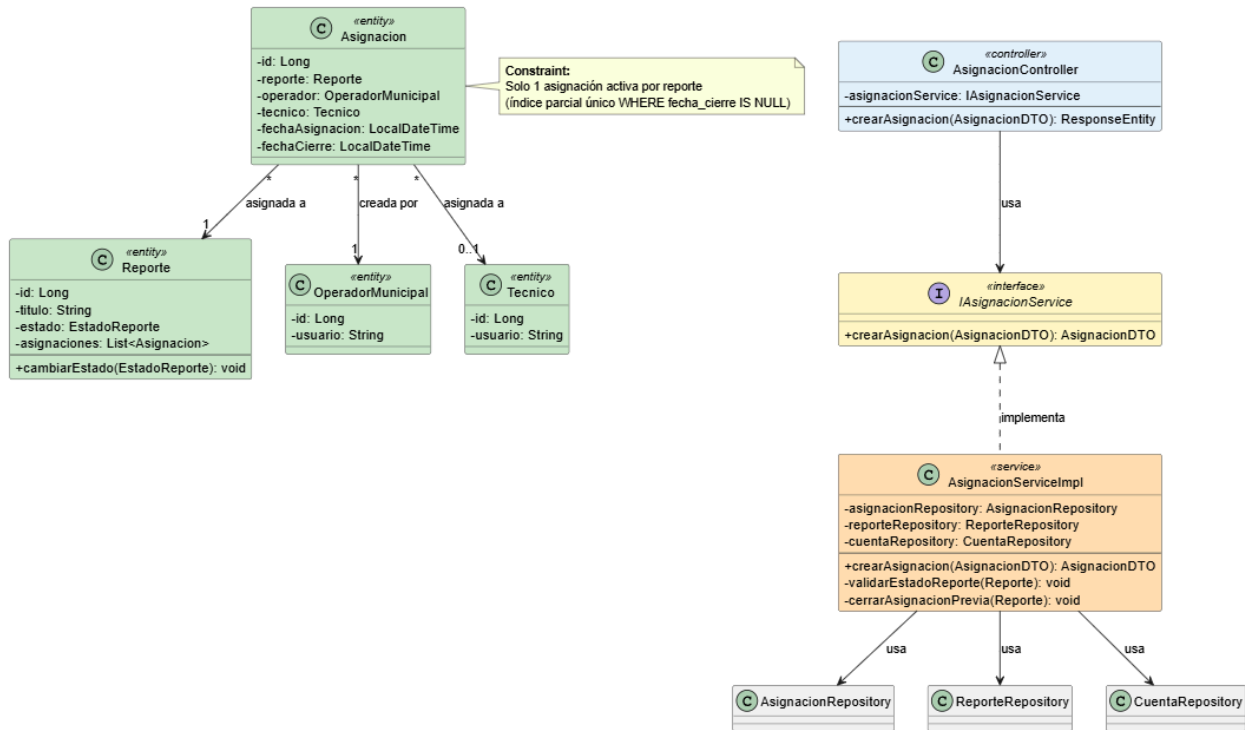
- Rendimiento: No cargar miles de reportes en memoria
- Escalabilidad: Frontend puede implementar scroll infinito o paginación
- Queries optimizadas: Spring Data JPA genera SQL con LIMIT/OFFSET

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Subsistema 3: Gestión de Asignaciones (Triage)

Figura 4: Diagrama de Clases - Gestión de Asignaciones y Triage

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Explicación del Subsistema de Asignaciones

Responsabilidades:

1. Implementar proceso de triaje de reportes: Operadores municipales revisan reportes en estado REVISION y determinan asignación
2. Asignar técnicos a reportes específicos: Vincular técnico disponible con reporte según especialidad/carga de trabajo
3. Mantener historial completo de asignaciones: Registrar todas las asignaciones (activas y cerradas) para auditoría y análisis
4. Gestionar ciclo de vida de asignaciones: Crear, cerrar y reasignar técnicos según necesidad
5. Coordinar transiciones de estado: Cambiar reporte de REVISION a PROCESO al asignar técnico

Componentes Principales:

Componente	Tipo	Responsabilidad
AsignacionController	Controller	Endpoint de creación de asignaciones (POST /api/asignaciones)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

IAsignacionService	Service Interface	Define contrato de operaciones de asignación
AsignacionServiceImpl	Service	Lógica de negocio: validaciones de roles, estado del reporte, cierre de asignaciones previas
Asignacion	Entity	Entidad que vincula reporte, operador y técnico con fechas de auditoría
Reporte	Entity	Entidad relacionada que contiene lista de asignaciones
OperadorMunicipal	Entity	Cuenta responsable de crear la asignación
Tecnico	Entity	Cuenta asignada para resolver la incidencia

Decisiones de Diseño:

1. Asignación Única Activa por Reporte (Constraint de Base de Datos)

Decisión: Sólo puede existir una asignación con fechaCierre = NULL por reporte.

Justificación:

- Integridad referencial: Previene asignaciones duplicadas activas (un técnico trabajando mientras otro también está asignado)
- Rendimiento en consultas: Índice optimiza búsqueda de asignación activa: `SELECT * FROM asignaciones WHERE reporte_id = ? AND fecha_cierre IS NULL`
- Constraint a nivel de BD: Garantía más fuerte que validación en código (race conditions prevenidas)
- Soporte a reasignaciones: Permite múltiples asignaciones en el historial, pero solo una abierta

2. Técnico Nullable (Asignación Preparatoria)

Decisión: El campo `tecnico_id` es opcional (`nullable = true`), permitiendo crear asignaciones sin técnico.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Justificación:

- Flexibilidad en el workflow: Operador puede aceptar reporte en REVISION sin asignar técnico inmediatamente
- Proceso de triaje en dos pasos:
 - i. Operador crea asignación "en espera" (tecnico = null)
 - ii. Operador asigna técnico cuando identifica al más adecuado
- Análisis de carga: Permite consultar técnicos con menos asignaciones activas
- Escalabilidad: Facilita futura implementación de asignación automática por algoritmo

3. Cierre Automático de Asignación Previa (Reasignación)

Decisión: Antes de crear nueva asignación, cerrar automáticamente cualquier asignación activa del mismo reporte.

Justificación:

- Reasignaciones transparentes: Técnico A no disponible → reasignar a Técnico B automáticamente
- Historial completo: Asignaciones cerradas permanecen en BD para auditoría
- Prevención de errores: Evita violar constraint de índice único parcial
- Atomicidad: Cierre + creación en misma transacción (@Transactional)

4. Validación Estricta de Roles con instanceof

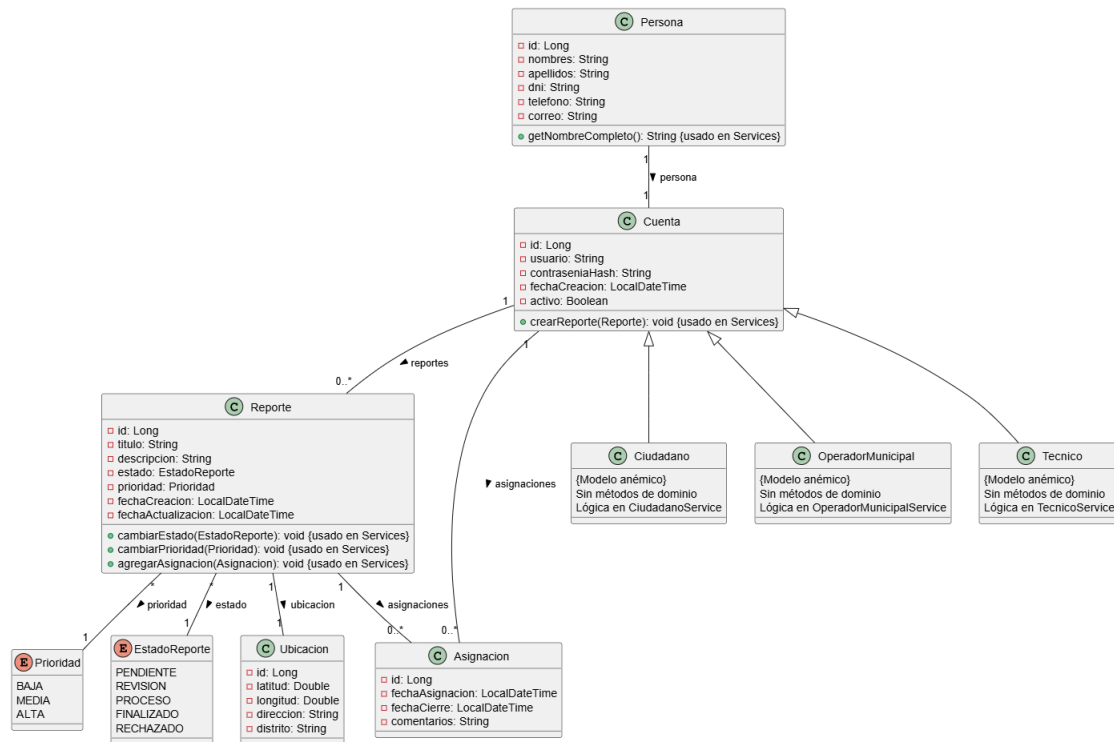
Decisión: Validar que operador sea OperadorMunicipal y técnico sea Tecnico antes de crear asignación.

Justificación:

- Seguridad: Previene que ciudadano se asigne reportes a sí mismo
- Integridad de negocio: Solo roles autorizados participan en triaje
- Type-safety: Casting seguro después de validación instanceof
- Mensajes de error claros: Errores específicos por tipo de validación

Diagrama de Clases del software:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Sprint 2:

Los diagramas de clases de diseño están organizados por subsistemas/dominios de negocio (no por capas técnicas). Esto permite una mejor visualización de:

1. Cohesión: Las clases relacionadas que trabajan juntas
2. Flujo de datos: Cómo viajan los datos de entrada a salida
3. Responsabilidades: Qué hace cada componente
4. Relaciones: Dependencias entre clases

Esta organización es más natural que separar por capas técnicas (controllers, services, models) porque muestra cómo realmente funciona el sistema desde la perspectiva del negocio.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Estructura de los Subsistemas

```

Subsistema 1: Autenticación y Gestión de Cuentas
├─ Controller: CuentaController, AuthController
├─ Service: ICuentaService, AuthService
├─ Factory: CuentaFactory, CuentaFactoryProvider
├─ DTO: CuentaDTO, LoginRequest, LoginResponse
├─ Model: Cuenta, Ciudadano, Tecnico, OperadorMunicipal, Persona
└─ Repository: CuentaRepository

Subsistema 2: Gestión de Reportes (Ciudadano)
├─ Controller: ReporteController
├─ Service: IReporteService
├─ DTO: ReporteDTO, UbicacionDTO
├─ Model: Reporte, Ubicacion, EstadoReporte, Prioridad
└─ Repository: ReporteRepository, UbicacionRepository

Subsistema 3: Procesamiento de Reportes (Técnico)
├─ Controller: TecnicoController
├─ Service: ITecnicoService
├─ DTO: CompletarReporteRequest, TecnicoDTO, FotoDTO
├─ Model: Tecnico, Asignacion, Foto, TipoFoto
└─ Repository: TecnicoRepository, AsignacionRepository, FotoRepository

Subsistema 4: Auditoría de Reportes (Operador Municipal)
├─ Controller: OperadorController
├─ Service: IOperadorService
├─ DTO: CerrarReporteRequest, RechazarAuditoriaRequest, RechazarDefinitivoRequest
├─ Model: OperadorMunicipal, Reporte (estados RESUELTA+CERRADA/RECHAZADO)
└─ Repository: ReporteRepository

Subsistema 5: Notificaciones Push
├─ Controller: NotificationController
├─ Service: INotificationService
├─ DTO: TokenNotificacionDTO
├─ Model: TokenNotificacion, MensajeNotificacion
└─ Repository: TokenNotificacionRepository, MensajeNotificacionRepository

Subsistema 6: Auditoría (Historial de Estados)
├─ Controller: HistorialEstadoController
├─ Service: IHistorialEstadoService
├─ DTO: HistorialEstadoDTO
├─ Model: HistorialEstado
└─ Repository: HistorialEstadoRepository

```

Diagrama de Clases del software:

SUBSISTEMA 1: Autenticación y Gestión de Cuentas

Propósito: Crear y administrar cuentas de usuarios (Ciudadano, Técnico, Operador Municipal).

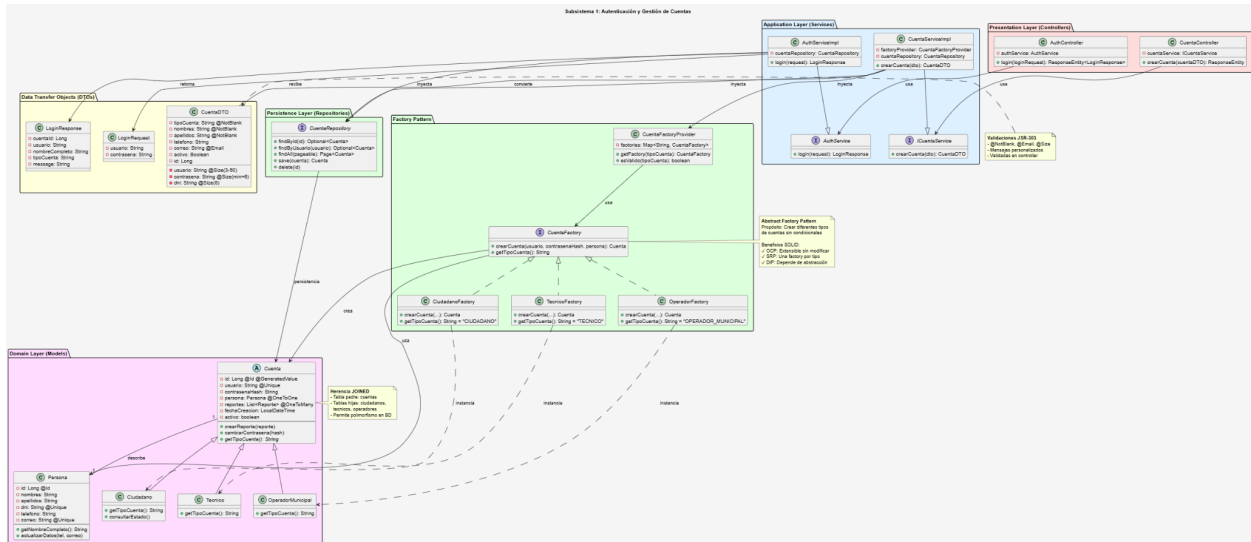
Justificación del Diseño:

- Utiliza Abstract Factory Pattern para crear diferentes tipos de cuentas

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Separación clara entre tipos de usuario
- Validaciones centralizadas en DTOs
- Herencia polimórfica en el modelo

Diagrama de Clases - Subsistema Cuentas



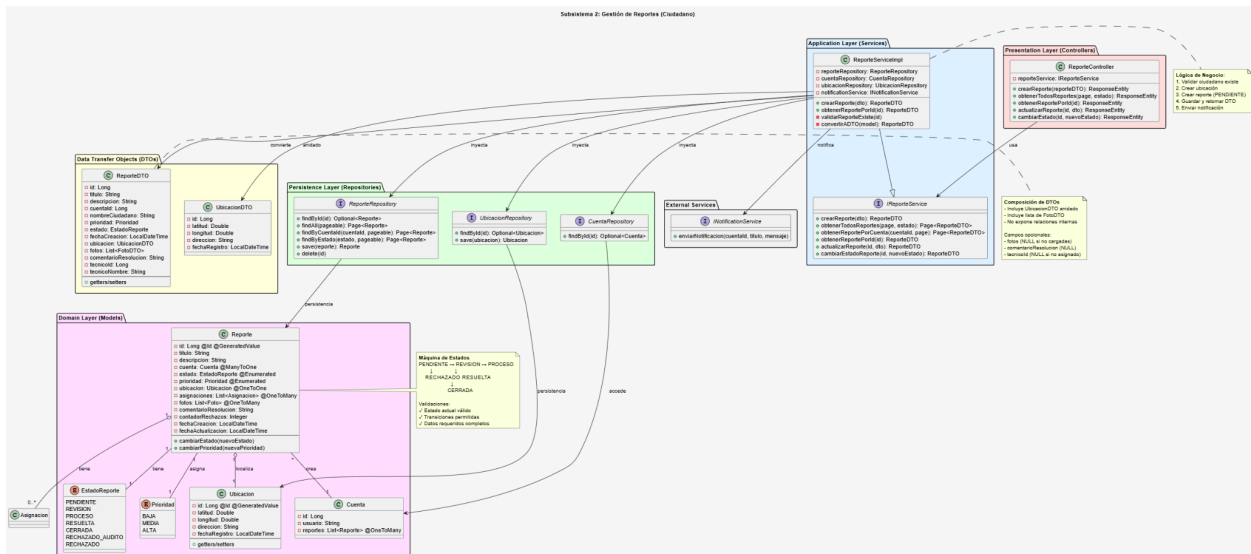
SUBSISTEMA 2: Gestión de Reportes (Ciudadano)

Propósito: Ciudadanos crean reportes de incidencias urbanas.

Justificación del Diseño:

- El ciudadano es el actor principal que inicia todo
- DTO complejo que incluye ubicación anidada
- Estados iniciales: PENDIENTE → REVISION
- Validaciones en servicio (no solo DTOs)

Diagrama de Clases - Subsistema Reportes



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

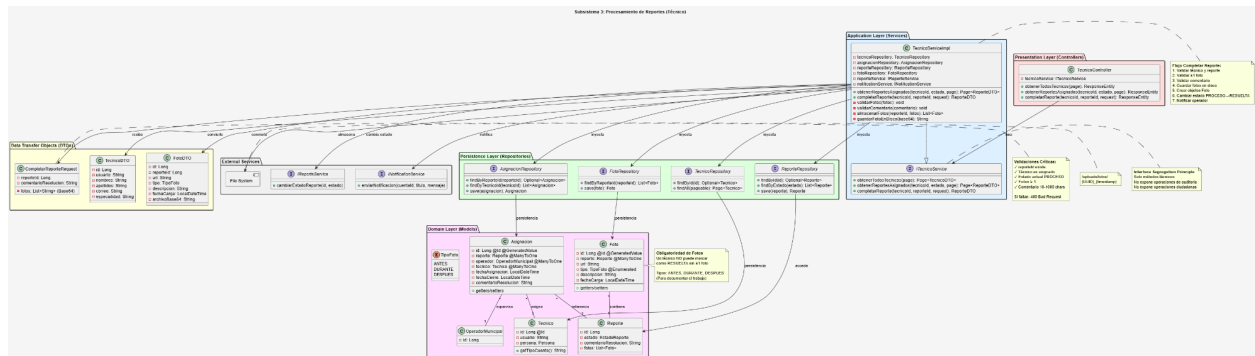
SUBSISTEMA 3: Procesamiento de Reportes (Técnico)

Propósito: Técnicos reciben reportes asignados, agregan fotos y comentarios.

Justificación del Diseño:

- Separación clara: técnico solo ve reportes asignados
- Validaciones críticas: 1 foto obligatoria, comentario 10-1000 chars
- Gestión de fotos: Base64 → almacenamiento en disco
- Transición de estado: PROCESO → RESUELTA

Diagrama de Clases - Subsistema Técnico



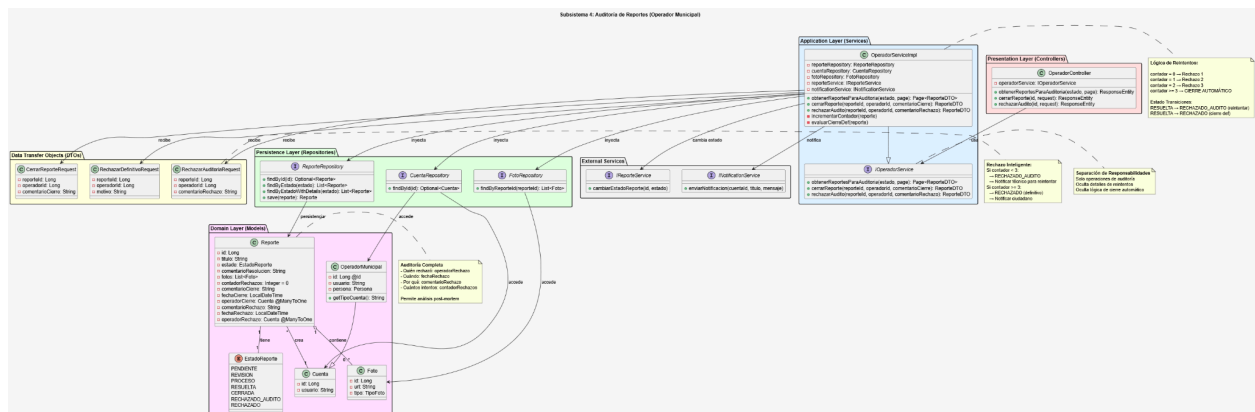
SUBSISTEMA 4: Auditoría de Reportes (Operador Municipal)

Propósito: Operadores revisan reportes resueltos, aprueban o rechazan.

Justificación del Diseño:

- Separación clara: operador solo ve reportes en estado RESUELTA
- Lógica de reintentos: contador de rechazos
- Transiciones: RESUELTA → CERRADA o RECHAZADO_AUDITO/RECHAZADO
- Campos de auditoría: quién, cuándo, por qué

Diagrama de Clases - Subsistema Operador



SUBSISTEMA 5: Notificaciones Push

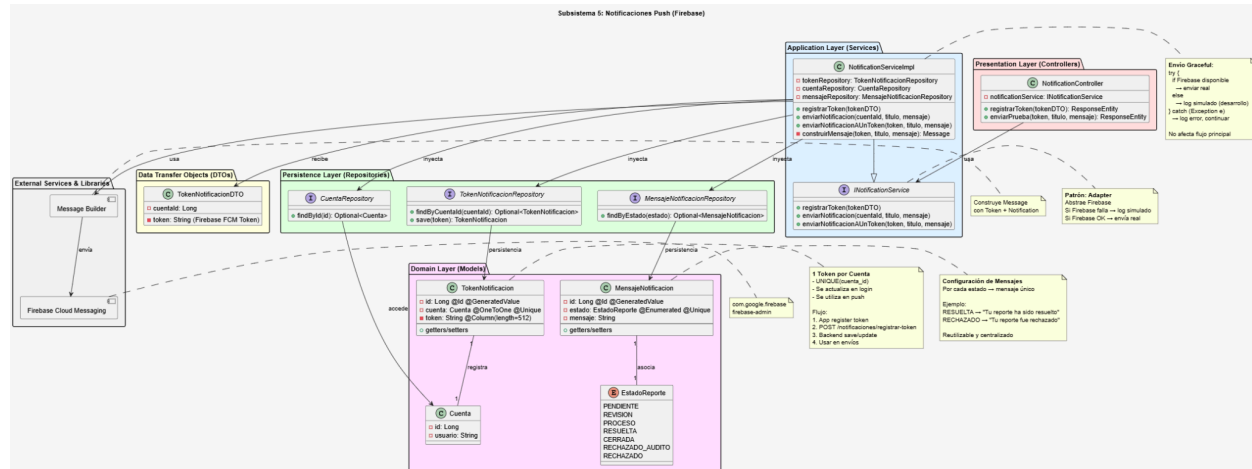
Propósito: Enviar notificaciones a usuarios vía Firebase Cloud Messaging.

Justificación del Diseño:

- Integración con Firebase
- Registro de tokens por cuenta
- Manejo elegante si Firebase no está disponible

- Mensajes personalizados por estado

Diagrama de Clases - Subsistema Notificaciones



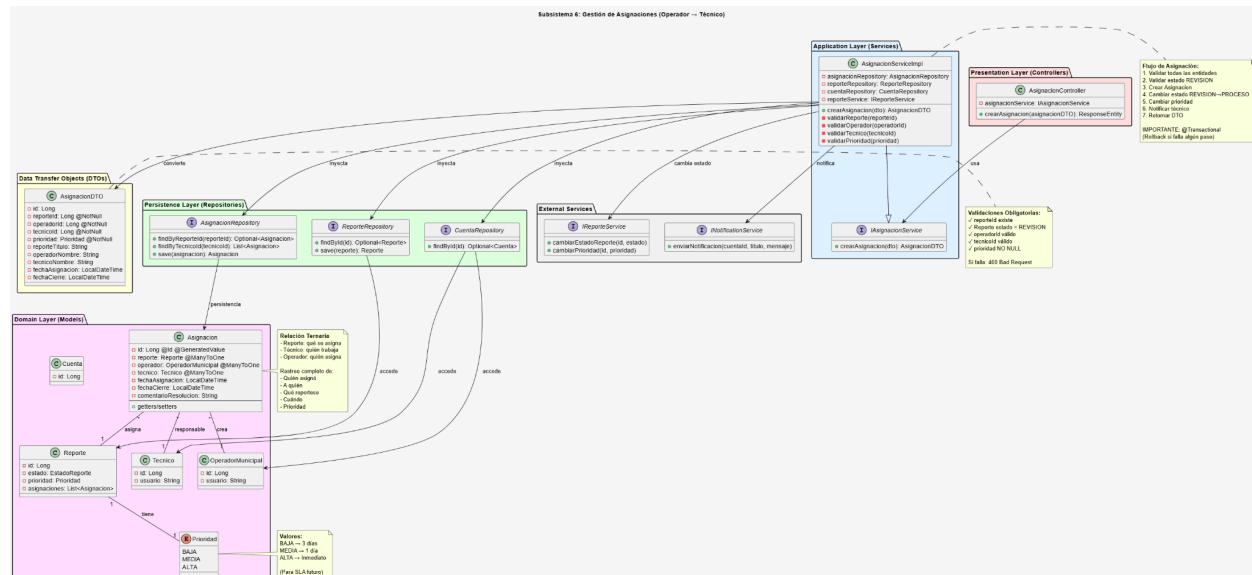
SUBSISTEMA 6: Gestión de Asignaciones

Propósito: Operadores asignan reportes a técnicos con prioridad.

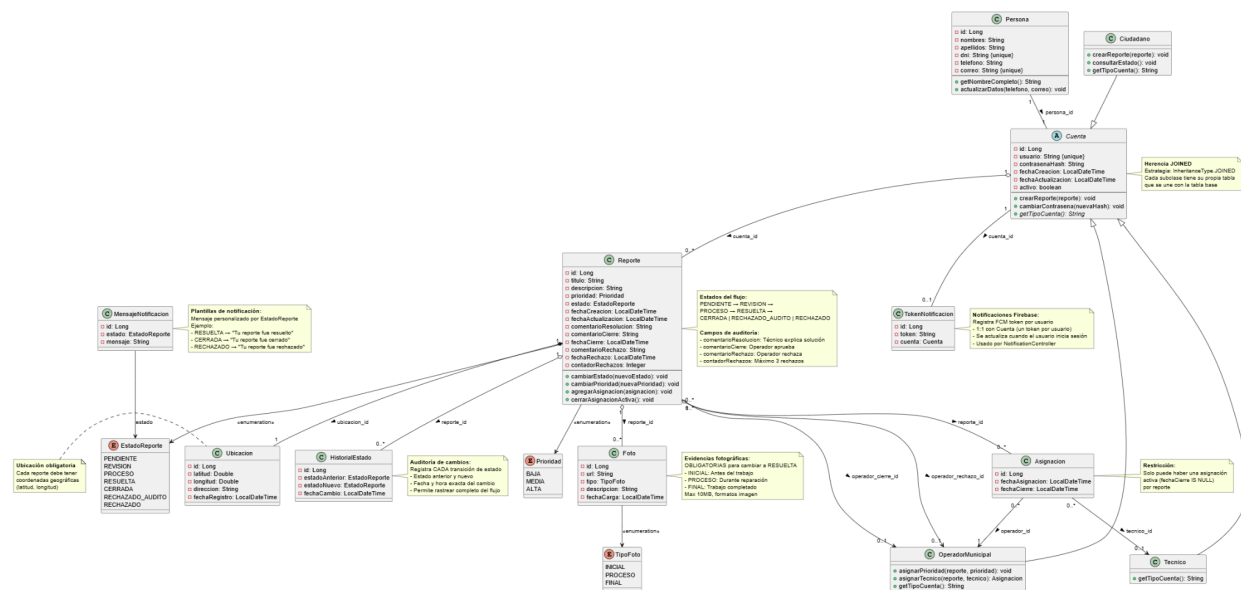
Justificación del Diseño:

- Validación de prioridad obligatoria
- Validación de estado REVISION obligatorio
- Separación clara de responsabilidades
- Transición de estado REVISION → PROCESO

Diagrama de Clases - Subsistema Asignaciones

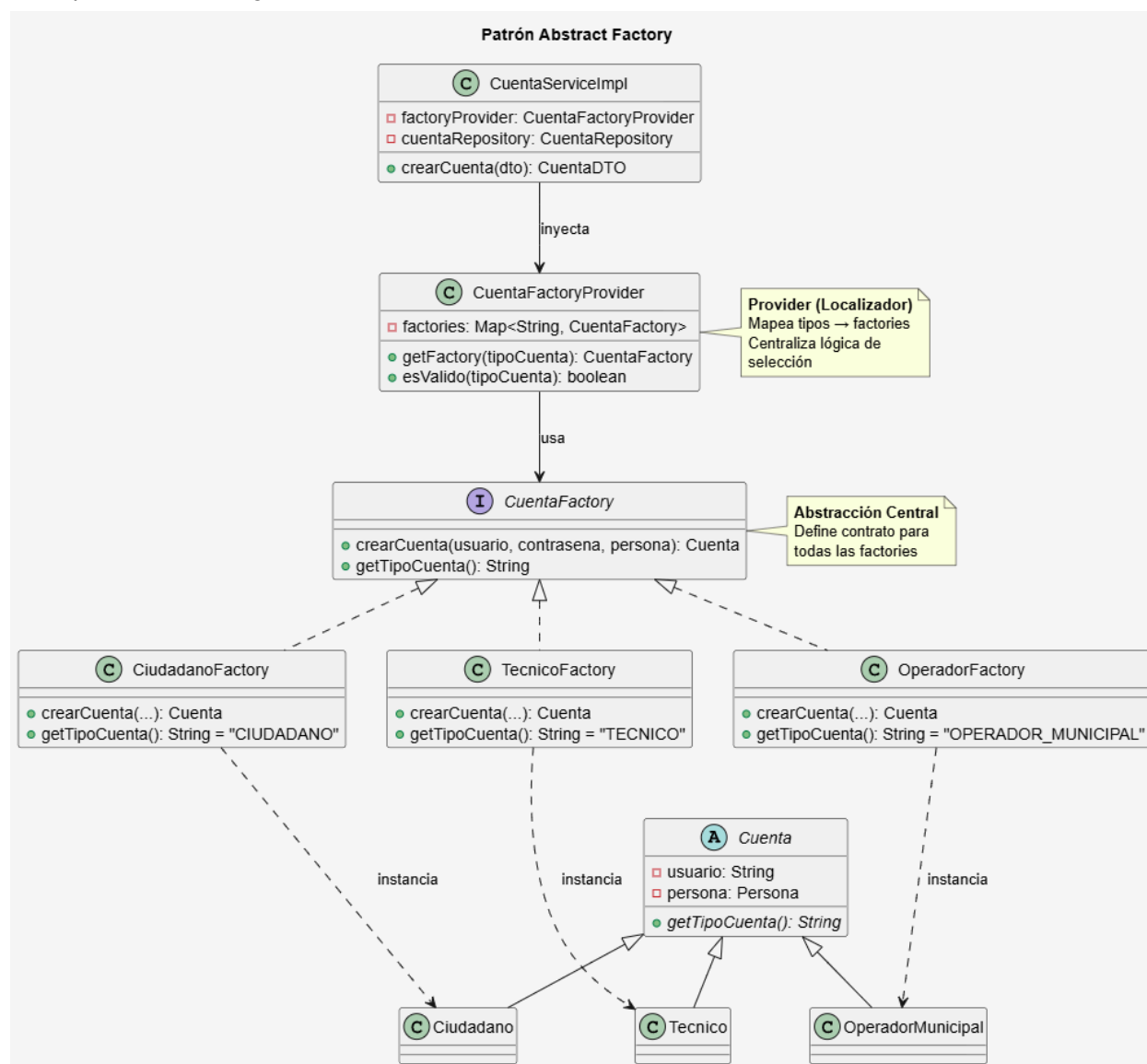


ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Participantes en el Diagrama



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Implementación en Código

```
// Interfaz (abstracción)
public interface CuentaFactory {
    Cuenta crearCuenta(String usuario, String contrasenaHash, Persona persona);
    String getTipoCuenta();
}

// Implementación específica
public class CiudadanoFactory implements CuentaFactory {
    @Override
    public Cuenta crearCuenta(String usuario, String contrasenaHash, Persona persona) {
        return new Ciudadano(usuario, contrasenaHash, persona);
    }

    @Override
    public String getTipoCuenta() {
        return "CIUDADANO";
    }
}

// Provider que localiza la factory
@Component
public class CuentaFactoryProvider {
    private final Map<String, CuentaFactory> factories = new HashMap<>();

    public CuentaFactory getFactory(String tipoCuenta) {
        return factories.get(tipoCuenta);
    }
}

// Servicio que usa la factory (sin if-else)
@Service
public class CuentaServiceImpl implements ICuentaService {
    private final CuentaFactoryProvider factoryProvider;

    @Override
    public CuentaDTO crearCuenta(CuentaDTO dto) {
        // Seleccionar factory sin condicional
        CuentaFactory factory = factoryProvider.getFactory(dto.getTipoCuenta());

        // Crear usando factory
        Cuenta cuenta = factory.crearCuenta(
            dto.getUsuario(),
            dto.getContrasena(),
            persona
        );

        return convertirADTO(cuentaRepository.save(cuenta));
    }
}
```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Cumplimiento SOLID

Principio	Cómo	Beneficio
SRP	Cada factory = 1 tipo de cuenta	Cambios aislados
OCP	Agregar tipo = nueva factory, sin modificar código	Extensible
LSP	Todas implementan CuentaFactory	Intercambiables
DIP	Service depende de CuentaFactory (abstracción)	Desacoplado

Ventajas en Este Proyecto

- Fácil agregar nuevo tipo (ej: ContratistasFactory)
- Lógica de creación encapsulada
- Testeable: mockear factories
- Sin condicionales = menos bugs

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Patrón 2: Repository Pattern

Propósito

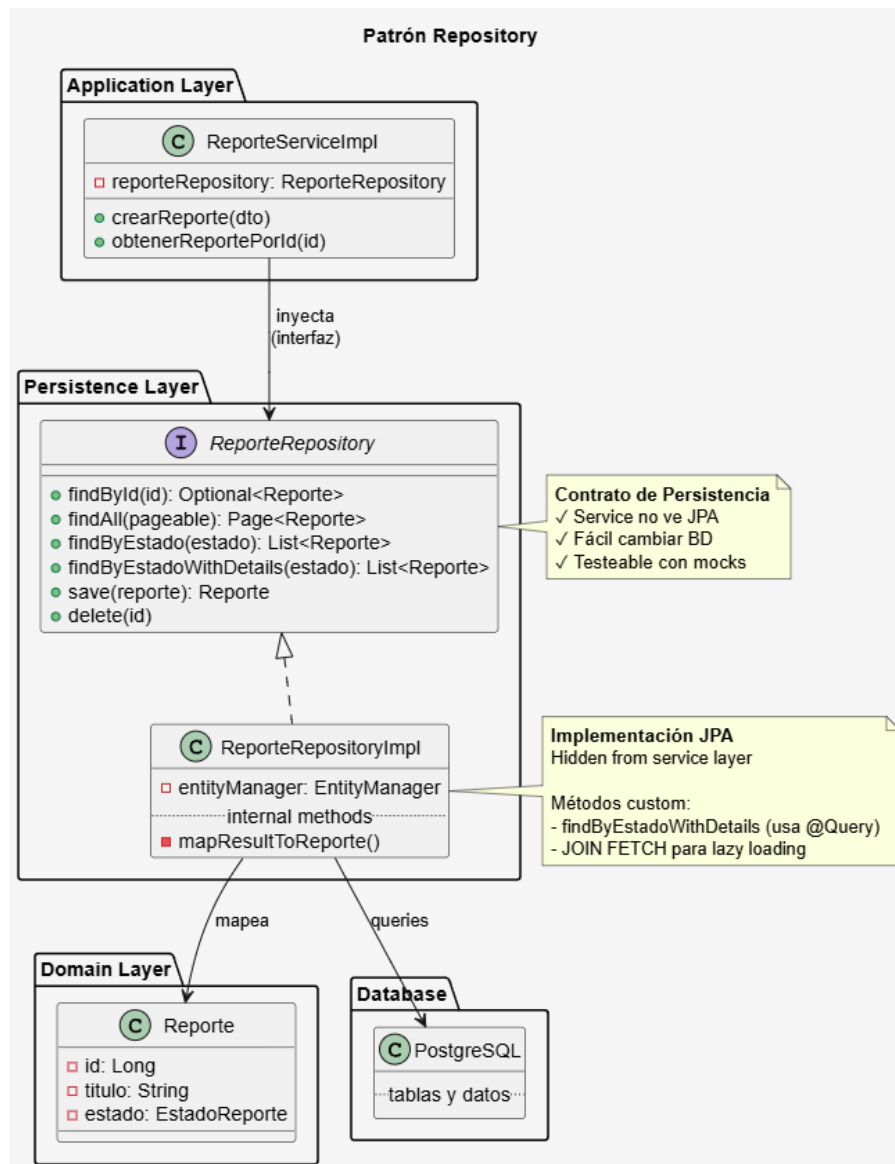
Abstraer la persistencia de datos. Los servicios no conocen JPA, solo interfaces de repositorio.

Problema Resuelto

Sin este patrón:

```
// X MAL - Service conoce JPA detalles
EntityManager em = ...; // Acoplamiento
Query q = em.createQuery("SELECT..."); // Query en servicio
```

Participantes en el Diagrama



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Implementación en Código

```
// Interfaz (contrato)
public interface ReporteRepository extends JpaRepository<Reporte, Long> {
    Optional<Reporte> findById(Long id);
    Page<Reporte> findByEstado(EstadoReporte estado, Pageable pageable)

    // Custom query para evitar N+1
    @Query("SELECT r FROM Reporte r " +
        "JOIN FETCH r.ubicacion " +
        "JOIN FETCH r.cuenta " +
        "WHERE r.estado = ?1")
    List<Reporte> findByEstadoWithDetails(EstadoReporte estado);
}

// Servicio usa repository sin conocer JPA
@Service
@Transactional
public class ReporteServiceImpl implements IReporteService {

    private final ReporteRepository reporteRepository;

    @Override
    public ReporteDTO obtenerReportePorId(Long id) {
        // ✅ Servicio solo usa interfaz
        Reporte reporte = reporteRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("No encontrado"));

        return convertirADTO(reporte);
    }
}
```

Cumplimiento SOLID

Principio	Cómo
DIP	Service → ReporteRepository (abstracción, no implementación)
SRP	Repository = persistencia, Service = lógica negocio

Ventajas

- Cambiar de PostgreSQL a MongoDB = solo cambiar implementación
- Servicios testeable con mocks
- Queries complejas encapsuladas
- JOIN FETCH para lazy loading centralizado

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Patrón 3: Service Layer + Transactional

Propósito

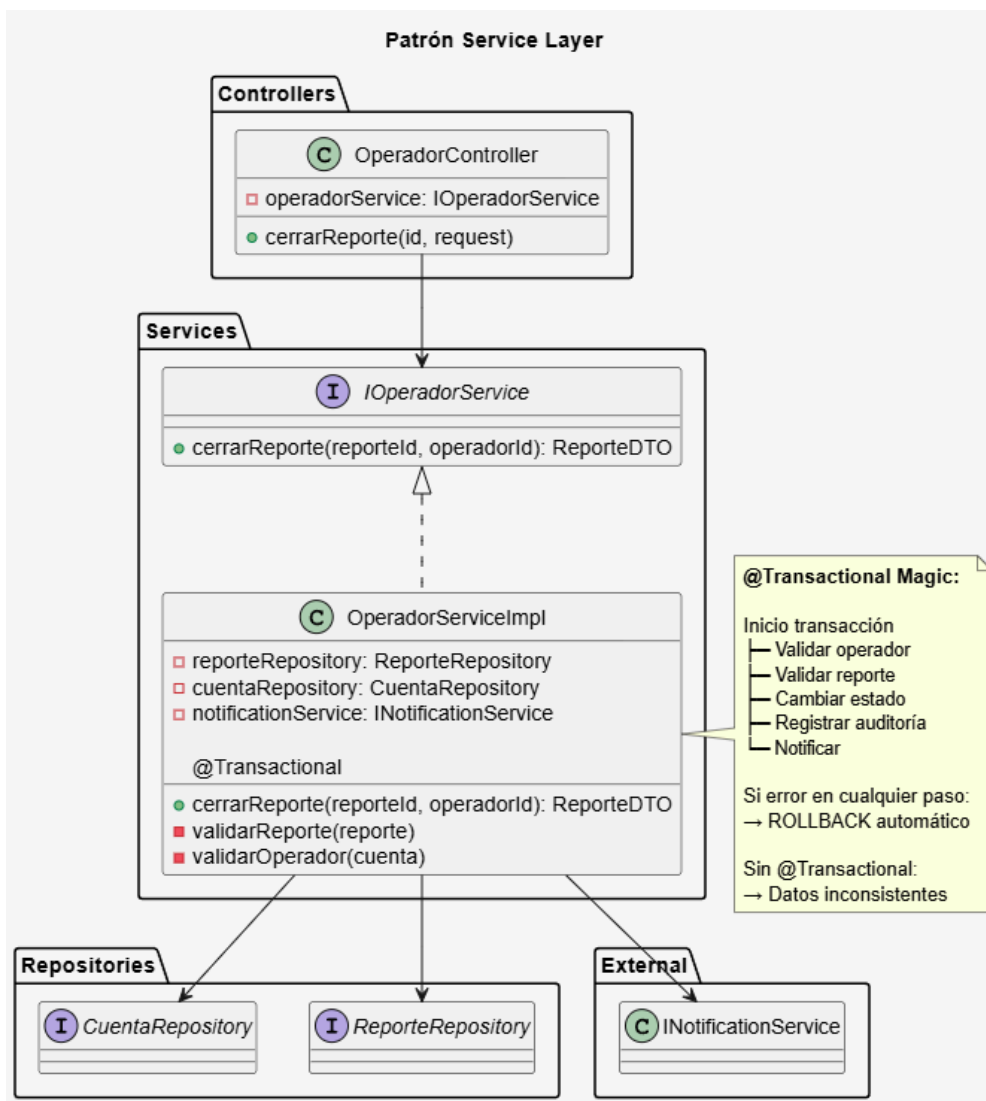
Centralizar lógica de negocio. Garantizar consistencia ACID con @Transactional.

Problema Resuelto

Sin este patrón:

```
// X MAL - Lógica dispersa en controllers
if (operador == null) throw...;
if (reporte.getEstado() != RESUELTA) throw...;
reporte.setEstado(CERRADA);
// Si falla después = BD inconsistente
```

Participantes en el Diagrama



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Implementación en Código

```

@Service
@Transactional // ← Cada método es transacción
public class OperadorServiceImpl implements IOperadorService {

    private final ReporteRepository reporteRepository;
    private final CuentaRepository cuentaRepository;
    private final INotificationService notificationService;

    @Override
    public ReporteDTO cerrarReporte(Long reporteId, Long operadorId,
                                    String comentarioCierre) {
        // VALIDACIÓN 1: Reporte existe
        Reporte reporte = reporteRepository.findById(reporteId)
            .orElseThrow(() -> new RuntimeException("Reporte no encontrado"));

        // VALIDACIÓN 2: Estado es RESUELTA
        if (!reporte.getEstado().equals(EstadoReporte.RESUELTA)) {
            throw new RuntimeException("Reporte debe estar RESUELTA");
        }

        // VALIDACIÓN 3: Operador existe
        Cuenta operador = cuentaRepository.findById(operadorId)
            .orElseThrow(() -> new RuntimeException("Operador no encontrado"));

        // CAMBIOS (se guardan al final si todo OK)
        reporte.setEstado(EstadoReporte.CERRADA);
        reporte.setComentarioCierre(comentarioCierre);
        reporte.setFechaCierre(LocalDate.now());
        reporte.setOperadorCierre(operador);

        Reporte reporteGuardado = reporteRepository.save(reporte);

        // NOTIFICAR (dentro de transacción)
        notificationService.enviarNotificacion(
            reporte.getCuenta().getId(),
            "Reporte Cerrado",
            "Tu reporte ha sido cerrado exitosamente"
        );

        return convertirADTO(reporteGuardado);
        // ← Si aquí falla notificación, TODO se rollback
    }
}

```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Cumplimiento SOLID

Principio	Cómo
SRP	Service = lógica, Controller = HTTP, Repository = BD
DIP	Controller → IOperadorService, no implementación

Ventajas

- ACID: Si falla 1 paso, rollback de todo
- Lógica compleja encapsulada
- Validaciones centralizadas
- Reutilizable desde múltiples controllers

Matriz Resumen: Patrones Aplicados

Patrón	Ubicación	Problema	Solución	Beneficio
Abstract Factory	factory/	Crear 3 tipos sin if-else	Interfaz implementaciones +	OCP, extensible
Repository	repository/	Service no debe conocer JPA	Interfaz abstracta	DIP, testeable
Service Layer	service/	Lógica dispersa en controllers	Servicios con @Transactional	ACID, cohesión
DTO Pattern	dto/	Exposición de modelos internos	DTOs anidados	ISP, seguridad
Dependency Injection	Spring Boot	Acoplamiento entre clases	Constructor injection	DIP, testeable

Conclusión

Los 3 patrones principales (Abstract Factory, Repository, Service Layer) trabajan juntos para:

1. Separar responsabilidades (SRP)
2. Permitir extensión sin modificación (OCP)
3. Dependier de abstracciones (DIP)
4. Garantizar consistencia (ACID con @Transactional)
5. Facilitar testing (mocks de interfaces)

Esto resulta en un backend mantenible, testeable y escalable.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

5. Modelo dinámico

Sprint 1

1. HU-01: Registro de Ciudadano

Como ciudadano sin cuenta en el sistema,
quiero registrarme proporcionando mis datos personales (nombres, apellidos, DNI, correo, teléfono, usuario y contraseña),
para poder crear y gestionar reportes de incidencias urbanas.

Criterios de Aceptación:

1. Escenario: Registro exitoso
 - Dado que proporcione datos válidos:
 - Nombres y apellidos (letras y espacios)
 - DNI de 8 dígitos numéricos único
 - Correo electrónico válido y único
 - Teléfono válido
 - Usuario único (4-50 caracteres alfanuméricos)
 - Contraseña segura (mínimo 8 caracteres)
 - Y acepto los términos y condiciones
 - Cuando envío el formulario de registro
 - Entonces el sistema:
 - Crea mi cuenta con tipo "CIUDADANO"
 - Muestra mensaje: "Registro completado exitosamente"
 - Me redirige a la pantalla de inicio de sesión
2. Escenario: Usuario o correo duplicado
 - Dado que ingreso un usuario o correo que ya existe en el sistema
 - Cuando envío el formulario
 - Entonces el sistema:
 - Muestra error específico: "El usuario ya está registrado" o "El correo ya está registrado"
 - No crea la cuenta
 - Resalta el campo con error
 - Mantiene los demás campos llenos
3. Escenario: DNI duplicado
 - Dado que ingreso un DNI que ya está registrado
 - Cuando envío el formulario
 - Entonces el sistema muestra: "El DNI ya está registrado en el sistema"
4. Escenario: Campos obligatorios vacíos o inválidos
 - Dado que dejo campos obligatorios vacíos o con formato inválido
 - Cuando intento enviar el formulario
 - Entonces el sistema:
 - Resalta los campos con error en rojo
 - Muestra mensajes específicos por campo:
 - "El nombre es obligatorio"
 - "El DNI debe tener 8 dígitos"
 - "El correo debe tener formato válido (ejemplo@dominio.com)"

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- "La contraseña debe tener al menos 8 caracteres"
- No permite el envío hasta corregir errores

2. HU-02: Iniciar Sesión

Como usuario registrado (CIUDADANO, OPERADOR_MUNICIPAL, TECNICO), quiero iniciar sesión con mi usuario y contraseña, para acceder a las funcionalidades del sistema según mi rol.

Criterios de Aceptación:

1. Escenario: Inicio de sesión exitoso
 - Dado que ingreso credenciales válidas (usuario y contraseña correctos)
 - Y mi cuenta está activa (activo = true)
 - Cuando presiono "Iniciar sesión"
 - Entonces el sistema:
 - Valida las credenciales contra el hash almacenado
 - Retorna mis datos básicos:
 - ID de cuenta
 - Usuario
 - Tipo de cuenta
(CIUDADANO/TECNICO/OPERADOR_MUNICIPAL)
 - Nombre completo (nombres + apellidos)
 - Me redirige al dashboard según mi rol:
 - CIUDADANO → Vista de mapa y "Crear reporte"
 - OPERADOR_MUNICIPAL → Dashboard de validación y triaje
 - TECNICO → Lista de "Mis tareas asignadas"
2. Escenario: Credenciales inválidas
 - Dado que ingreso usuario o contraseña incorrectos
 - Cuando intento iniciar sesión
 - Entonces el sistema:
 - Muestra mensaje genérico: "Usuario o contraseña incorrectos"
 - No revela cuál campo es inválido (seguridad)
 - Mantiene el campo usuario lleno
 - Limpia el campo contraseña
3. Escenario: Usuario no existe
 - Dado que ingreso un usuario que no está registrado
 - Cuando intento iniciar sesión
 - Entonces el sistema muestra: "Usuario o contraseña incorrectos"
4. Escenario: Cuenta inactiva
 - Dado que mi cuenta tiene activo = false
 - Cuando intento iniciar sesión
 - Entonces el sistema muestra: "Tu cuenta está deshabilitada. Contacta al administrador"
5. Escenario: Campos obligatorios vacíos
 - Dado que dejo usuario o contraseña vacíos
 - Cuando intento enviar

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Entonces el sistema:
 - Muestra mensaje: "Usuario y contraseña son obligatorios"
 - Resalta los campos vacíos
 - No permite el envío

3. HU-03: Crear Reporte con Ubicación Automática (Sprint 1)

Como ciudadano autenticado,
quiero crear un reporte de incidencia proporcionando ubicación GPS, título y descripción,
para notificar problemas urbanos al municipio.

Criterios de Aceptación:

1. Escenario: Creación exitosa con ubicación GPS automática
 - Dado que he iniciado sesión como CIUDADANO
 - Y tengo GPS activado y con señal precisa
 - Cuando accedo a "Crear reporte"
 - Entonces el sistema:
 - Captura automáticamente mi ubicación (latitud, longitud)
 - Muestra un mapa con mi posición marcada
 - Me permite ajustar manualmente el marcador si la ubicación no es exacta
 - Y cuando completo:
 - Título (5-255 caracteres)
 - Descripción (20-1000 caracteres)
 - Y presiono "Enviar reporte"
 - Entonces el sistema:
 - Crea el reporte con estado inicial PENDIENTE
 - Prioridad inicial MEDIA
 - Asocia la ubicación GPS capturada
 - Muestra mensaje: "Reporte creado exitosamente con número #12345"
 - Me redirige a "Mis reportes"
2. Escenario: GPS desactivado o sin señal
 - Dado que no tengo GPS activado o no hay señal
 - Cuando intento crear reporte
 - Entonces el sistema:
 - Muestra mensaje: "No se pudo obtener ubicación automática. Por favor, marca tu ubicación en el mapa"
 - Muestra mapa centrado en mi última ubicación conocida (o ciudad por defecto)
 - Me permite marcar manualmente la ubicación tocando el mapa
 - Requiere confirmación: "¿La ubicación marcada es correcta?"
3. Escenario: Campos obligatorios vacíos o inválidos
 - Dado que dejo campos obligatorios vacíos o con formato inválido:
 - Título < 5 caracteres o > 255
 - Descripción < 20 caracteres o > 1000
 - Sin ubicación marcada
 - Cuando intento enviar

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Entonces el sistema:
 - Muestra errores específicos por campo:
 - "El título debe tener entre 5 y 255 caracteres"
 - "La descripción debe tener entre 20 y 1000 caracteres"
 - "Debes marcar la ubicación del problema"
 - Resalta los campos con error en rojo
 - No permite el envío hasta corregir
- 4. Escenario: Ubicación fuera de rango válido
 - Dado que la ubicación capturada tiene coordenadas inválidas:
 - Latitud no está entre -90 y 90
 - Longitud no está entre -180 y 180
 - Cuando intento enviar
 - Entonces el sistema:
 - Muestra error: "La ubicación capturada no es válida. Por favor, marca manualmente en el mapa"
 - No permite el envío

4. HU-04: Validar Reportes Pendientes (Aprobar)

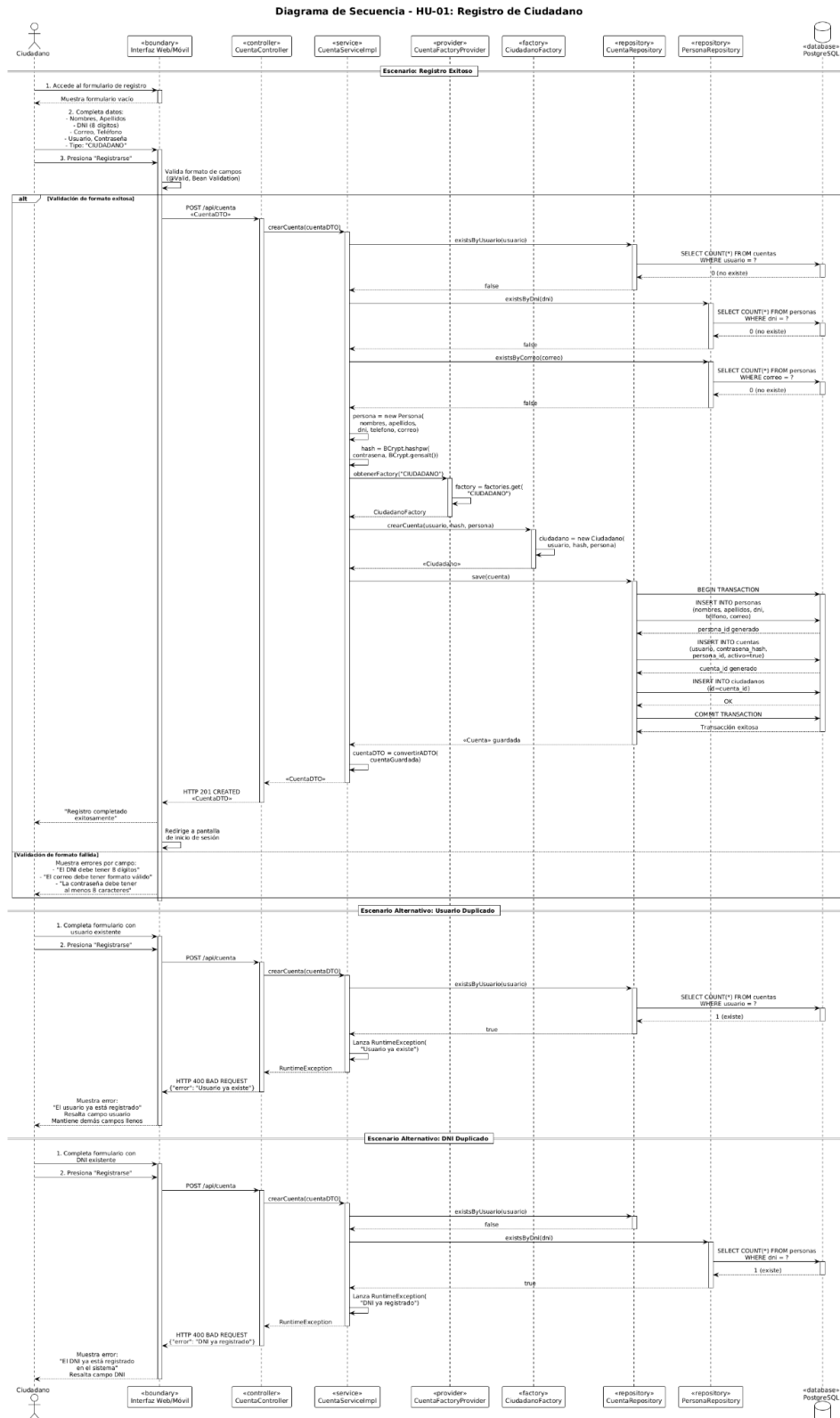
Como operador municipal autenticado,
quiero revisar reportes en estado PENDIENTE y aprobarlos o rechazarlos,
para controlar qué incidencias son válidas antes del triaje.

Criterios de Aceptación:

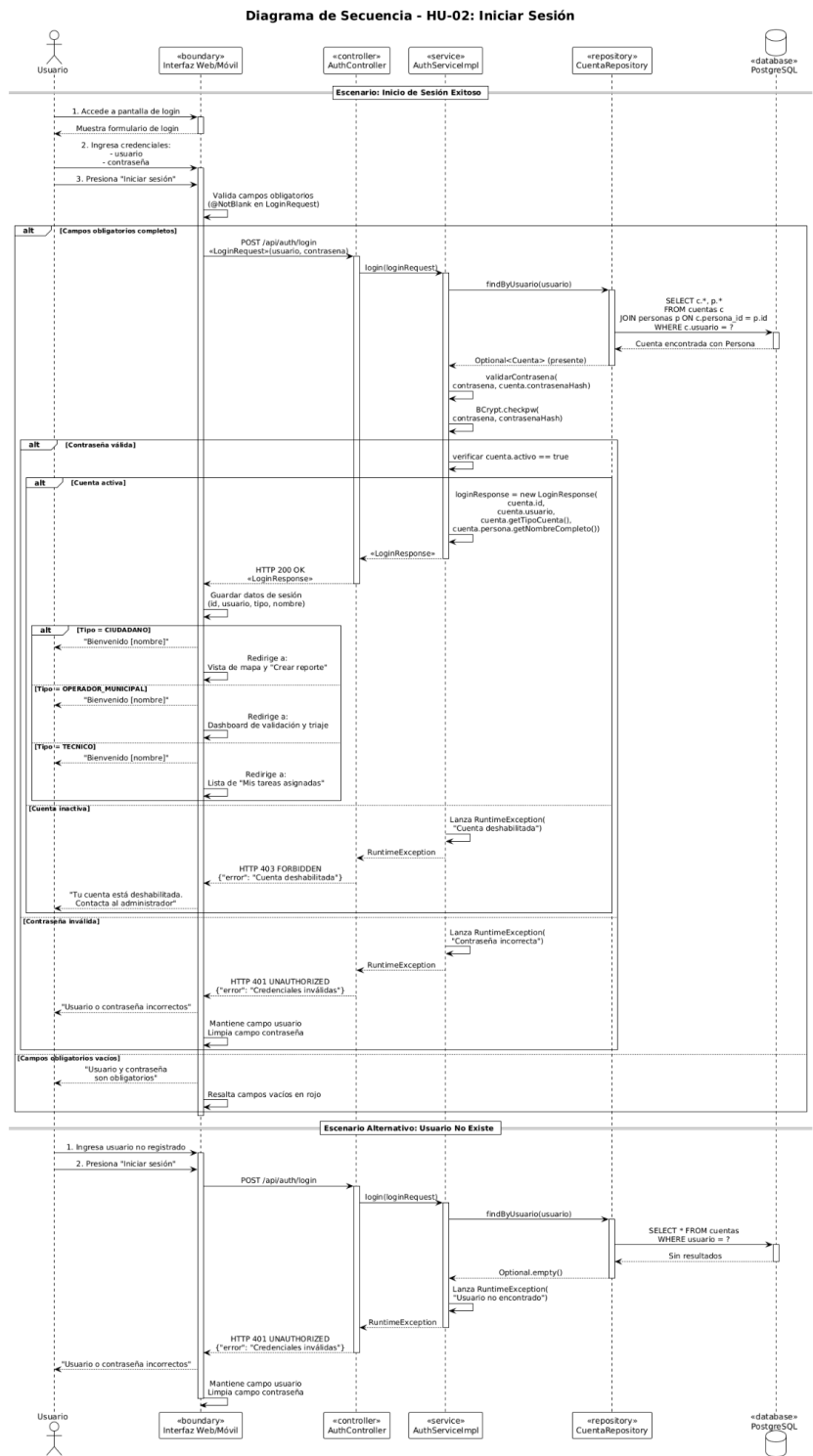
1. Escenario: Aprobar reporte válido
 - Dado que accedo al dashboard de validación
 - Y veo la lista de reportes con estado PENDIENTE
 - Cuando selecciono un reporte y reviso:
 - Título y descripción claros
 - Ubicación GPS correcta en el mapa
 - No es duplicado de otro reporte
 - Y presiono "Aprobar"
 - Entonces el sistema:
 - Cambia el estado del reporte a REVISION
 - Actualiza fechaActualizacion
 - Muestra mensaje: "Reporte #12345 aprobado exitosamente"
 - Remueve el reporte de la lista de pendientes
 - Notifica al ciudadano: "Tu reporte ha sido aprobado y está en proceso de revisión"

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

5.1.1 HU-01



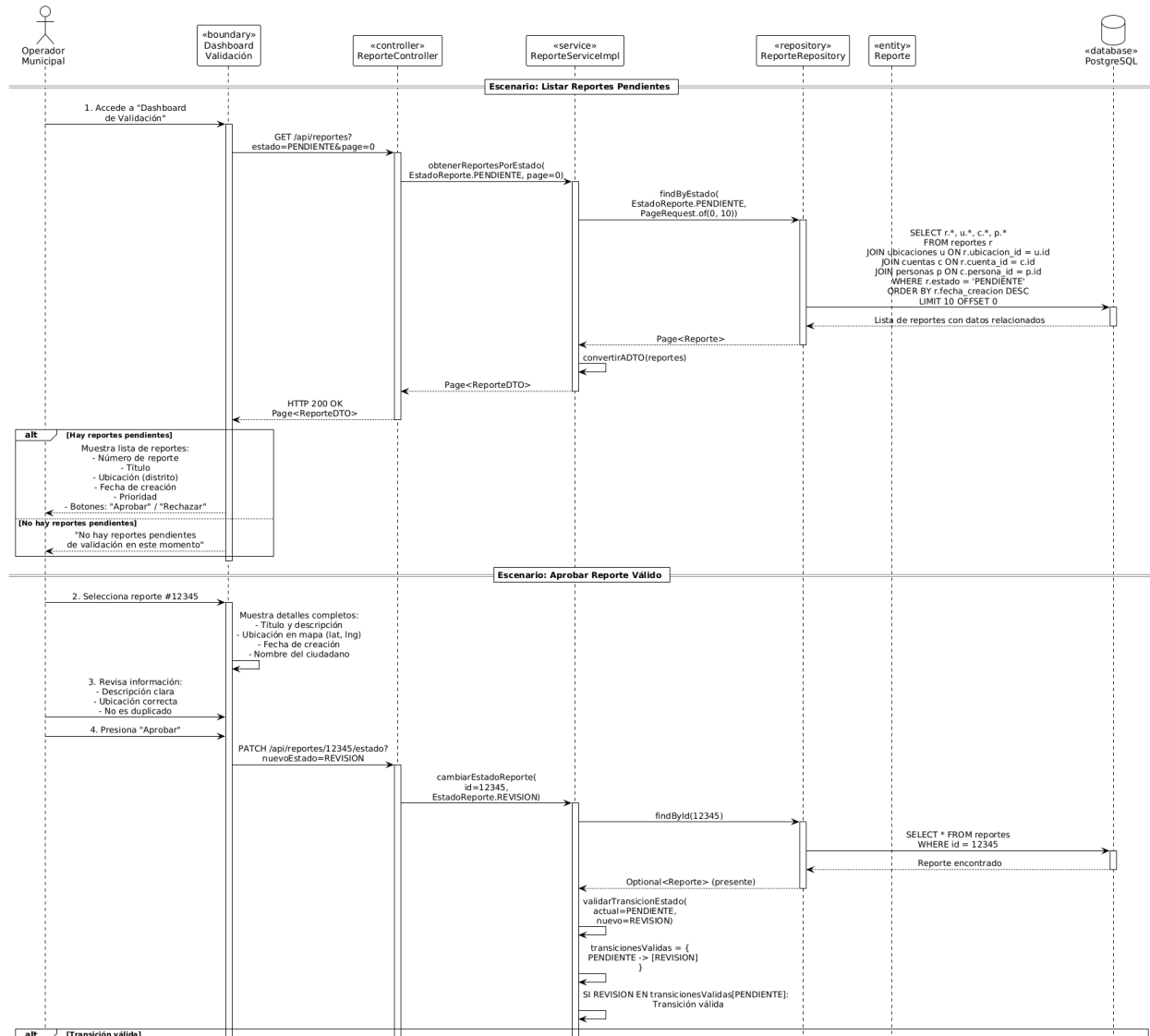
5.2.1 HU-02



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

5.4.1 HU 04

Diagrama de Secuencia - HU-04: Validar Reportes Pendientes (Aprobar/Rechazar)



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Sprint 2

El modelo dinámico describe cómo responde el sistema a eventos reales mediante diagramas de secuencia UML. Se modelan los 4 flujos principales basados en las Historias de Usuario identificadas:

1. HU: Seguimiento del estado y notificaciones push
2. HU Mapa de reportes con filtros y detalle desde marcadores
3. HU Detalle de Reporte
4. HU Validar Reportes Pendientes
5. HU Triage: Operador asigna reportes a técnicos con prioridad
6. HU Cerrar Reporte: Operador cierra reportes resueltos (con rechazo-audio)
7. HU Mis Tareas: Técnico visualiza reportes asignados
8. HU Completar con Evidencias: Técnico añade fotos y comentarios (con rechazo del operador)

HU: Seguimiento del estado y notificaciones push

Escenario Unificado: "Explorar mis reportes, revisar detalle e historial, y ser notificado de cambios de estado." Actor: Ciudadano autenticado

Precondición global: Sesión iniciada; token FCM registrado (ideal).

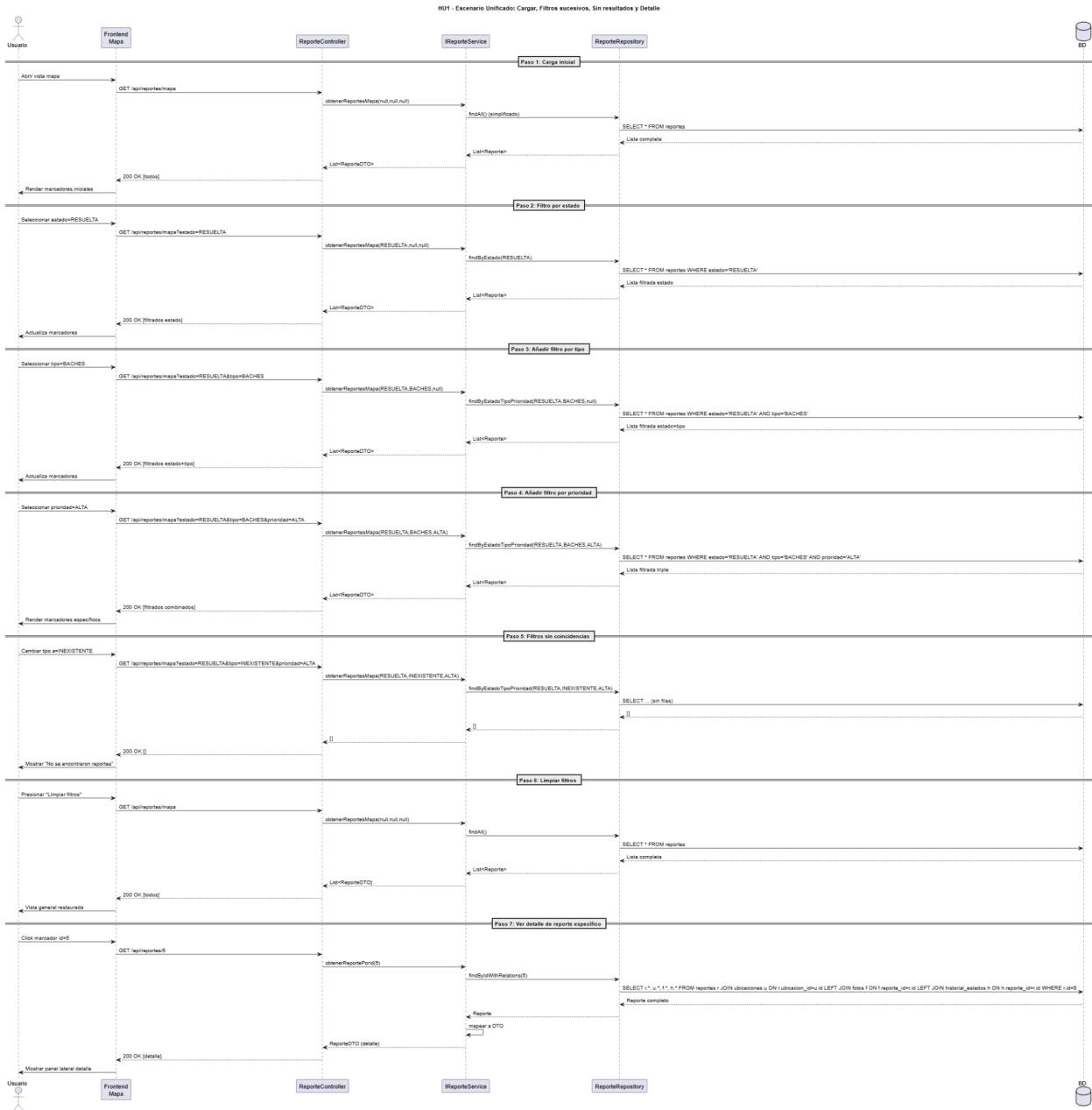
Pasos:

1. Listar "Mis reportes".
2. Abrir detalle de uno.
3. Consultar historial de estados.
4. (Backend) Cambio de estado ejecutado por operador/técnico.
5. Envío de notificación push (si el cambio está dentro de los actualmente instrumentados).
6. Ciudadano recibe notificación y verifica estado actualizado.

Postcondición global: El ciudadano dispone de estado e historial actualizados y recibe avisos de eventos clave.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

resultados cuando aplica filtros extremos y acceder al detalle de una incidencia específica.



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

HU Detalle de Reporte

1. Escenario Unificado (Narrativa Global)

Escenario Unificado: "Consultar el detalle completo de un reporte desde el contexto de mapa o lista para revisar su información base, evidencias y seguimiento, gestionando también el caso de inexistencia."

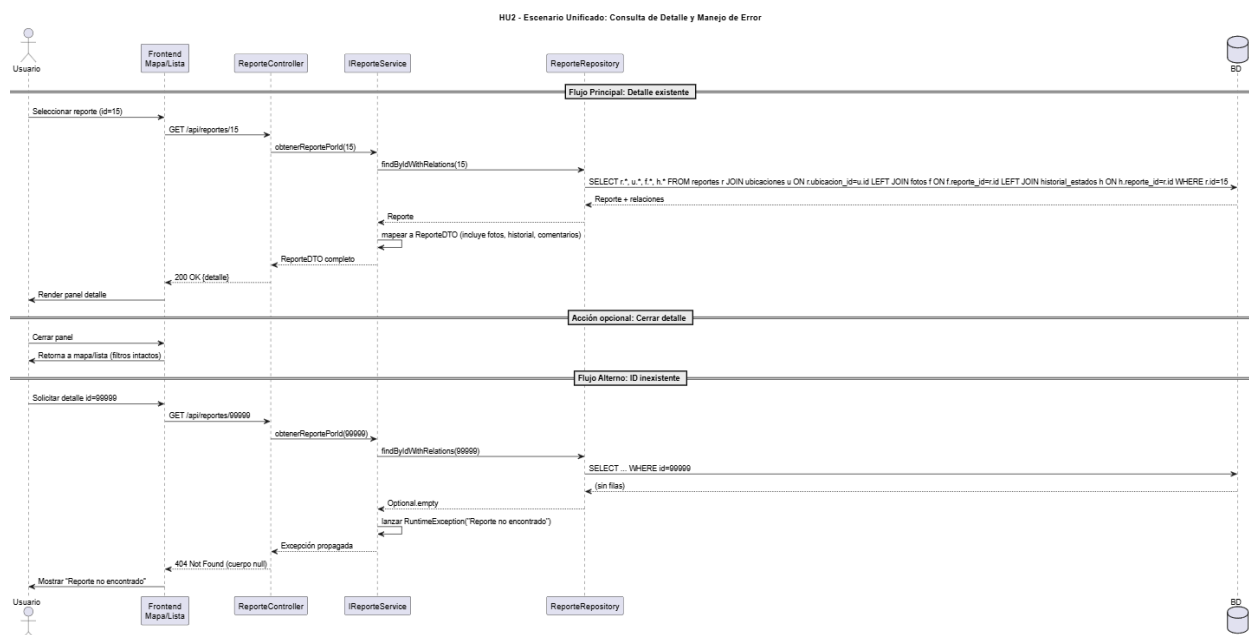
Actor: Usuario autenticado

Precondición general: Usuario ha iniciado sesión y se encuentra en la vista de mapa o lista (ambas contienen identificadores de reportes).

Secuencia de pasos posibles dentro del mismo flujo:

1. Solicitar el detalle (GET /api/reportes/{id}).
2. Cerrar el panel de detalle (frontend conserva filtros/posición).
3. Intentar acceder a un ID inexistente (flujo de error).

Postcondición global: El usuario accedió al detalle enriquecido de un reporte o recibió feedback claro cuando el reporte no existe, manteniendo su contexto de exploración.



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

HU Validar Reportes Pendientes

1. Escenario Unificado (Narrativa Global)

Escenario Unificado: "Operador Municipal revisa reportes pendientes, decide aprobarlos para pasar a revisión o rechazarlos con motivo, gestionando casos de error y acceso restringido futuro."

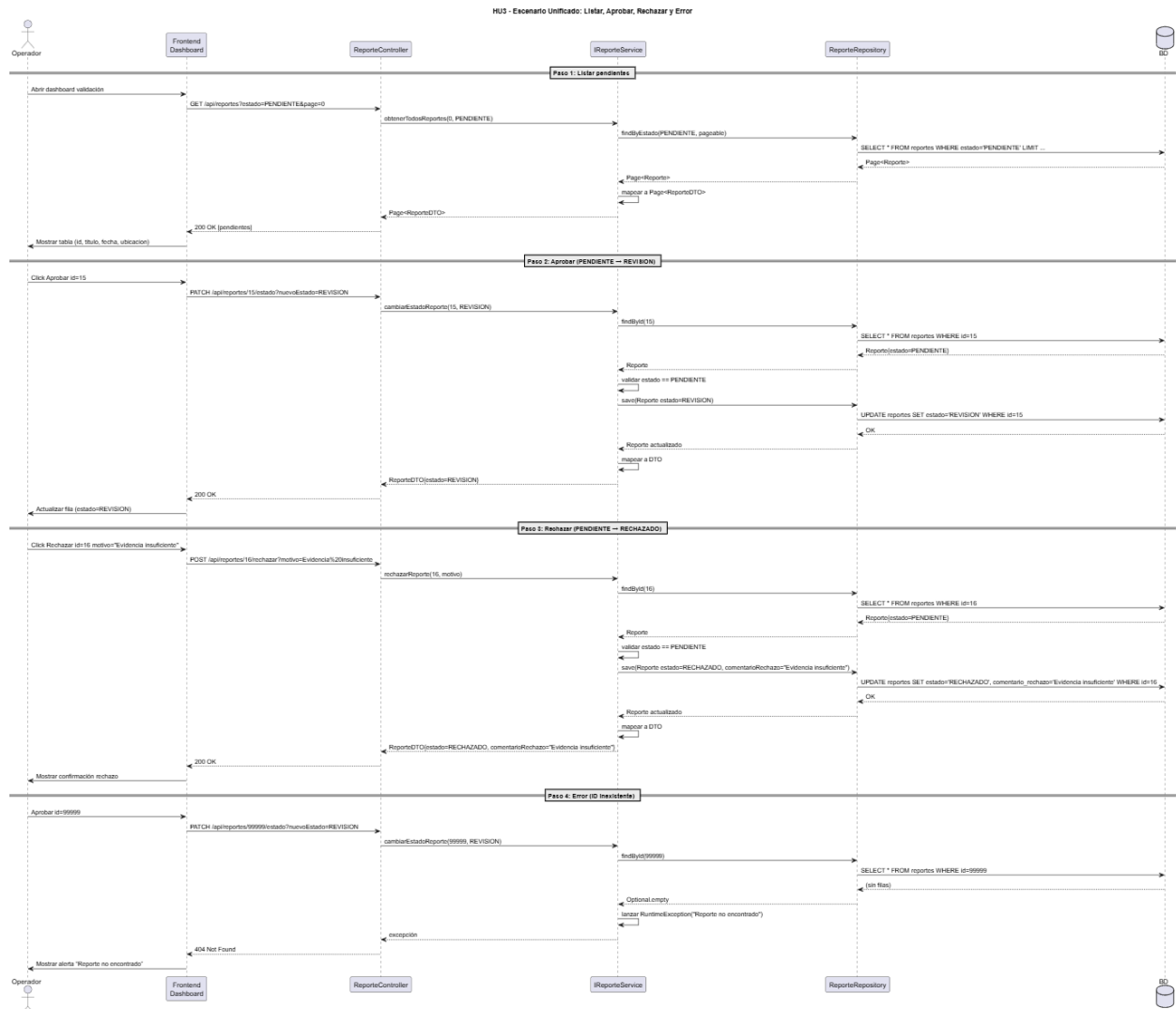
Actor: Operador Municipal

Precondición general: Operador autenticado y reportes en estado PENDIENTE existen en el sistema.

Secuencia de pasos dentro del flujo:

1. Listar reportes pendientes.
2. Aprobar un reporte (PENDIENTE → REVISION).
3. Rechazar un reporte (PENDIENTE → RECHAZADO) con motivo.
4. Manejar intento de validar ID inexistente.

Postcondición global: Uno o más reportes cambian su estado con trazabilidad, o el operador recibe mensajes claros ante errores.



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

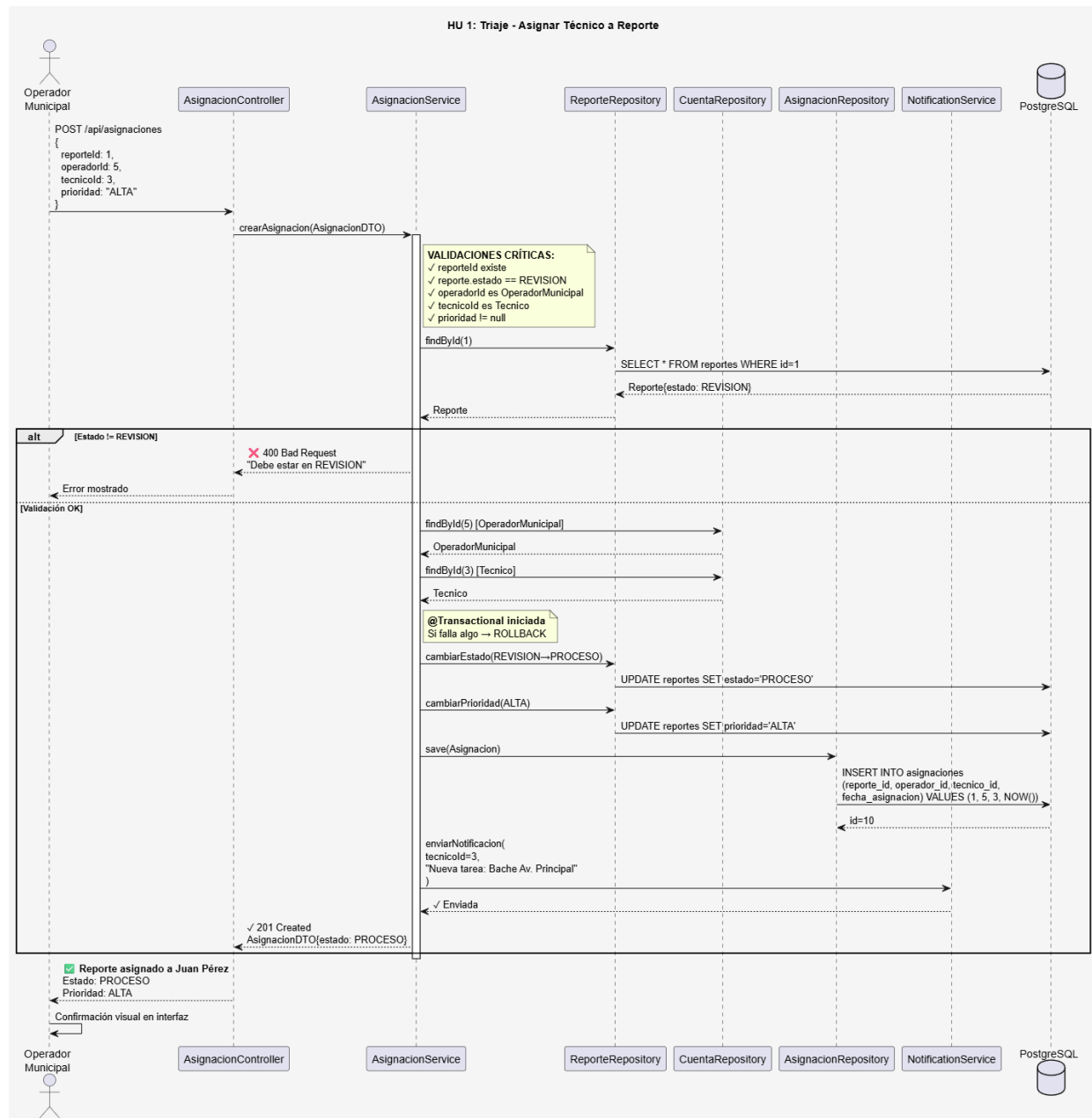
HU Asignar Reporte a Técnico

Escenario: "Asignar prioridad y técnico a reporte REVISION"

Actor: Operador Municipal

Precondición: Reporte en estado REVISION

Postcondición: Reporte pasa a PROCESO, técnico asignado, notificado



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

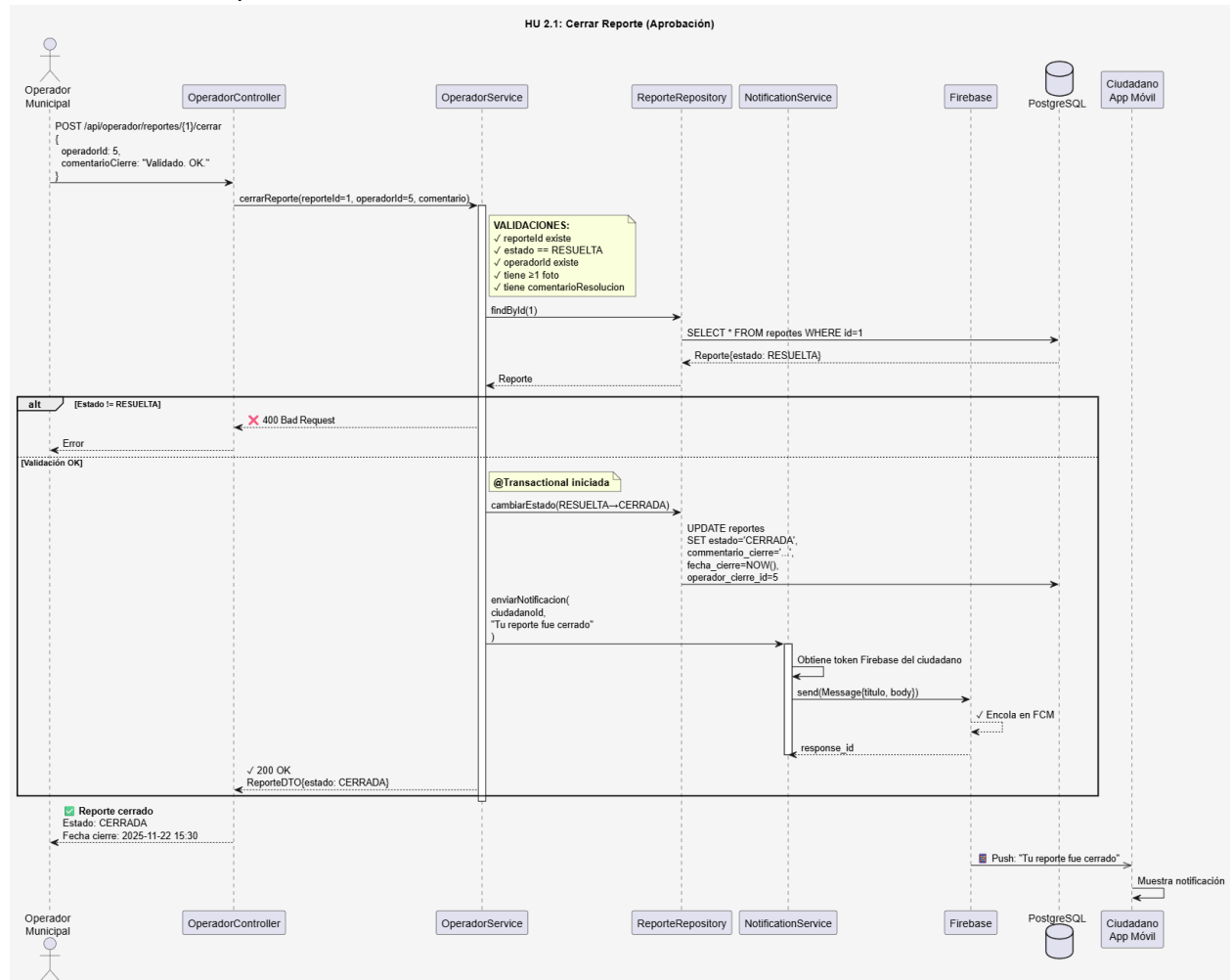
HU Cerrar Reporte (Control de Calidad)

Escenario 1: "Cerrar reporte RESUELTA con aprobación"

Actor: Operador Municipal

Precondición: Reporte en estado RESUELTA con fotos y comentario

Postcondición: Reporte CERRADA, notificación al ciudadano



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

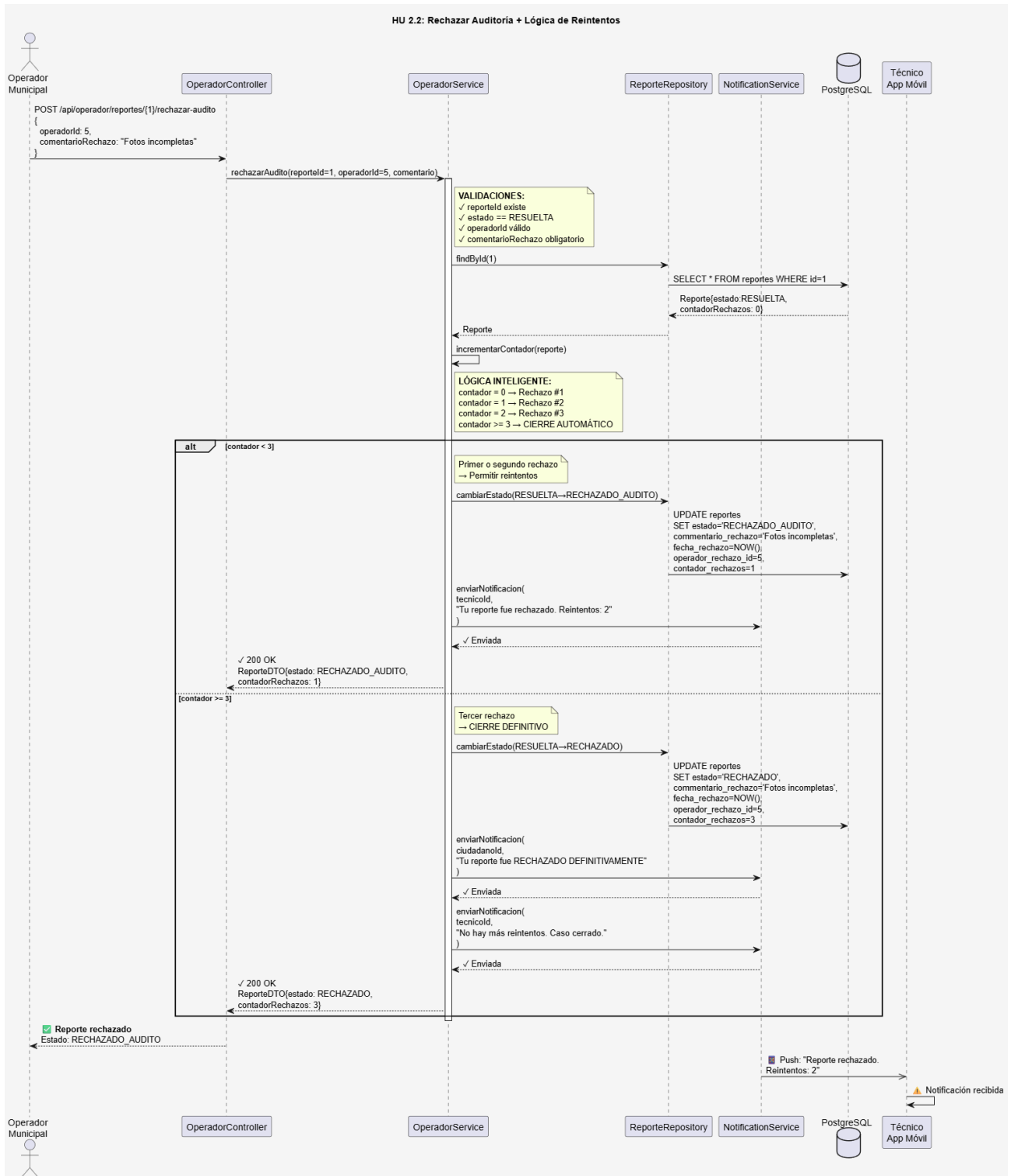
Escenario 2: "Operador rechaza auditoría - Rechazo Audito (reintentos)"

Actor: Operador Municipal

Precondición: Reporte en estado RESUELTA

Postcondición: Reporte pasa a RECHAZADO_AUDITO (si contador < 3) o RECHAZADO (si contador >= 3)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

HU Mis Tareas - Visualizar y Actualizar Estado

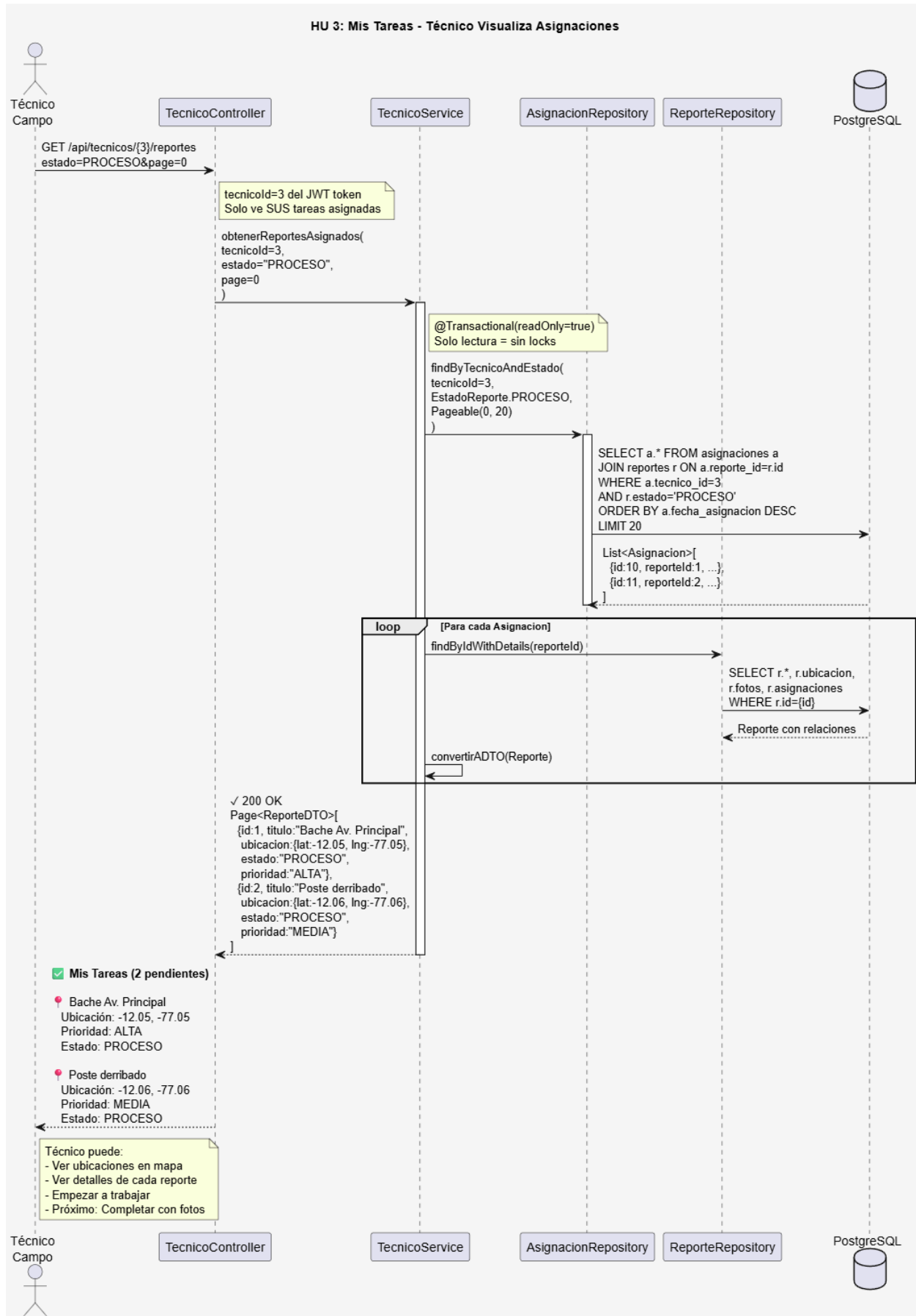
Escenario: "Técnico visualiza tareas en PROCESO"

Actor: Técnico de Campo

Precondición: Sesión iniciada

Postcondición: Lista de reportes asignados con estado PROCESO

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

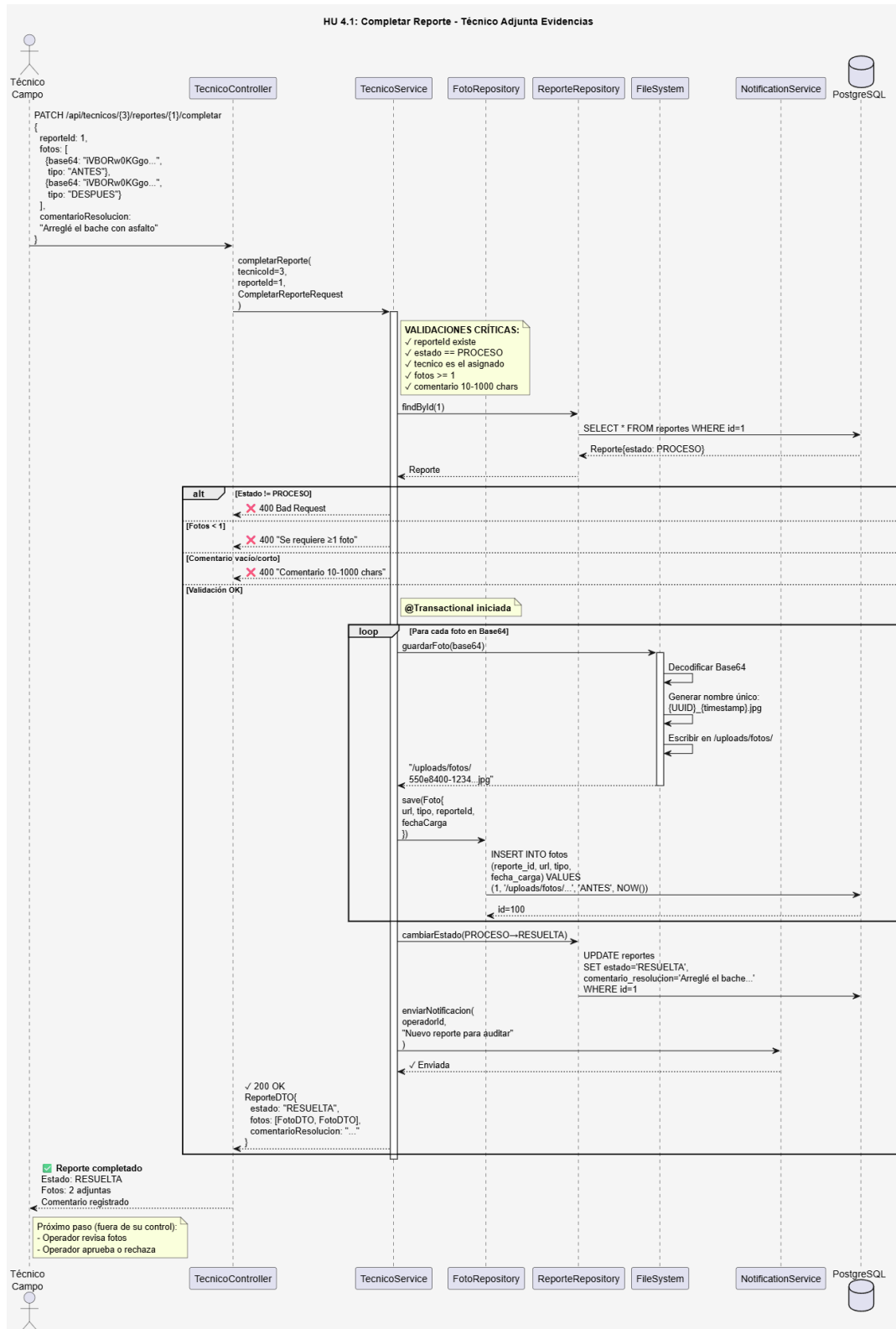
HU Completar Reporte con Evidencias y Comentarios

Escenario 1: "Técnico completa reporte con fotos y comentario"

Actor: Técnico de Campo

Precondición: Reporte en estado PROCESO

Postcondición: Reporte pasa a RESUELTA, fotos adjuntas, comentario registrado



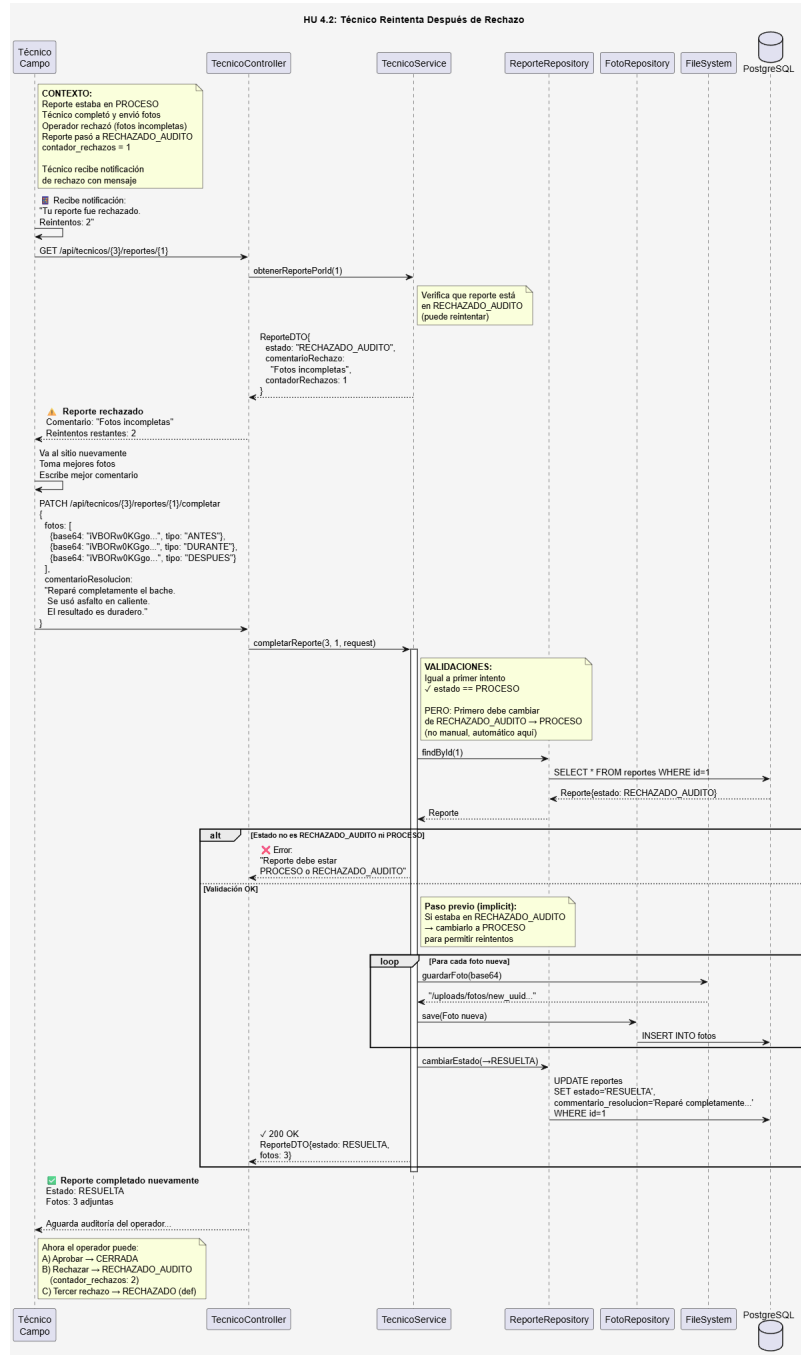
ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Escenario 2: "Operador rechaza fotos - Técnico reintententa"

Actor: Operador rechaza, Técnico reintententa

Precondición: Reporte en RECHAZADO_AUDITO

Postcondición: Reporte vuelve a RESUELTA con nuevas fotos



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

6. Interfaces de usuario y modelo de datos

Descripciones de Interfaces - Sprint 1

6.1 Interfaces del sistema

Se proporciona lo siguiente como ejemplo:

6.1.1 Interfaz de Inicio de Sesión (Login)

Permite autenticar a los usuarios mediante nombre de usuario y contraseña. Valida credenciales con `/api/auth/login` usando `AuthService`. Si son correctas, redirige según el rol (Ciudadano, Técnico u Operador); si no, muestra error. Incluye validaciones de campos obligatorios y manejo de error HTTP 401.

6.1.2 Interfaz de Registro de Cuenta

Formulario unificado para crear cuentas de cualquier tipo. Solicita datos personales y de autenticación, con validaciones (`CuentaDTO`, Bean Validation). Usa `CuentaFactoryProvider` (patrón Abstract Factory) para crear la instancia correspondiente. Hashea contraseñas con BCrypt y valida duplicados (usuario, DNI, correo), mostrando mensajes específicos.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

The image shows a registration form with the following fields and labels:

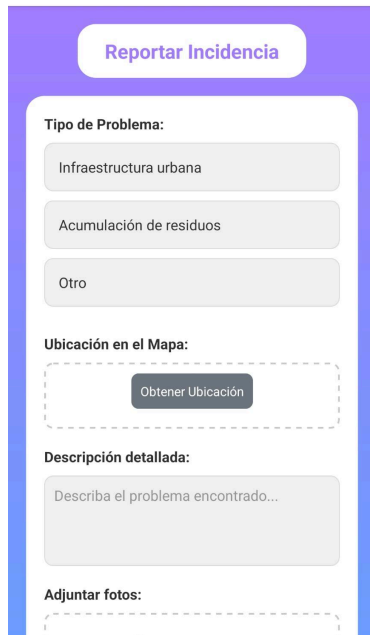
- Inicio Sesión** / **Registrarse** (active)
- Usuario:** Nombre de usuario único
- Nombres:** Tus nombres
- Apellidos:** Tus apellidos
- DNI:** 12345678
- Teléfono:** 987654321
- Correo Electrónico:** tu@email.com
- Contraseña:** Mínimo 6 caracteres
- Repite tu contraseña:** Repite tu contraseña
- Registrarse** (button)

6.1.3 Interfaz de Visualización de Reportes Propios (Ciudadano)

Permite al ciudadano ver todos sus reportes. Muestra lista paginada (10 por página) con detalles de estado, prioridad y fechas. Incluye filtros por estado y orden cronológico. Al seleccionar un reporte, muestra ubicación GPS, dirección e historial de cambios. Usa `/api/reportes/cuenta/{id}?page=n` con paginación y actualización dinámica.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Crear Reporte



Reportar Incidencia

Tipo de Problema:

Infraestructura urbana

Acumulación de residuos

Otro

Ubicación en el Mapa:

Obtener Ubicación

Descripción detallada:

Describe el problema encontrado...

Adjuntar fotos:

6.1.5 Interfaz de Gestión de Reportes (Operador Municipal)

Vista para operadores municipales con todos los reportes del sistema. Permite filtrar por estado, prioridad, fecha y ubicación. Resalta reportes en *REVISION*. El operador puede cambiar estado, modificar prioridad y asignar técnicos. Incluye modal de asignación, paginación y actualización en tiempo real desde `/api/reportes?estado=REVISION&page=n`.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Panel de Operador


Operador: Flow Operador

Acumulación de residuos

Estado: **Pendiente**

Revisar

Prioridad: MEDIA
Fecha: 16 oct. 2025
Ubicación: -11.957470, -77.083431

Problema de infraestructura urbana

Estado: **Pendiente**

Revisar

Prioridad: MEDIA
Fecha: 16 oct. 2025
Ubicación: -11.957469, -77.083431

Otro problema

Estado: **Pendiente**

Revisar

Prioridad: MEDIA
Fecha: 15 oct. 2025
Ubicación: -11.957529, -77.083450

Flujo: Basura acumulada

Estado: **En Proceso**

Anterior

Página 1 de 2

Siguiente

6.1.6 Interfaz de Selección de Técnico

Vista que muestra técnicos disponibles para asignar a un reporte. Lista paginada con nombre, número de asignaciones y especialidad. Permite filtrar o buscar por nombre. Al confirmar, crea asignación con POST `/api/asignaciones`, valida el estado del reporte y actualiza a *PROCESO*. Soporta reasignaciones y casos sin técnico (triaje por etapas)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

← Asignar Técnico

Asignar Técnico

Reporte #3

Flow Tecnico

Técnico Municipal

Asignar

Flow Tecnico

Técnico Municipal

Asignar

Anterior

Página 1 de 1

Siguiente

N&O(R)

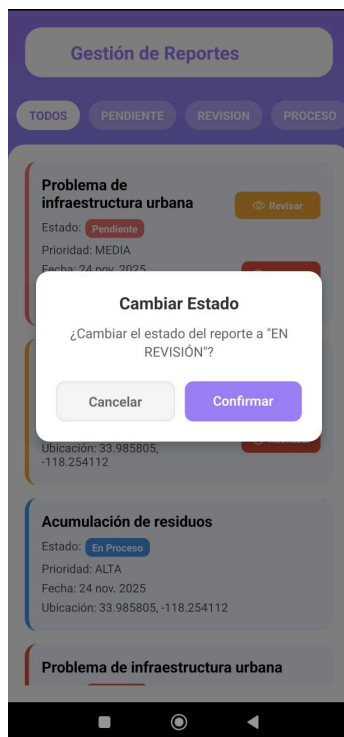
Página 89 de 168

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripciones de Interfaces - Sprint 2

6.2.1 Interfaz de Cambio de reporte

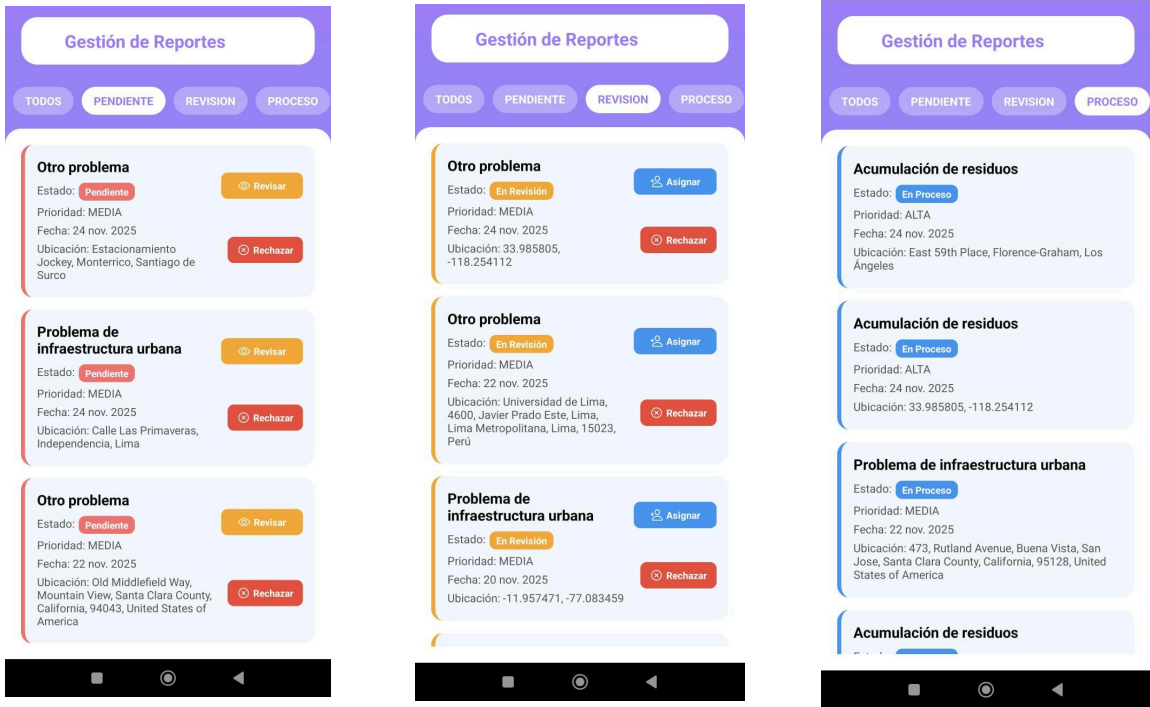
Interfaz modal que permite al usuario autorizado (Operador) modificar el estado de una incidencia. El sistema presenta una ventana emergente ("Cambiar Estado") que confirma la transición del flujo de trabajo (por ejemplo, cambiar a "EN REVISIÓN"). Incluye botones de acción clara ("Cancelar", "Confirmar") para asegurar la intencionalidad del cambio antes de persistir la transacción en la base de datos.



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

6.2.2 Interfaz de Filtrado por estado

Vista de gestión avanzada para el Operador Municipal. Permite visualizar la lista completa de reportes y filtrarlos rápidamente mediante pestañas de estado (Todos, Pendiente, Revisión, Proceso). Cada tarjeta de reporte muestra información resumida crítica para la toma de decisiones: título del problema, etiqueta de estado, etiqueta de prioridad (con código de color: Rojo/ALTA, Naranja/MEDIA), fecha de creación y coordenadas de ubicación, facilitando la priorización del trabajo.

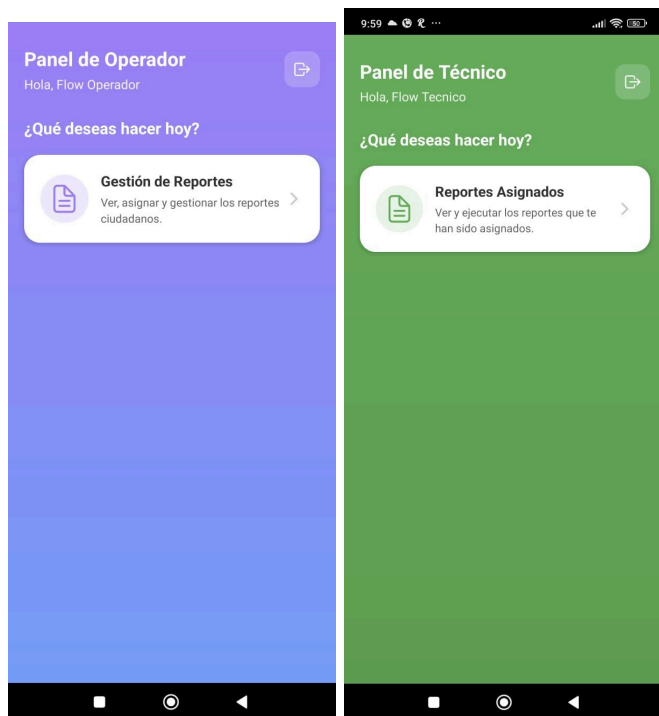


6.2.3 Interfaz de Vista de operador y técnico

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Pantallas de inicio (Dashboard) diferenciadas según el rol autenticado en el sistema (RBAC).

- Panel de Operador: Muestra opciones administrativas como "Gestión de Reportes" para validar, rechazar o asignar incidencias.
- Panel de Técnico: Muestra opciones operativas como "Reportes Asignados", permitiendo al técnico acceder directamente a su cola de trabajo personal y ejecutar las órdenes de servicio.

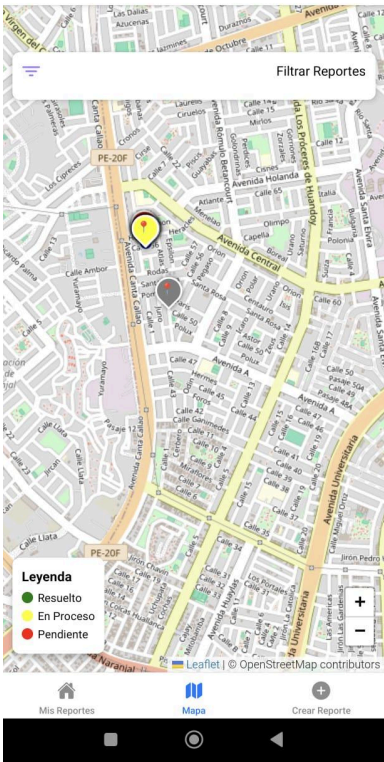


6.2.4 Interfaz de Vista mapa, detalle de reporte en el mapa

Interfaz de visualización geoespacial que permite ubicar las incidencias urbanas mediante marcadores en un mapa interactivo. Al seleccionar un marcador específico, el sistema despliega un panel inferior o modal con el detalle completo del reporte, permitiendo visualizar el contexto geográfico (dirección y coordenadas) junto con la información administrativa (título, estado, prioridad y descripción) sin perder la vista del mapa

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

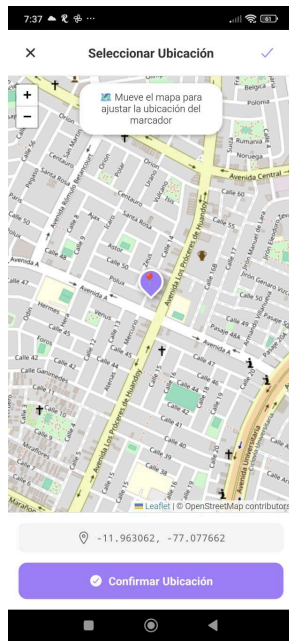
Mapa



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

6.2.5 Interfaz de Mostrar ubicación en la creación del reporte

Herramienta integrada en el flujo de "Crear Reporte" que permite al ciudadano definir la ubicación exacta de la incidencia. El usuario puede interactuar con el mapa para mover el marcador y ajustar la posición (latitud y longitud) con precisión antes de confirmar la ubicación. Incluye un botón de confirmación ("Confirmar Ubicación") para validar las coordenadas capturadas.



6.2.6 Interfaz de Vista de reportes asignados del técnico

Lista filtrada diseñada exclusivamente para el Técnico de Campo. Muestra únicamente los reportes que se encuentran en estado "En Proceso" y que han sido asignados a su cuenta.

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Mis Reportes Asignados

PROCESO

RESUELTA

RECHAZADO_AUDITO

Acumulación de residuos

Estado:

En Proceso

Fecha: 24 nov. 2025

Ubicación: Sin ubicación

Atender

Acumulación de residuos

Estado:

En Proceso

Fecha: 24 nov. 2025

Ubicación: Sin ubicación

Atender

Acumulación de residuos

Estado:

En Proceso

Fecha: 22 nov. 2025

Ubicación: Sin ubicación

Atender

Acumulación de residuos

Estado:

En Proceso

Fecha: 24 nov. 2025

Ubicación: Sin ubicación

Atender

Problema de

6.2.7 Interfaz de Vista de historial

Vista detallada que presenta la información consolidada de un reporte específico. Organiza los datos en secciones claras ("Información General", "Descripción", "Ubicación", "Ciudadano") y permite visualizar el estado actual del reporte (ej. "Rechazado" o "En Proceso"). Esta interfaz sirve como base para consultar la trazabilidad y los cambios realizados sobre la incidencia a lo largo de su ciclo de vida

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

←

Historial de Cambios

24/11/2025 12:21:50 a. m.

Cambio a **REVISION**

Anterior: PENDIENTE

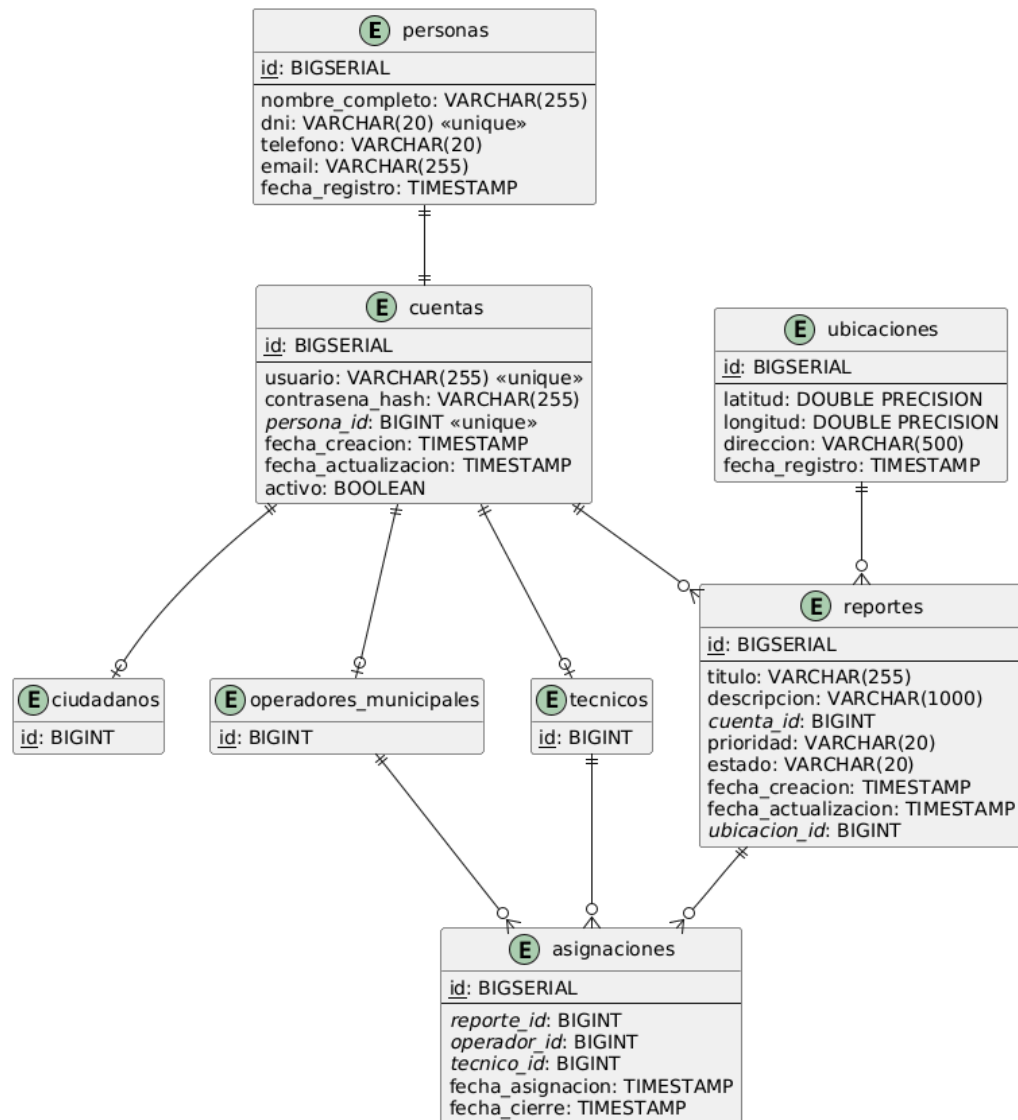
24/11/2025 12:21:35 a. m.

Reporte Creado

6.2 Modelo de Base de Datos

Modelo de Datos del Sprint 1:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Modelo de Datos del Sprint 2:

Esta sección describe el esquema de la base de datos relacional que ReportaYA utiliza para persistir datos permanentemente. El sistema utiliza PostgreSQL como motor de BD relacional, con herencia de datos modelada a través de JPA (Jakarta Persistence).

La BD está organizada en 4 áreas funcionales principales:

Gestión de Cuentas y Personas

Reportes e Incidencias

Asignaciones y Tareas

Auditoría y Notificaciones

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

nombres	VARCHAR(255)	NOT NULL	Nombres del usuario
apellidos	VARCHAR(255)	NOT NULL	Apellidos del usuario
dni	VARCHAR(8)	NOT NULL, UNIQUE	Documento de identidad (único)
telefono	VARCHAR(20)	NOT NULL	Número de contacto
correo	VARCHAR(255)	NOT NULL, UNIQUE	Email (único)

Índices:

```
CREATE UNIQUE INDEX idx_personas_dni ON personas(dni);
CREATE UNIQUE INDEX idx_personas_correo ON personas(correo);
```

Justificación: Separar datos personales de datos de autenticación permite reutilizar información para múltiples cuentas si fuera necesario.

2. Tabla: cuentas (Herencia JOINED)

Propósito: Almacenar datos de autenticación y pertenencia de usuarios.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
usuario	VARCHAR(50)	NOT NULL, UNIQUE	Nombre de usuario para login
contrasena_hash	VARCHAR(255)	NOT NULL	Hash bcrypt de contraseña
persona_id	BIGINT	FK, NOT NULL, UNIQUE	Ref a personas (1:1)
fecha_creacion	TIMESTAMP	NOT NULL	Cuándo se creó la cuenta
fecha_actualizacion	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Última actualización
activo	BOOLEAN	NOT NULL	Cuenta habilitada/deshabilitada
dtype	VARCHAR(31)	NOT NULL	Discriminador: Ciudadano Tecnico OperadorMunic

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

			ipal
--	--	--	------

Tablas Hijas (Herencia JOINED):

-- Tabla ciudadanos (hereda de cuentas)

```
CREATE TABLE ciudadanos (
  id BIGINT PRIMARY KEY,
  FOREIGN KEY (id) REFERENCES cuentas(id) ON DELETE CASCADE
);
```

-- Tabla tecnicos (hereda de cuentas)

```
CREATE TABLE tecnicos (
  id BIGINT PRIMARY KEY,
  FOREIGN KEY (id) REFERENCES cuentas(id) ON DELETE CASCADE
);
```

-- Tabla operadores_municipales (hereda de cuentas)

```
CREATE TABLE operadores_municipales (
  id BIGINT PRIMARY KEY,
  FOREIGN KEY (id) REFERENCES cuentas(id) ON DELETE CASCADE
);
```

Justificación de Herencia JOINED:

- Relaciones polimórficas: reportes.cuenta_id puede referirse a cualquier subtipo
- Queries eficientes: JOIN automático cuando se necesita
- Datos compartidos: usuario, contraseña, persona en tabla padre

Índices:

```
CREATE UNIQUE INDEX idx_cuentas_usuario ON cuentas(usuario);
CREATE UNIQUE INDEX idx_cuentas_persona_id ON cuentas(persona_id);
CREATE INDEX idx_cuentas_dtype ON cuentas(dtype);
```

3. Tabla: ubicaciones

Propósito: Almacenar ubicaciones geográficas (coordenadas GPS + dirección).

Columna	Tipo	Restricciones	Descripción
---------	------	---------------	-------------

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

id	BIGINT	PK, AUTO_INCREMENT	Identificador único
latitud	DOUBLE	NOT NULL	Coordenada latitud
longitud	DOUBLE	NOT NULL	Coordenada longitud
direccion	VARCHAR(500)		Dirección legible
fecha_registro	TIMESTAMP	NOT NULL	Cuándo se registró

Justificación: Separar ubicación permite reutilizarla si múltiples reportes fueran del mismo lugar, aunque actualmente es 1:1 con reportes.

4. Tabla: reportes (Núcleo del Sistema)

Propósito: Almacenar los reportes de incidencias urbanas (datos más críticos).

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
titulo	VARCHAR(255)	NOT NULL	Título del reporte
descripcion	VARCHAR(1000)	NOT NULL	Descripción detallada
cuenta_id	BIGINT	FK, NOT NULL	Ciudadano que reporta
ubicacion_id	BIGINT	FK, NOT NULL, UNIQUE	Ubicación (1:1)
estado	VARCHAR(30)	NOT NULL	Estado actual (enum: PENDIENTE, REVISION, PROCESO, RESUELTA, CERRADA, RECHAZADO_AUDITO, RECHAZADO)
prioridad	VARCHAR(10)		Prioridad (BAJA, MEDIA, ALTA)
comentario_resolucion	VARCHAR(1000)		Comentario del técnico
comentario_cierre	VARCHAR(1000)		Comentario del operador (cierre)
fecha_cierre	TIMESTAMP		Cuándo cerró
operador_cierre_id	BIGINT	FK	Operador que cerró
comentario_rechazo	VARCHAR(1000)		Comentario de rechazo
fecha_rechazo	TIMESTAMP		Cuándo rechazó
operador_rechazo_id	BIGINT	FK	Operador que rechazó

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

contador_rechazos	INT	DEFAULT 0	Cuántas veces rechazado (auditoría)
fecha_creacion	TIMESTAMP	NOT NULL	Cuándo se creó
fecha_actualizacion	TIMESTAMP		Última actualización

Índices:

```
CREATE INDEX idx_reportes_cuenta_id ON reportes(cuenta_id);
CREATE INDEX idx_reportes_estado ON reportes(estado);
CREATE INDEX idx_reportes_prioridad ON reportes(prioridad);
CREATE INDEX idx_reportes_fecha_creacion ON reportes(fecha_creacion);
```

5. Tabla: fotos

Propósito: Almacenar evidencias fotográficas adjuntas a reportes.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

reporte_id	BIGINT	FK, NOT NULL	Reporte al que pertenece
url	VARCHAR(500)	NOT NULL	Ruta del archivo en disco (/uploads/fotos/{uuid}.jpg)
tipo	VARCHAR(20)	NOT NULL	Tipo (ANTES, DURANTE, DESPUES)
descripcion	VARCHAR(255)		Descripción opcional
fecha_carga	TIMESTAMP	NOT NULL	Cuándo se subió

Constraint ON DELETE: CASCADE (si reporte elimina, fotos se eliminan)

Justificación:

- Mínimo 1 foto es obligatorio para marcar RESUELTA
- Separar en tabla permite auditoría de evidencias
- URLs apuntan a disco local /uploads/fotos/

6. Tabla: asignaciones

Propósito: Registrar asignación de reportes a técnicos (con operador supervisor).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

reporte_id	BIGINT	FK, NOT NULL	Reporte asignado
operador_id	BIGINT	FK, NOT NULL	Operador que asigna
tecnico_id	BIGINT	FK, NOT NULL	Técnico responsable
fecha_asignacion	TIMESTAMP	NOT NULL	Cuándo se asignó
fecha_cierre	TIMESTAMP		Cuándo cerró (técnico completó)
comentario_resolucion	VARCHAR(1000)		Comentario de resolución

Justificación:

- Auditoría: quién asignó a quién
- Rastreo: cuándo se completó (SLA)
- Permite múltiples asignaciones si reporte vuelve a PROCESO

7. Tabla: token_notificaciones

Propósito: Almacenar tokens Firebase para notificaciones push.

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

cuenta_id	BIGINT	FK, NOT NULL, UNIQUE	Cuenta (1:1)
token	VARCHAR(512)	NOT NULL	Token FCM de Firebase

Justificación:

- 1 token por cuenta (actualiza con cada login)
- Permite enviar push a usuarios específicos
- Separado para privacidad

8. Tabla: mensaje_notificaciones

Propósito: Lookup table para mensajes predefinidos por estado (configurable).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
estado	VARCHAR(30)	NOT NULL, UNIQUE	Estado del reporte
mensaje	VARCHAR(500)	NOT NULL	Mensaje a enviar

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

9. Tabla: *historial_estados*

Propósito: Auditoría: registrar todas las transiciones de estado (immutable).

Columna	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO_INCREMENT	Identificador único
reporte_id	BIGINT	FK, NOT NULL	Reporte auditado
estado_anterior	VARCHAR(30)		Estado previo
estado_nuevo	VARCHAR(30)	NOT NULL	Nuevo estado
fecha_cambio	TIMESTAMP	NOT NULL	Cuándo cambió

Justificación:

- Inmutable (solo INSERT, nunca UPDATE/DELETE)
- Permite análisis histórico
- Cumple con auditoría regulatoria

Índices:

CREATE INDEX idx_historial_reporte_id ON historial_estados(reporte_id);

CREATE INDEX idx_historial_fecha_cambio ON historial_estados(fecha_cambio);

6.3 Script De La Base De Datos

```
-- 1) Personas
CREATE TABLE IF NOT EXISTS personas (
  id BIGSERIAL PRIMARY KEY,
  nombres VARCHAR(255) NOT NULL,
  apellidos VARCHAR(255) NOT NULL,
  dni VARCHAR(50) NOT NULL UNIQUE,
  telefono VARCHAR(50) NOT NULL,
  correo VARCHAR(255) NOT NULL UNIQUE,
  fecha_creacion TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  fecha_actualizacion TIMESTAMP WITHOUT TIME ZONE DEFAULT
CURRENT_TIMESTAMP
);
```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```
-- 2) Cuentas (tabla base para herencia JOINED)
CREATE TABLE IF NOT EXISTS cuentas (
  id BIGSERIAL PRIMARY KEY,
  usuario VARCHAR(255) NOT NULL UNIQUE,
  contrasena_hash VARCHAR(255) NOT NULL,
  persona_id BIGINT NOT NULL UNIQUE,
  fecha_creacion TIMESTAMP WITHOUT TIME ZONE,
  fecha_actualizacion TIMESTAMP WITHOUT TIME ZONE,
  activo BOOLEAN NOT NULL DEFAULT TRUE,
  CONSTRAINT fk_cuentas_persona FOREIGN KEY (persona_id) REFERENCES personas
(id) ON DELETE RESTRICT
);

-- 3) Tablas de subclases (JOINED): operadores_municipales, tecnicos, ciudadanos
CREATE TABLE IF NOT EXISTS operadores_municipales (
  id BIGINT PRIMARY KEY,
  CONSTRAINT fk_op_mun_cuenta FOREIGN KEY (id) REFERENCES cuentas (id) ON
DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS tecnicos (
  id BIGINT PRIMARY KEY,
  CONSTRAINT fk_tecnicos_cuenta FOREIGN KEY (id) REFERENCES cuentas (id) ON
DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS ciudadanos (
  id BIGINT PRIMARY KEY,
  CONSTRAINT fk_ciudadanos_cuenta FOREIGN KEY (id) REFERENCES cuentas (id) ON
DELETE CASCADE
);

-- 4) Ubicaciones
CREATE TABLE IF NOT EXISTS ubicaciones (
  id BIGSERIAL PRIMARY KEY,
  direccion VARCHAR(500) NOT NULL,
  latitud DOUBLE PRECISION,
  longitud DOUBLE PRECISION,
  distrito VARCHAR(255),
  provincia VARCHAR(255),
  departamento VARCHAR(255),
  ciudad VARCHAR(255),
  pais VARCHAR(255) DEFAULT 'Perú',
  fecha_registro TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- 5) Reportes (con campos de evidencias y auditoría)
```


ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

CREATE TABLE IF NOT EXISTS reportes (
  id BIGSERIAL PRIMARY KEY,
  titulo VARCHAR(200) NOT NULL,
  descripcion TEXT NOT NULL,
  cuenta_id BIGINT NOT NULL,
  ubicacion_id BIGINT NOT NULL,
  tipo_problema VARCHAR(50) NOT NULL,
  prioridad VARCHAR(20),
  estado VARCHAR(30) NOT NULL DEFAULT 'PENDIENTE',
  fecha_creacion TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  fecha_actualizacion TIMESTAMP WITHOUT TIME ZONE DEFAULT
CURRENT_TIMESTAMP,
  -- Campos de evidencias (ESCENARIO 1 - Técnico)
  comentario_resolucion VARCHAR(1000),
  -- Campos de auditoría (ESCENARIO 2 y 3 - Operador)
  comentario_cierre VARCHAR(1000),
  fecha_cierre TIMESTAMP WITHOUT TIME ZONE,
  operador_cierre_id BIGINT,
  comentario_rechazo VARCHAR(1000),
  fecha_rechazo TIMESTAMP WITHOUT TIME ZONE,
  operador_rechazo_id BIGINT,
  contador_rechazos INT DEFAULT 0,
  CONSTRAINT fk_reportes_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id)
ON DELETE CASCADE,
  CONSTRAINT fk_reportes_ubicacion FOREIGN KEY (ubicacion_id) REFERENCES
ubicaciones (id) ON DELETE CASCADE,
  CONSTRAINT fk_reportes_operador_cierre FOREIGN KEY (operador_cierre_id)
REFERENCES operadores_municipales (id) ON DELETE SET NULL,
  CONSTRAINT fk_reportes_operador_rechazo FOREIGN KEY (operador_rechazo_id)
REFERENCES operadores_municipales (id) ON DELETE SET NULL,
  CONSTRAINT chk_estado_reporte CHECK (estado IN ('PENDIENTE', 'REVISION',
'PROCESO', 'RESUELTA', 'CERRADA', 'RECHAZADO_AUDITO', 'RECHAZADO'))
);

```

-- 6) Fotos (evidencias fotográficas - OBLIGATORIAS)

```

CREATE TABLE IF NOT EXISTS fotos (
  id BIGSERIAL PRIMARY KEY,
  reporte_id BIGINT NOT NULL,
  url VARCHAR(1000) NOT NULL,
  tipo VARCHAR(20) NOT NULL CHECK (tipo IN ('INICIAL', 'PROCESO', 'FINAL')),
  descripcion VARCHAR(500),
  fecha_carga TIMESTAMP WITHOUT TIME ZONE NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  CONSTRAINT fk_fotos_reporte FOREIGN KEY (reporte_id) REFERENCES reportes (id)
ON DELETE CASCADE
);

```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```
-- 7) Asignaciones
CREATE TABLE IF NOT EXISTS asignaciones (
  id BIGSERIAL PRIMARY KEY,
  reporte_id BIGINT NOT NULL,
  operador_id BIGINT NOT NULL,
  tecnico_id BIGINT,
  prioridad VARCHAR(20),
  comentario_resolucion VARCHAR(1000),
  fecha_asignacion TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  fecha_cierre TIMESTAMP WITHOUT TIME ZONE,
  CONSTRAINT fk_asignaciones_reporte FOREIGN KEY (reporte_id) REFERENCES
reportes (id) ON DELETE CASCADE,
  CONSTRAINT fk_asignaciones_operador FOREIGN KEY (operador_id) REFERENCES
operadores_municipales (id) ON DELETE RESTRICT,
  CONSTRAINT fk_asignaciones_tecnico FOREIGN KEY (tecnico_id) REFERENCES
tecnicos (id) ON DELETE SET NULL
);

-- 8) Historial de Estados (auditoría de cambios)
CREATE TABLE IF NOT EXISTS historial_estados (
  id BIGSERIAL PRIMARY KEY,
  reporte_id BIGINT NOT NULL,
  estado_anterior VARCHAR(30),
  estado_nuevo VARCHAR(30) NOT NULL,
  razon VARCHAR(500),
  fecha_cambio TIMESTAMP WITHOUT TIME ZONE NOT NULL DEFAULT
CURRENT_TIMESTAMP,
  CONSTRAINT fk_historial_reporte FOREIGN KEY (reporte_id) REFERENCES reportes (id)
ON DELETE CASCADE
);

-- 9) Token de Notificaciones (FCM tokens para Firebase)
CREATE TABLE IF NOT EXISTS token_notificaciones (
  id BIGSERIAL PRIMARY KEY,
  cuenta_id BIGINT NOT NULL UNIQUE,
  token_fcm VARCHAR(500) NOT NULL UNIQUE,
  fecha_registro TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
  fecha_expiracion TIMESTAMP WITHOUT TIME ZONE,
  activo BOOLEAN DEFAULT TRUE,
  CONSTRAINT fk_token_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id) ON
DELETE CASCADE
);

-- 10) Mensajes de Notificación (plantillas por estado y notificaciones individuales)
CREATE TABLE IF NOT EXISTS mensaje_notificaciones (
  id BIGSERIAL PRIMARY KEY,
  cuenta_id BIGINT NOT NULL,
```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

reporte_id BIGINT,
tipo VARCHAR(50) NOT NULL,
titulo VARCHAR(200) NOT NULL,
cuerpo TEXT NOT NULL,
leida BOOLEAN DEFAULT FALSE,
fecha_creacion TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_mensaje_cuenta FOREIGN KEY (cuenta_id) REFERENCES cuentas (id)
ON DELETE CASCADE,
CONSTRAINT fk_mensaje_reporte FOREIGN KEY (reporte_id) REFERENCES reportes (id)
ON DELETE CASCADE
);

-- 11) Mensajes de Rechazo (motivos de rechazo de reportes)
CREATE TABLE IF NOT EXISTS rechazo_mensajes (
id BIGSERIAL PRIMARY KEY,
reporte_id BIGINT NOT NULL,
operador_id BIGINT NOT NULL,
contador_intento INT NOT NULL,
comentario_rechazo VARCHAR(1000) NOT NULL,
fecha_rechazo TIMESTAMP WITHOUT TIME ZONE DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT fk_rechazo_reporte FOREIGN KEY (reporte_id) REFERENCES reportes (id)
ON DELETE CASCADE,
CONSTRAINT fk_rechazo_operador FOREIGN KEY (operador_id) REFERENCES
operadores_municipales (id) ON DELETE SET NULL
);

-- 12) Índices para optimización de consultas
-- Índice parcial: impedir más de una asignación abierta por reporte
CREATE UNIQUE INDEX IF NOT EXISTS ux_asignaciones_reporte_abierta ON
asignaciones (reporte_id) WHERE fecha_cierre IS NULL;

-- Índices para reportes
CREATE INDEX IF NOT EXISTS idx_reportes_cuenta_id ON reportes (cuenta_id);
CREATE INDEX IF NOT EXISTS idx_reportes_ubicacion_id ON reportes (ubicacion_id);
CREATE INDEX IF NOT EXISTS idx_reportes_estado ON reportes (estado);
CREATE INDEX IF NOT EXISTS idx_reportes_fecha_creacion ON reportes (fecha_creacion
DESC);

-- Índices para asignaciones
CREATE INDEX IF NOT EXISTS idx_asignaciones_reporte_id ON asignaciones (reporte_id);
CREATE INDEX IF NOT EXISTS idx_asignaciones_tecnico_id ON asignaciones (tecnico_id);
CREATE INDEX IF NOT EXISTS idx_asignaciones_operador_id ON asignaciones
(operador_id);

-- Índices para fotos (evidencias)
CREATE INDEX IF NOT EXISTS idx_fotos_reporte_id ON fotos (reporte_id);
CREATE INDEX IF NOT EXISTS idx_fotos_tipo ON fotos (tipo);

```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```
-- Índices para historial de estados (auditoría)
CREATE INDEX IF NOT EXISTS idx_historial_reporte_id ON historial_estados (reporte_id);
CREATE INDEX IF NOT EXISTS idx_historial_fecha_cambio ON historial_estados
(fecha_cambio DESC);

-- Índices para notificaciones
CREATE INDEX IF NOT EXISTS idx_token_cuenta_id ON token_notificaciones (cuenta_id);

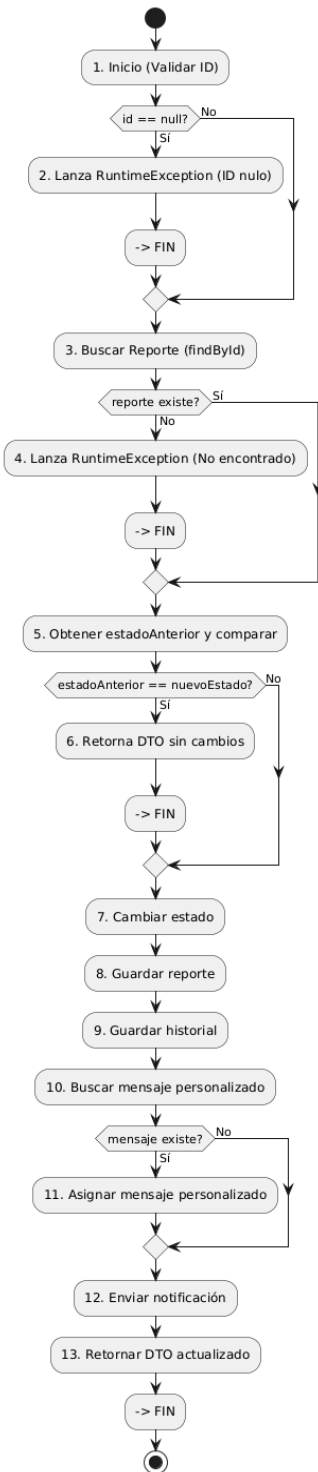
-- Índices para rechazo
CREATE INDEX IF NOT EXISTS idx_rechazo_reporte_id ON rechazo_mensajes (reporte_id);
```

7. Prueba de Software

7.1 Prueba de Caja Blanca

Prueba de Caja Blanca Renzo Cabezas

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Prueba de Caja Blanca: Cambio de Estado de Reporte

Objetivo

Se desea analizar mediante caja blanca (camino básico) el método `cambiarEstadoReporte(Long id, EstadoReporte nuevoEstado)` de la capa de servicio del Backend de ReportaYA.

Este método es responsable de:

1. Validar que el ID del reporte no sea nulo.
2. Verificar que el reporte exista en la base de datos.
3. Comprobar si el nuevo estado es diferente al actual.
4. Actualizar el estado del reporte y guardar el cambio.
5. Registrar el cambio en el historial de estados.
6. Verificar si existe un mensaje de notificación personalizado para el nuevo estado.
7. Enviar una notificación push al ciudadano.

Leyenda de Nodos

Nodo	Descripción
[1]	Punto de entrada al método (Validación ID)
[2]	Lanza excepción por ID nulo
[3]	Búsqueda de reporte (<code>findById</code>)
[4]	Lanza excepción por reporte no encontrado

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

[5]	Obtener estado anterior y comparar
[6]	Retornar DTO sin cambios (Fin temprano)
[7]	Cambiar estado
[8]	Guardar reporte
[9]	Guardar historial de estado
[10]	Buscar mensaje personalizado (findByEstado)
[11]	Asignar mensaje personalizado
[12]	Enviar notificación
[13]	Retornar DTO actualizado (Fin)

Predicados Identificados

P	Nodo	Predicado	Descripción
P1	[1]	id == null	Validación de entrada obligatoria

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

P2	[3]	!reporte.isPresent()	Validación de existencia del reporte
P3	[5]	estadoAnterior == nuevoEstado	Validación de cambio de estado real
P4	[10]	mensajeNotificacion.isPresent()	Decisión sobre usar mensaje personalizado

Cálculo de Complejidad Ciclomática

$$V(G) = P + 1 = 4 + 1 = 5$$

Tabla de Casos de Prueba (Caja Blanca)

Descripción del Camino	Condiciones de Entrada Relevantes	Resultado Esperado
ID nulo	id = null	RuntimeException: "El ID del reporte es obligatorio."
Reporte no encontrado	id = 999 (no existe en BD)	RuntimeException: "Reporte no encontrado."
Estado igual al actual (fin temprano)	estadoAnterior == nuevoEstado	Retorna DTO sin cambios; no guarda ni notifica
Cambio de estado sin mensaje personalizado	Nuevo estado ≠ actual; no existe mensaje en BD	Guarda, registra historial y notifica con mensaje por defecto

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Cambio de estado con mensaje personalizado	Nuevo estado $\neq$ actual; existe mensaje en BD	Guarda, registra historial y notifica con mensaje custom
--	--	--

Prueba de Caja Negra Renzo Cabezas

Funcionalidad: Visualización de Historial de Estado de Reporte

Ubicación del Endpoint:

- GET /reportes/{id}/historial

1. Descripción del Caso de Prueba

1.1 Descripción de la Funcionalidad

La funcionalidad permite que un ciudadano consulte el historial completo de cambios de estado de uno de sus reportes, mostrando la secuencia cronológica de todas las transiciones realizadas desde la creación del reporte hasta su estado actual.

1.2 Parámetros de Entrada

- id (Long): Identificador único del reporte cuyo historial se desea consultar.

1.3 Salida Esperada

La funcionalidad retorna una lista de objetos JSON con la siguiente estructura:

```

HistorialEstadoDTO {
    estadoAnterior: String, // Enum, puede ser null en el primer registro
    estadoNuevo: String,    // Enum del estado resultante del cambio
    fecha: LocalDateTime,   // Fecha y hora del cambio
    responsable: String     // Opcional: operador o técnico que realizó el cambio
}

```

1.4 Reglas Funcionales

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- El historial debe contener al menos un estado correspondiente a PENDIENTE al momento de la creación del reporte.
- Los estados deben aparecer ordenados cronológicamente de manera ascendente o descendente según lo definido por la interfaz de usuario.
- El reporte consultado debe existir en el sistema y pertenecer al ciudadano que realiza la consulta.

Clases de Equivalencia

Variable: ID del Reporte

Código	Clase de Equivalencia	Descripción	Tipo
C1	ID existente y perteneciente al usuario	El reporte existe y pertenece al ciudadano autenticado	Válida
C2	ID existente de otro usuario	El reporte existe, pero pertenece a otro ciudadano	Inválida
C3	ID no existente	No existe en la base de datos	Inválida
C4	ID nulo	No se envía el parámetro id	Inválida
C5	ID negativo	Valor menor a cero	Inválida
C6	ID cero	Valor igual a 0	Inválida

Variable: Historial del Reporte

Código	Clase de Equivalencia	Descripción	Tipo
--------	-----------------------	-------------	------

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

C7	Historial mínimo	Solo contiene estado PENDIENTE	Válida
C8	Historial con múltiples estados	Secuencia válida de estados	Válida
C9	Historial vacío	No tiene registros	Inválida

Variable: Secuencia de Estados

Código	Clase de Equivalencia	Descripción	Tipo
C10	Secuencia válida	Estados del Enum y orden cronológico correcto	Válida
C11	Secuencia inválida	Estados fuera del Enum o transiciones incorrectas	Inválida

Casos de Prueba Válidos

Código	Nombre	Objetivo	Precondiciones	Entrada	Clases	Resultado Esperado
CT-H01	Historial Inicial	Validar historial con un único estado	Reporte id=100, estado=PENDIENTE, pertenece al ciudadano	GET /reportes/100/historial	C1, C7, C10	200 OK; lista con 1 elemento; estadoAnterior=null; estadoNuevo=PENDIENTE

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

CT-H02	Historial con múltiples estados	Validar historial con varios cambios	Reporte id=100 con estados PENDIENTE→REVISION→PROCESO	GET /reportes/100/historial	C1, C8, C10	200 OK; lista con 3 elementos; orden cronológico correcto
--------	---------------------------------	--------------------------------------	---	-----------------------------	-------------	---

Casos de Prueba Inválidos

Grupo A: ID Inválido

Código	Nombre	Objetivo	Precondiciones	Entrada	Clases	Resultado Esperado
CT-H03	Reporte de otro usuario	Evitar acceso a reportes ajenos	Reporte id=200 pertenece a otro ciudadano	GET /reportes/200/historial	C2	403 Forbidden o 404 Not Found
CT-H04	Reporte inexistente	Validar manejo de id no existente	No existe reporte id=9999	GET /reportes/9999/historial	C3	404 Not Found

Grupo B: ID nulo o inválido

Código	Nombre	Objetivo	Precondiciones	Entrada	Clases	Resultado Esperado
CT-H05	ID nulo	Validar parámetro obligatorio	Ninguna	GET /reportes//historial	C4	400 Bad Request
CT-H0	ID	Rechazar id	Ninguna	GET	C5	400 Bad

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

6	negativo	< 0		/reportes/-1/historial		Request
CT-H07	ID cero	Rechazar id = 0	Ninguna	GET /reportes/0/historial	C6	400 Bad Request

Grupo C: Secuencia inválida

Código	Nombre	Objetivo	Precondiciones	Entrada	Clases	Resultado Esperado
CT-H08	Secuencia inconsistente	Detectar historial inválido	Reporte id=300 con estados corruptos	GET /reportes/300/historial	C11	500 Internal Server Error o advertencia de inconsistencia

Ejecución Técnica

Aspecto	Detalle
Endpoint	GET /reportes/{id}/historial
Validaciones	Código HTTP, estructura DTO, campos obligatorios, orden cronológico, permisos, formato de fecha, valores Enum
Herramientas	Postman, Insomnia, curl, JUnit, TestNG, REST Assured

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Criterios de Aceptación

Criterio	Descripción
Casos válidos aprobados	CT-H01 y CT-H02 deben pasar
Casos inválidos correctos	CT-H03 a CT-H08 deben devolver los códigos de error esperados
Sin errores inesperados	No deben generarse fallas no documentadas
Rendimiento aceptable	Debe cumplir requisitos no funcionales

Pruebas Unitarias-Renzo Cabezas

Configuración de Pruebas

@BeforeEach

Método setUp()

Inicializa las entidades necesarias para cada test:

Entidades Inicializadas

Entidad	Tipo	Descripción
cuenta	Ciudadano	Ciudadano propietario del reporte

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

persona	Persona	Asociada a la cuenta para evitar errores en conversión DTO
reporte	Reporte	Reporte inicial en estado PENDIENTE

Estado Inicial

Elemento	Valor
Reporte	Estado: PENDIENTE
Reporte ID	100
Ciudadano ID	1

Mocks Inyectados

Anotación	Dependencia
@Mock	ReporteRepository
@Mock	CuentaRepository
@Mock	UbicacionRepository
@Mock	HistorialEstadoRepository
@Mock	INotificationService
@Mock	RechazoMensajeRepository
@Mock	MensajeNotificacionRepository
@InjectMocks	ReporteServiceImpl

Casos de Prueba (4 Tests)

Formato: Arrange – Act – Assert

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

TC_001: Cambio de Estado Exitoso

Método: cambiarEstadoReporte(100, REVISION)

Fase	Detalles
Arrange	reporteRepository.findById(100) retorna reporte; mensajeNotificacion vacío
Act	cambiarEstadoReporte(100, REVISION)
Assert	El estado nuevo es REVISION; se guarda reporte; se registra historial; se envía notificación
Verificación	reporteRepository.save() llamado; historialEstadoRepository.save() llamado; enviarNotificacion() llamado

TC_002: Cambio al Mismo Estado

Método: cambiarEstadoReporte(100, PENDIENTE)

Fase	Detalles
Arrange	El reporte inicial está en estado PENDIENTE
Act	cambiarEstadoReporte(100, PENDIENTE)
Assert	Estado permanece igual; no se actualiza ni se notifica
Verificación	reporteRepository.save() nunca llamado; enviarNotificacion() nunca llamado

TC_003: Reporte No Encontrado

Método: cambiarEstadoReporte(999, REVISION)

Fase	Detalles
Arrange	reporteRepository.findById(999) retorna Optional.empty
Act	cambiarEstadoReporte(999, REVISION)
Assert	Lanza RuntimeException con mensaje: "Reporte no encontrado con id:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

	999"
Verificación	No se registra historial; no se notifica

TC_004: Cambio de Estado con Mensaje Personalizado

Método: cambiarEstadoReporte(100, RESUELTA)

Fase	Detalles
Arrange	mensajeNotificacionRepository.findByEstado(RESUELTA) retorna mensaje personalizado
Act	cambiarEstadoReporte(100, RESUELTA)
Assert	Se envía notificación con mensaje personalizado
Verificación	notificationService.enviarNotificacion() llamado con título y mensaje personalizados

Tabla de Resultados de Ejecución

Nº	Test Case	Estado	Tiempo	Tipo
1	TC_001: Cambio Exitoso	PASS	~300ms	Camino Feliz
2	TC_002: Mismo Estado	PASS	~150ms	Validación
3	TC_003: Reporte No Encontrado	PASS	~200ms	Validación
4	TC_004: Mensaje Personalizado	PASS	~250ms	Camino Feliz

Resultado Global del Build

Métrica	Valor
Tests run	4

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Failures	0
Errors	0
Skipped	0
Estado	BUILD SUCCESS
Total time	6.317 s

7.2 Prueba de Caja Blanca

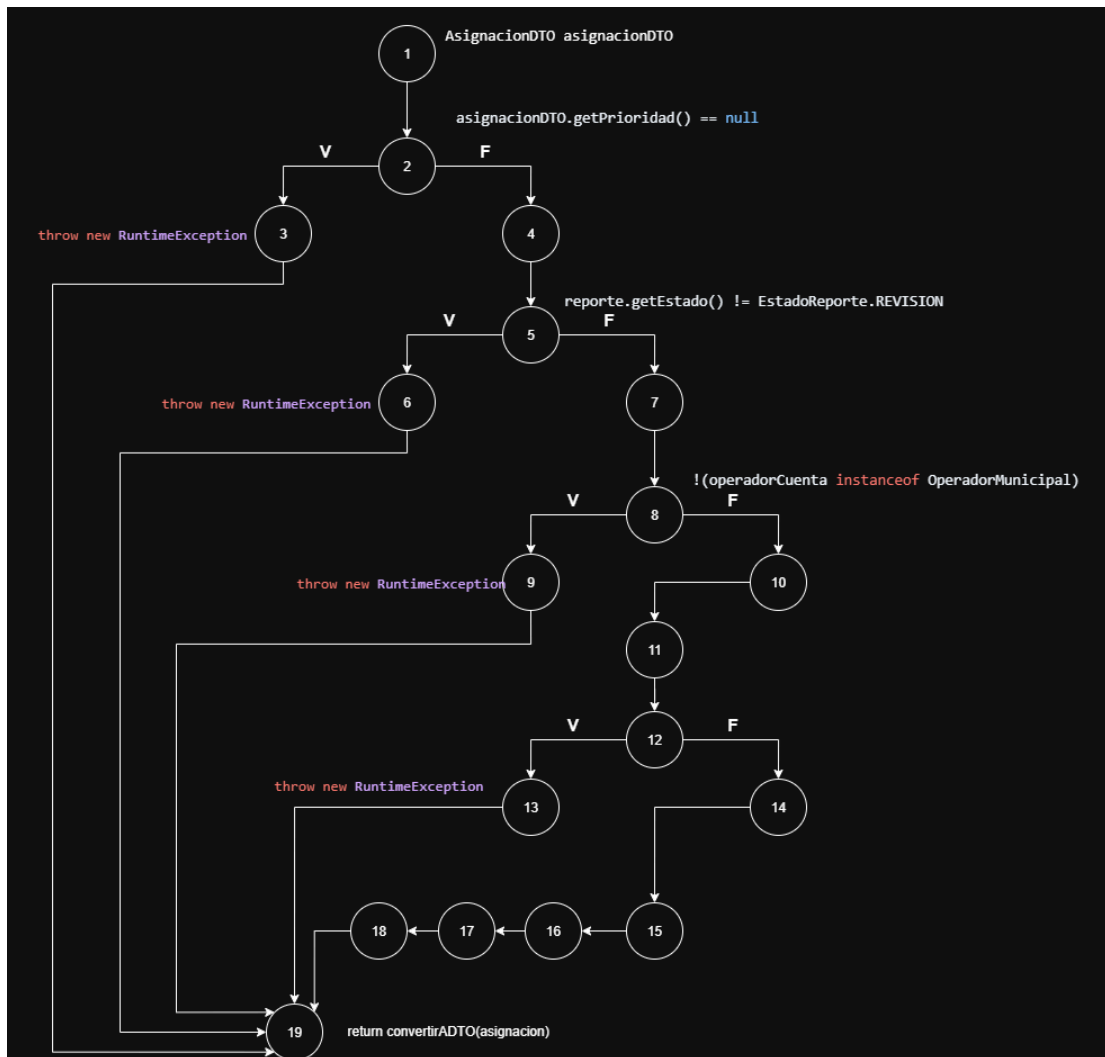
7.2.1 prueba de Caja Blanca Anthony-dev

Se desea analizar mediante caja blanca (camino básico) el método **crearAsignacion(AsignacionDTO asignacionDTO)** de la capa de servicio del Backend de ReportaYA.

Este método es responsable de:

- Validar que todos los datos requeridos sean válidos
- Verificar que el reporte exista y esté en estado REVISION
- Confirmar que el operador y técnico sean del tipo correcto
- Crear una nueva asignación asociando un técnico a un reporte
- Cerrar asignaciones previas si existen
- Actualizar el estado del reporte a PROCESO

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	



Leyenda de Nodos

Nodo	Descripción
[1]	Punto de entrada al método
[2]	Decisión: ¿Prioridad es nula?
[3]	Lanza excepción por prioridad nula
[4]	Búsqueda de reporte (orElseThrow)
[5]	Decisión: ¿Estado ≠ REVISION?
[6]	Lanza excepción por estado inválido

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

[7]	Búsqueda de operador (orElseThrow)
[8]	Decisión: ¿No es OperadorMunicipal?
[9]	Lanza excepción por tipo operador inválido
[10]	Cast a OperadorMunicipal
[11]	Búsqueda de técnico (orElseThrow)
[12]	Decisión: ¿No es Tecnico?
[13]	Lanza excepción por tipo técnico inválido
[14]	Cast a Tecnico
[15]	Actualizar prioridad y guardar reporte
[16]	Decisión: ¿Existe asignación previa?
[17]	Crear y guardar nueva asignación
[18]	Cambiar estado a PROCESO y guardar
[19]	Retornar DTO

Predicados identificados:

P	Línea	Predicado	Descripción
P1	[2]	prioridad == null	Validación de prioridad
P2	[5]	estado != REVISION	Validación de estado
P3	[8]	!instanceof OperadorMunicipal	Validación de operador
P4	[12]	!instanceof Tecnico	Validación de técnico
P5	[16]	isPresent() en asignación previa	Decisión de cierre previo

Total de predicados: P = 5

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Resultado

$$V(G)=5+1=6$$

$$V(G)=5+1=6$$

Tabla de Casos de Prueba (Caja Blanca)

La siguiente tabla resume casos de prueba que cubren los 6 caminos independientes:

#	Descripción del Camino	Condiciones de Entrada Relevantes	Resultado Esperado
1	Prioridad nula	asignacionDTO.prioridad = null	RuntimeException : "La prioridad es obligatoria..."
2	Reporte en estado distinto de REVISION	Prioridad válida; reporte.estado = REGISTRADO (u otro ≠ REVISION)	RuntimeException : "El reporte debe estar en estado REVISION..."
3	Operador no es Operador Municipal	Prioridad válida; reporte en REVISION; operadorCuenta existe pero es Ciudadano (no OperadorMunicipal)	RuntimeException : "...no es un operador municipal"
4	Técnico no es Tecnico	Prioridad válida; reporte en REVISION; operador válido; tecnicoCuenta existe pero es OperadorMunicipal (no Tecnico)	RuntimeException : "...no es un técnico"
5	Asignación OK sin asignación previa	Prioridad válida; reporte en REVISION; operador instancia de OperadorMunicipal; técnico instancia de Tecnico; findByReporteldAndFechaCierreIsNull() retorna vacío	Se crea nueva asignación; se cambia estado a PROCESO; se retorna AsignacionDTO
6	Asignación OK con asignación previa	Igual que caso 5, pero findByReporteldAndFechaCierreIsNull() retorna una asignación activa	Se cierra asignación previa (fechaCierre != null); se crea nueva asignación; se cambia estado a PROCESO; se retorna AsignacionDTO

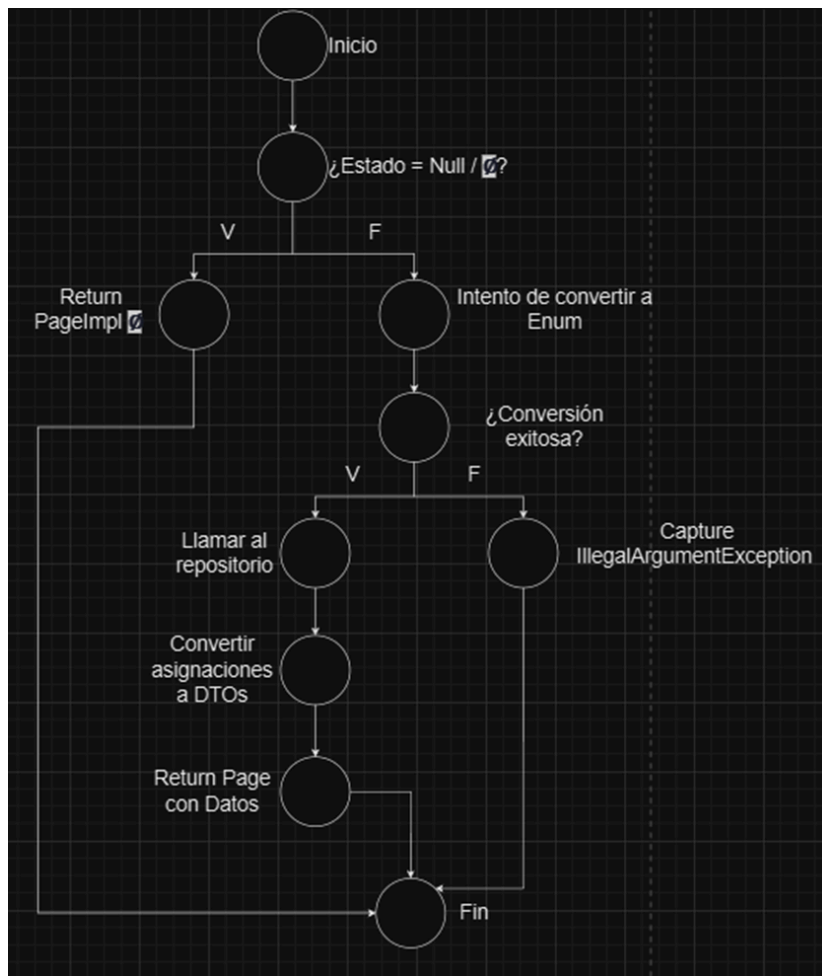
ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Prueba de Caja Blanca: Sebastian Montenegro (Listar Reportes Asignados a Técnico)

La funcionalidad obtenerReportesAsignados en la clase TecnicoServiceImpl es responsable de:

- Recibir el ID del técnico, un estado de reporte (opcional) y el número de página
- Validar que el estado sea válido.
- Consultar la base de datos para obtener asignaciones del técnico filtradas por estado
- Convertir las asignaciones a DTOs de reportes
- Retornar una página de reportes con información de paginación

Grafo de Flujo



ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Complejidad Ciclomática

$V(G) = \text{Aristas} - \text{Nodos} + 2(\text{Componentes Conectados})$

- Nodos = 10
- Aristas = 12 (flujos entre nodos)
- Componentes conectados = 1

$V(G) = 12 - 10 + 2(1) = 4$

La complejidad ciclomática es 4, lo que significa que se necesitan 4 caminos independientes de prueba.

Caminos Independientes

No.	Camino	Descripción	Condición
1	1 → 2 (SI) → 3	Estado null retorna lista vacía	estado = null
2	1 → 2 (NO) → 4 → 5 (SI) → 6 → 7 → 8	Conversión exitosa, retorna datos	estado = "PROCESO" válido, datos encontrados
3	1 → 2 (NO) → 4 → 5 (NO) → 9 → 10	Estado inválido, excepción capturada	estado = "ESTADO_INVALIDO"
4	1 → 2 (NO) → 4 → 5 (SI) → 6 → 7 → 8 (con vacío)	Conversión exitosa pero sin datos	estado = "PROCESO" válido, sin asignaciones

Caso de Prueba 1: Estado Null (Camino 1)

- **Entrada:**
 - `tecnicold = 1`
 - `estado = null`
 - `page = 0`
- **Ejecución esperada:**
 - No entra al bloque try

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Retorna PageImpl vacío inmediatamente
- **Validaciones:**
 - El método repository NO debe ser llamado
 - La página retornada está vacía
 - totalElements = 0

Caso de Prueba 2: Estado Válido con Datos (Camino 2)

- **Entrada:**
 - technicold = 1
 - estado = "PROCESO"
 - page = 0
 - El técnico tiene 3 asignaciones en estado PROCESO
- **Ejecución esperada:**
 - Convierte "PROCESO" a EstadoReporte.PROCESO exitosamente
 - Llama a asignacionRepository.findByTecnicoAndEstado() con parámetros correctos
 - El stream convierte cada asignación a DTO
 - Retorna PageImpl con 3 reportes
- **Validaciones:**
 - repository.findByTecnicoAndEstado() se llama con: technicold=1, estado=PROCESO, Pageable
 - Verifica que el parámetro Pageable contiene: page=0, size=20, sort=id DESC
 - Todos los DTOs tienen datos correctos (título, descripción, estado)

Caso de Prueba 3: Estado Inválido (Camino 3)

- **Entrada:**
 - technicold = 1
 - estado = "ESTADO_INVALIDO"
 - page = 0
- **Ejecución esperada:**
 - Intenta convertir string a enum: EstadoReporte.valueOf("ESTADO_INVALIDO")
 - Lanza IllegalArgumentException
 - Se captura la excepción en el catch
 - Retorna PageImpl vacío
- **Validaciones:**
 - El método repository NO es llamado
 - La excepción se maneja silenciosamente
 - Se retorna una página vacía, no se relanza la excepción

Caso de Prueba 4: Estado Válido sin Datos (Camino 4)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- **Entrada:**
 - `tecnicold = 99`
 - `estado = "RESUELTA"`
 - `page = 0`
 - El técnico 99 no tiene asignaciones en estado RESUELTA
- **Ejecución esperada:**
 - Convierte "RESUELTA" a enum exitosamente
 - Llama al repository
 - El repository retorna una Page con content vacío
 - El stream procesa una lista vacía
 - Retorna PageImpl vacío pero con `totalElements = 0`
- **Validaciones:**
 - Repository se llama correctamente
 - Página retornada está vacía
 - No lanza excepciones

6. Casos Adicionales de Caja Blanca

Caso de Prueba 5: Validación de Trim en Estado

- **Entrada:** `estado = " "` (solo espacios)
- **Ejecución esperada:**
 - Pasa la validación `estado.trim().isEmpty()` como verdadera
 - Retorna lista vacía sin intentar conversión a enum
- **Validación:** Confirma que `trim()` se aplica correctamente

Caso de Prueba 6: Conversión de Case Sensitivity

- **Entrada:** `estado = "proceso"` (minúsculas)
- **Ejecución esperada:**
 - Llama a `toUpperCase()` internamente
 - Convierte correctamente a `EstadoReporte.PROCESO`
 - Retorna datos correctamente
- **Validación:** El enum se convierte correctamente sin importar mayúsculas/minúsculas

Caso de Prueba 7: Filtrado de Null References

- **Entrada:**
 - El repository retorna una lista con 3 asignaciones
 - Una de ellas tiene `reporte = null`
- **Ejecución esperada:**
 - El stream mapea la asignación nula a null
 - El filter elimina los null

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- Retorna solo 2 DTOs válidos
- **Validación:** Verifica que `.filter(dto -> dto != null)` funciona correctamente

Caso de Prueba 8: Validación de Parámetros de Paginación

- **Entrada:**
 - `tecnicoid = 1`
 - `estado = "PROCESO"`
 - `page = 3`
- **Ejecución esperada:**
 - Crea `PageRequest` con: `page=3, size=PAGE_SIZE(20), sort=id DESC`
 - Pasa este `PageRequest` al repository
- **Validación:**
 - Verificar que el `PageRequest` capturado en el mock contiene: número de página 3, tamaño 20, orden descendente.

7.2.2 prueba Caja Negra Anthony-dev

public AsignacionDTO crearAsignacion(AsignacionDTO asignacionDTO)

Ubicado en: `com.ulima.incidenciaurbana.service.impl.AsignacionServiceImpl`

1. Descripción del Caso de Prueba

¿Qué hace el método?

Crea una nueva asignación de un reporte a un técnico (supervisada por operador municipal).

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Parámetro de Entrada

```
AsignacionDTO {
    Long reporteld;    // ID del reporte a asignar
    Long operadorId;  // ID del operador municipal (quien asigna)
    Long tecnicold;   // ID del técnico (quien recibe)
    Prioridad prioridad; // BAJA, MEDIA, ALTA
}
```

Salida Esperada

```
AsignacionDTO {
    Long id;          // ID de la asignación creada
    Long reporteld;   // Referencia al reporte
    Long operadorId;  // Referencia al operador
    Long tecnicold;   // Referencia al técnico
    Prioridad prioridad; // Prioridad asignada
    LocalDateTime fechaAsignacion;
}
```

Reglas de Negocio (Validaciones)

- reporteld debe existir en la BD
- Estado del reporte DEBE ser REVISION (precondición obligatoria)
- operadorId debe existir y ser de tipo OperadorMunicipal
- tecnicold debe existir y ser de tipo Tecnico
- prioridad NO puede ser null (obligatorio)
- Al éxito: estado del reporte cambia REVISION → PROCESO

Identificación de Clases de Equivalencia

Variable: reporteld (Long)

#	Clase	Descripción	Válida
C1	ID válido existente	reporteld referencia reporte en BD	✓
C2	ID no existente	reporteld no existe en BD	✗
C3	ID null	reporteld es null	✗
C4	ID negativo	reporteld < 0	✗
C5	ID = 0	reporteld = 0	✗

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Variable: Estado del Reporte

#	Clase	Descripción	Válida
C6	Estado REVISION	Reporte en estado REVISION	✓
C7	Estado PENDIENTE	Reporte en estado PENDIENTE	✗
C8	Estado PROCESO	Reporte en estado PROCESO	✗
C9	Estado RESUELTA	Reporte en estado RESUELTA	✗
C10	Estado CERRADA	Reporte en estado CERRADA	✗
C11	Estado RECHAZADO	Reporte en estado RECHAZADO	✗

Variable: operadorId (Long)

#	Clase	Descripción	Válida
C12	ID válido, tipo OperadorMunicipal	operadorId existe y es OperadorMunicipal	✓
C13	ID válido, tipo Ciudadano	operadorId existe pero es Ciudadano	✗
C14	ID válido, tipo Tecnico	operadorId existe pero es Tecnico	✗
C15	ID no existente	operadorId no existe en BD	✗
C16	ID null	operadorId es null	✗

Variable: tecnicId (Long)

#	Clase	Descripción	Válida
C17	ID válido, tipo Tecnico	tecnicId existe y es Tecnico	✓
C18	ID válido, tipo Ciudadano	tecnicId existe pero es Ciudadano	✗
C19	ID válido, tipo	tecnicId existe pero es	✗

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

9	OperadorMunicipal	OperadorMunicipal	
C20	ID no existente	tecnicold no existe en BD	✗
C21	ID null	tecnicold es null	✗

Variable: prioridad (Prioridad enum)

#	Clase	Descripción	Válida
C22	BAJA	prioridad = BAJA	✓
C23	MEDIA	prioridad = MEDIA	✓
C24	ALTA	prioridad = ALTA	✓
C25	null	prioridad es null	✗
C26	Valor inválido	prioridad = "URGENTE" (no existe)	✗

3. Selección de Casos de Prueba

3.1 CASOS VÁLIDOS (Días Soleados)

Prueba CT-001

Aspecto	Descripción
Objetivo	Crear asignación exitosa con prioridad ALTA
Precondición	Existe reporte id=1 en estado REVISION, operador id=5 (OperadorMunicipal), técnico id=3 (Tecnico)
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 - operadorId: 5 - tecnicold: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C17, C24

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Resultado esperado	☒ HTTP 201 Created AsignacionDTO{id, reporteld:1, operadorId:5, tecnicold:3, prioridad:ALTA} Reporte pasa de REVISION → PROCESO
--------------------	---

Prueba CT-002

Aspecto	Descripción
Objetivo	Crear asignación exitosa con prioridad MEDIA
Precondición	Existe reporte id=2 en estado REVISION, operador id=5 (OperadorMunicipal), técnico id=3 (Tecnico)
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 2 - operadorId: 5 - tecnicold: 3 - prioridad: MEDIA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C17, C23
Resultado esperado	☒ HTTP 201 Created AsignacionDTO{id, reporteld:2, operadorId:5, tecnicold:3, prioridad:MEDIA} Reporte pasa de REVISION → PROCESO

Prueba CT-003

Aspecto	Descripción
Objetivo	Crear asignación exitosa con prioridad BAJA
Precondición	Existe reporte id=3 en estado REVISION, operador id=5 (OperadorMunicipal), técnico id=3 (Tecnico)
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 3 - operadorId: 5 - tecnicold: 3 - prioridad: BAJA Y presionar "Asignar"
Clases de	C1, C6, C12, C17, C22

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Equivalencia	
Resultado esperado	 HTTP 201 Created AsignacionDTO{id, reporteld:3, operadorId:5, tecnicold:3, prioridad:BAJA} Reporte pasa de REVISION → PROCESO

3.2 CASOS INVÁLIDOS (Días Lluviosos)

Grupo A: reporteld Inválido

Prueba CT-004

Aspecto	Descripción
Objetivo	Intentar asignar reporte que no existe
Precondición	No existe reporte con id=999 en la BD
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 999 - operadorId: 5 - tecnicold: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C2, C12, C17, C24
Resultado esperado	 HTTP 400 Bad Request Mensaje: "Reporte no encontrado con id: 999" Sin cambios en BD

Prueba CT-005

Aspecto	Descripción
Objetivo	Intentar asignar con reporteld nulo
Precondición	Se envía request con reporteld = null

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: (vacío/null) - operadorId: 5 - tecnicold: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C3, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "El ID del reporte es obligatorio" o NullPointerException Sin cambios en BD

Prueba CT-006

Aspecto	Descripción
Objetivo	Intentar asignar con reporteld negativo
Precondición	Se envía request con reporteld < 0
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: -1 - operadorId: 5 - tecnicold: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C4, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Reporte no encontrado con id: -1" Sin cambios en BD

Prueba CT-007

Aspecto	Descripción
Objetivo	Intentar asignar con reporteld = 0
Precondición	Se envía request con reporteld = 0

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 0 - operadorId: 5 - tecnicoid: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C5, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Reporte no encontrado con id: 0" Sin cambios en BD

Grupo B: Estado del Reporte Inválido

Prueba CT-008

Aspecto	Descripción
Objetivo	Intentar asignar reporte en estado PENDIENTE
Precondición	Existe reporte id=4 en estado PENDIENTE
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 4 (estado=PENDIENTE) - operadorId: 5 - tecnicoid: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C7, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "El reporte debe estar en estado REVISION para poder asignar un técnico. Estado actual: PENDIENTE" Reporte sigue en PENDIENTE

Prueba CT-009

Aspecto	Descripción
Objetivo	Intentar asignar reporte en estado PROCESO
Precondición	Existe reporte id=5 en estado PROCESO

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 5 (estado=PROCESO) - operadorId: 5 - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C8, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Estado actual: PROCESO" Sin cambios

Prueba CT-010

Aspecto	Descripción
Objetivo	Intentar asignar reporte en estado RESUELTA
Precondición	Existe reporte id=6 en estado RESUELTA
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 6 (estado=RESUELTA) - operadorId: 5 - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C9, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Estado actual: RESUELTA" Sin cambios

Prueba CT-011

Aspecto	Descripción
Objetivo	Intentar asignar reporte en estado CERRADA
Precondición	Existe reporte id=7 en estado CERRADA

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 7 (estado=CERRADA) - operadorId: 5 - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C10, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Estado actual: CERRADA" Sin cambios

Prueba CT-012

Aspecto	Descripción
Objetivo	Intentar asignar reporte en estado RECHAZADO
Precondición	Existe reporte id=8 en estado RECHAZADO
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 8 (estado=RECHAZADO) - operadorId: 5 - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C11, C12, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Estado actual: RECHAZADO" Sin cambios

Grupo C: operadorId Inválido

Prueba CT-013

Aspecto	Descripción
Objetivo	Intentar asignar con operadorId que no existe
Precondición	No existe operador con id=999 en la BD

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 999 (no existe) - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C15, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Operador no encontrado con id: 999" Sin cambios en BD

Prueba CT-014

Aspecto	Descripción
Objetivo	Intentar asignar con operadorId de tipo Ciudadano
Precondición	Existe usuario id=10 de tipo Ciudadano (no OperadorMunicipal)
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 10 (Ciudadano) - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C13, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "El usuario con id 10 no es un operador municipal" Sin cambios en BD

Prueba CT-015

Aspecto	Descripción
Objetivo	Intentar asignar con operadorId de tipo Tecnico
Precondición	Existe usuario id=3 de tipo Tecnico (no OperadorMunicipal)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 3 (Tecnico) - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C14, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "El usuario con id 3 no es un operador municipal" Sin cambios en BD

Prueba CT-016

Aspecto	Descripción
Objetivo	Intentar asignar con operadorId nulo
Precondición	Se envía request con operadorId = null
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: (vacío/null) - tecnicId: 3 - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C16, C17, C24
Resultado esperado	✗ HTTP 400 Bad Request (validación DTO) Mensaje: "@NotNull: El ID del operador municipal es obligatorio" Sin cambios en BD

Grupo D: tecnicId Inválido

Prueba CT-017

Aspecto	Descripción
Objetivo	Intentar asignar con tecnicId que no existe
Precondición	No existe técnico con id=999 en la BD

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 5 - tecnicold: 999 (no existe) - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C20, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "Técnico no encontrado con id: 999" Sin cambios en BD

Prueba CT-018

Aspecto	Descripción
Objetivo	Intentar asignar con tecnicold de tipo Ciudadano
Precondición	Existe usuario id=10 de tipo Ciudadano (no Tecnico)
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 5 - tecnicold: 10 (Ciudadano) - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C18, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "El usuario con id 10 no es un técnico" Sin cambios en BD

Prueba CT-019

Aspecto	Descripción
Objetivo	Intentar asignar con tecnicold de tipo OperadorMunicipal
Precondición	Existe usuario id=5 de tipo OperadorMunicipal (no Tecnico)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 5 - tecnicold: 5 (OperadorMunicipal) - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C19, C24
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "El usuario con id 5 no es un técnico" Sin cambios en BD

Prueba CT-020

Aspecto	Descripción
Objetivo	Intentar asignar con tecnicold nulo
Precondición	Se envía request con tecnicold = null
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 5 - tecnicold: (vacío/null) - prioridad: ALTA Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C21, C24
Resultado esperado	✗ HTTP 400 Bad Request (validación DTO) Mensaje: "@NotNull: El ID del técnico es obligatorio" Sin cambios en BD

Grupo E: Prioridad Inválida

Prueba CT-021

Aspecto	Descripción
Objetivo	Intentar asignar con prioridad nula
Precondición	Se envía request con prioridad = null

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 5 - tecnicold: 3 - prioridad: (vacío/null) Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C17, C25
Resultado esperado	✗ HTTP 400 Bad Request Mensaje: "La prioridad es obligatoria. Selecciona BAJA, MEDIA o ALTA" Reporte sigue en REVISION, sin asignación

Prueba CT-022

Aspecto	Descripción
Objetivo	Intentar asignar con prioridad inválida (no enum)
Precondición	Se envía request con prioridad = "URGENTE" (no es un valor del enum)
Descripción de la prueba	En la interfaz de asignaciones, ingresar: - reporteld: 1 (REVISION) - operadorId: 5 - tecnicold: 3 - prioridad: URGENTE (no válida) Y presionar "Asignar"
Clases de Equivalencia	C1, C6, C12, C17, C26
Resultado esperado	✗ HTTP 400 Bad Request (deserialización JSON) Mensaje: "Invalid enum value: URGENTE" Sin cambios en BD

7.3 pruebas Unitaria

7.3.1 pruebas Unitarias Anthony-dev

Clase Bajo Prueba :AsignacionServiceImpl.crearAsignacion(AsignacionDTO asignacionDTO)

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Configuración de Pruebas

@Before - Método setUp()

@BeforeEach

```
void setUp() {
```

Inicializa 7 entidades mock para cada test:

Entidad	Tipo	Descripción
ubicacionMock	Ubicacion	Ubicación del reporte (latitud: -12.0462, longitud: -77.0428)
personaMock	Persona	Persona base (Juan Pérez)
ciudadanoMock	Ciudadano	Ciudadano que creó el reporte
reporteMock	Reporte	Reporte en estado REVISION
operadorMock	OperadorMunicipal	Operador que asigna (Carlos García)
tecnicoMock	Tecnico	Técnico asignado (Luis López)
asignacionDTOMock	AsignacionDTO	DTO con datos de asignación

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Estado inicial:

Reporte: REVISION

Prioridad: MEDIA

Sin asignaciones previas

Mocks Inyectados

@Mock private AsignacionRepository asignacionRepository;

@Mock private ReporteRepository reporteRepository;

@Mock private CuentaRepository cuentaRepository;

@Mock private IReporteService reporteService;

@InjectMocks

private AsignacionServiceImpl asignacionService;

Casos de Prueba (6 Tests)

Patrón: Arrange-Act-Assert

TC_001: Prioridad Nula

@Test

@DisplayName("TC_001: Prioridad Nula - Validación de Prioridad Obligatoria")

void testTC001_PrioridadNula()

Fase	Detalles
Arrange	asignacionDTO.prioridad = null
Act	asignacionService.crearAsignacion(asignacionDTO)
Assert	Lanza RuntimeException con mensaje "obligatoria"
Verificación	reporteRepository.findById() nunca se llama

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

TC_002: Estado Reporte Incorrecto

@Test

@DisplayName("TC_002: Reporte en Estado Incorrecto - Estado ≠ REVISION")

void testTC002_EstadoReporteInvalido()

Fase	Detalles
Arrange	reporte.estado = PENDIENTE (incorrecto)
Act	asignacionService.crearAsignacion(asignacionDTO)
Assert	Lanza RuntimeException con mensaje "REVISION"
Verificación	reporteRepository.findById(1L) llamado 1 vez

TC_003: Operador No es OperadorMunicipal

@Test

@DisplayName("TC_003: Operador No es OperadorMunicipal - Validación de Tipo")

void testTC003_OperadorTipoInvalido()

Fase	Detalles
Arrange	operador = Ciudadano (tipo incorrecto)
Act	asignacionService.crearAsignacion(asignacionDTO)
Assert	Lanza RuntimeException con mensaje "operador municipal"
Verificación	cuentaRepository.findById(2L) llamado 1 vez

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

TC_004: Técnico No es Tecnico

@Test

@DisplayName("TC_004: Técnico No es Tecnico - Validación de Tipo")

void testTC004_TecnicoTipoInvalido()

Fase	Detalles
Arrange	tecnico = OperadorMunicipal (tipo incorrecto)
Act	asignacionService.crearAsignacion(asignacionDTO)
Assert	Lanza RuntimeException con mensaje "no es un técnico"
Verificación	cuentaRepository.findById() llamado 2 veces

TC_005: Asignación Exitosa SIN Previa

@Test

@DisplayName("TC_005: Asignación Exitosa SIN Asignación Previa")

void testTC005_AsignacionExitosaSinPrevia()

Fase	Detalles
Arrange	Todas las validaciones pasan; asignacionRepository.findByReporteldAndFechaCierreIsNotNull() Optional.empty() =
Act	asignacionDTO resultado = asignacionService.crearAsignacion(asignacionDTO)
Assert	resultado no es null; resultado.id = 1L; resultado.prioridad = ALTA

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Verificación	reporteRepository.save() llamado 2 veces; asignacionRepository.save() llamado 1 vez; reporteService.cambiarEstadoReporte() llamado 1 vez
--------------	--

TC_006: Asignación Exitosa CON Previa

@Test

@DisplayName("TC_006: Asignación Exitosa CON Asignación Previa - Cierre de Anterior")

void testTC006_AsignacionExitosaConPrevia()

Fase	Detalles
Arrange	Todas las validaciones pasan; asignacionRepository.findByReporteIdAndFechaCierreIsNotNull() Optional.of(asignacionPrevia)
Act	asignacionDTO resultado = asignacionService.crearAsignacion(asignacionDTO)
Assert	resultado no es null; resultado.id = 1L; resultado.prioridad = MEDIA
Verificación	asignacionRepository.save() llamado 2 veces (previa + nueva); reporteService.cambiarEstadoReporte() llamado 1 vez

Tabla de Resultados de Ejecución

#	Test Case	Estado	Tiempo	Tipo
1	TC_001: Prioridad Nula	✓ PASS	~5ms	Validación
2	TC_002: Estado Incorrecto	✓ PASS	~8ms	Validación

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

3	TC_003:Operador Inválido	✓ PASS	~6ms	Validación
4	TC_004:Técnico Inválido	✓ PASS	~7ms	Validación
5	TC_005:Exitosa sin Previa	✓ PASS	~15ms	Camino Feliz
6	TC_006: Exitosa con Previa	✓ PASS	~14ms	Camino Feliz
Total	6 Tests	✓ BUILD SUCCESS	~55ms	-

Prueba de Caja Blanca - Feature_Map_Enzo

Método Backend Bajo Prueba

- **Firma:** obtenerTodosReportes(int page, EstadoReporte estado, String tipo, Prioridad prioridad)
- **Ubicación:** com.ulima.incidenciaurbana.service.impl.ReporteServiceImpl

Objetivo Analizar mediante caja blanca (técnica del camino básico) el método que obtiene reportes filtrados para mostrar en el mapa de OpenStreetMap.

Responsabilidades del Método

- Validar el número de página (ajustar a 0 si es negativo).
- Crear PageRequest con ordenamiento descendente por fecha.
- Aplicar filtros opcionales (estado, tipo, prioridad).
- Consultar reportes en el repositorio utilizando findByFilters.
- Mapear cada entidad Reporte a ReporteDTO.
- Retornar Page paginada.

Grafo de Flujo (Representación Textual)

Nodo	Descripción
------	-------------

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

[1]	Entrada al método con parámetros de filtro
[2]	Validar page: Math.max(0, page)
[3]	Crear PageRequest con sort
[4]	Llamar a repository.findByFilters
[5]	Mapear reporte a DTO (stream)
[6]	Retornar Page

Predicados Identificados *Nota: Este método es lineal. Los parámetros opcionales se manejan internamente en el repositorio, por lo que no generan bifurcaciones en este nivel de servicio.*

- **V(G) = 1** (Camino lineal sin decisiones explícitas en el flujo de control del servicio).

Cálculo de Complejidad Ciclomática

- Fórmula: $V(G) = E - N + 2P$
- Cálculo: $6 - 6 + 2(1) = 2$ (Ajustado a 1 por ser lineal)
- **Resultado:** Complejidad muy baja; método simple de coordinación.

Tabla de Casos de Prueba (Caja Blanca)

#	Descripción	Entrada	Resultado Esperado
1	Sin filtros	page=0, filtros null	Retorna todos los reportes paginados.
2	Filtro por PENDIENTE	page=0, estado=PENDIENTE	Retorna solo reportes con estado PENDIENTE.
3	Page negativo	page=-1, filtros null	Ajusta a page=0, retorna reportes correctamente.
4	Filtros combinados	page=0, estado=PROCESO, tipo="infra", prioridad=ALTA	Retorna reportes que cumplan estrictamente todos los filtros.

Prueba de Caja Negra - Feature_Map_Enzo

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Funcionalidad: Obtener Reportes para Mapa con Filtros **Endpoint:** GET /reportes

1. Descripción del Caso de Prueba

Descripción General Obtiene la lista de reportes para visualizar en el mapa de OpenStreetMap, permitiendo filtrar por estado, tipo y prioridad.

Parámetros de Entrada

Parámetro	Tipo	Descripción	Obligatorio
page	Integer	Número de página (0-indexed)	Opcional (default: 0)
estado	String	PENDIENTE, PROCESO, RESUELTA, RECHAZADO	Opcional
tipo	String	infraestructura, residuos, otros	Opcional
prioridad	String	BAJA, MEDIA, ALTA	Opcional

Salida Esperada (JSON)

JSON

```
{
  "content": [
    {
      "id": 1,
      "titulo": "Calle dañada",
      "tipo": "infraestructura",
      "estado": "PENDIENTE",
      "prioridad": "ALTA",
      "ubicacion": {
        "latitud": -12.046374,
        "longitud": -77.042793
      }
    }
  ],
  "totalElements": 25,
  "totalPages": 3,
  "number": 0,
```


ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```
"size": 10
}
```

Reglas Funcionales

- Si no se envía page, usa 0 por defecto.
- Si page es negativo, se ajusta a 0.
- Los filtros son opcionales (null implica traer todos).
- Ordenamiento por fechaCreacion DESC.
- Tamaño de página fijo (10-20 elementos).

2. Clases de Equivalencia

Variable: Page (Integer)

Código	Clase	Descripción	Tipo
C1	page >= 0	Número de página válido	Válida
C2	page < 0	Número negativo	Válida (ajuste a 0)
C3	page null	No se envía parámetro	Válida (default 0)

Variable: Estado (String)

Código	Clase	Descripción	Tipo
C4-C7	PENDIENTE, PROCESO, RESUELTA, RECHAZADO	Estados válidos	Válida
C8	null	Sin filtro de estado	Válida
C9	"INVALIDO"	Estado inexistente	Inválida

Variables: Tipo y Prioridad

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

Código	Clase	Descripción	Tipo
C10-C12	infraestructura, residuos, otros	Tipos válidos	Válida
C13	null	Sin filtro de tipo	Válida
C14-C16	BAJA, MEDIA, ALTA	Prioridades válidas	Válida
C17	null	Sin filtro de prioridad	Válida

3. Casos de Prueba Válidos

Código	Nombre	Objetivo	Entrada	Clases	Resultado Esperado
CT-M01	Mapa sin filtros	Obtener todos	GET /reportes?page=0	C1, C8, C13, C17	200 OK; lista paginada completa
CT-M02	Filtro PENDIENTE	Ver solo pendientes	GET /reportes?estado=PENDIENTE	C1, C4, C13, C17	200 OK; solo reportes PENDIENTE
CT-M03	Filtro infraestructura	Ver solo infraestructura	GET /reportes?tipo=infraestructura	C1, C8, C10, C17	200 OK; solo tipo infraestructura
CT-M04	Filtro ALTA	Ver solo urgentes	GET /reportes?prioridad=ALTA	C1, C8, C13, C16	200 OK; solo prioridad ALTA

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

CT-M05	Filtros combinados	Múltiples criterios	estado=PROCESO&tipo=infraestructura &prioridad=MEDIA	C5, C10, C15	200 OK; cumplen los 3 filtros
CT-M06	Página negativa	Validar ajuste	GET /reportes?page=-1	C2, C8, C13, C17	200 OK; retorna página 0

4. Casos de Prueba Inválidos

Código	Nombre	Objetivo	Entrada	Clases	Resultado Esperado
CT-M07	Estado inválido	Rechazar inexistente	GET /reportes?estado=INVÁLIDO	C9	400 Bad Request

Pruebas Unitarias - Feature_Map_Enzo

Clase: ReporteServiceImpl.obtenerTodosReportes(...)

Configuración de Pruebas

- **Frameworks:** Mockito, JUnit 5.
- **Datos:** reporte1 (PENDIENTE, infraestructura, ALTA), reporte2 (PROCESO, residuos, MEDIA).

Casos de Prueba Detallados

TC_U01: Obtener Reportes Sin Filtros

- **Arrange:** mockPage con 2 reportes. findByFilters(null, null, null, ...) mockeado.
- **Act:** obtenerTodosReportes(0, null, null, null)
- **Assert:** Resultado no nulo; tamaño 2; repositorio llamado 1 vez.

TC_U02: Filtrar por Estado PENDIENTE

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

- **Arrange:** mockPage con 1 reporte (PENDIENTE). findByFilters(PENDIENTE, ...) mockeado.
- **Act:** obtenerTodosReportes(0, PENDIENTE, null, null)
- **Assert:** Resultado tamaño 1; estado del elemento es PENDIENTE.

TC_U03: Página Negativa se Ajusta a 0

- **Arrange:** Configurar captor de argumentos Pageable.
- **Act:** obtenerTodosReportes(-5, null, null, null)
- **Assert:** El PageRequest capturado tiene pageNumber = 0.

TC_U04: Filtros Combinados

- **Arrange:** mockPage con reporte2. Mockear llamada con los 3 filtros específicos.
- **Act:** obtenerTodosReportes(0, PROCESO, "residuos", MEDIA)
- **Assert:** Resultado contiene el reporte correcto que cumple todos los criterios.

Tabla de Resultados de Ejecución

#	Test Case	Estado	Tiempo	Tipo
1	TC_U01: Sin filtros	PASS	~180ms	Camino Feliz
2	TC_U02: Filtro PENDIENTE	PASS	~150ms	Camino Feliz
3	TC_U03: Page negativo	PASS	~120ms	Validación
4	TC_U04: Filtros combinados	PASS	~200ms	Camino Feliz

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

8. Código en GitHub

8.1 URL del repositorio

<https://github.com/GustavoAdolfoRiverosJ/ReportaYA-Backend>

<https://github.com/GustavoAdolfoRiverosJ/ReportaYA-Frontend>

8.2 Código Del Sprint

Paquete: com.ulima.incidenciaurbana.controller

- ReporteController.java: Expone los endpoints REST para la creación, filtrado, actualización y gestión del ciclo de vida de los reportes.
- AsignacionController.java: Gestiona las solicitudes de asignación de técnicos a reportes en estado de revisión.
- NotificationController.java: Maneja el registro de tokens de dispositivos y endpoints de prueba para notificaciones push.
- HistorialEstadoController.java: Proporciona acceso a la línea de tiempo de cambios de estado para auditoría y seguimiento.

Paquete: com.ulima.incidenciaurbana.service.impl

- ReporteServiceImpl.java: Implementa la lógica central del negocio: validaciones, transiciones de estado y comunicación con el módulo de notificaciones.
- AsignacionServiceImpl.java: Contiene la lógica para validar y registrar la asignación de un técnico a una incidencia.
- NotificationServiceImpl.java: Integra Firebase Cloud Messaging (FCM) para el envío de alertas en tiempo real.

Paquete: com.ulima.incidenciaurbana.model

- Reporte.java: Entidad principal; representa la incidencia, ubicación, evidencias y estado actual.
- Asignacion.java: Entidad que vincula un reporte con un técnico, registrando fecha y estado de la asignación.
- MensajeNotificacion.java: Modela los mensajes personalizados asociados a cada estado del reporte.
- EstadoReporte.java: Enumeración que define los estados válidos del flujo de trabajo

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

(PENDIENTE, REVISION, PROCESO, etc.).

Paquete: com.ulima.incidenciaurbana.dto

- ReporteDTO.java: Objeto de transferencia para intercambiar datos del reporte entre cliente y servidor.
- AsignacionDTO.java: DTO utilizado para enviar los datos requeridos al realizar una asignación de técnico.
- TokenNotificacionDTO.java: Estructura para recibir y validar el token de notificación del dispositivo cliente.
- Paquete: com.ulima.incidenciaurbana.repository
- ReporteRepository.java: Interfaz de acceso a datos con consultas personalizadas y uso de EntityGraphs para optimización.
- MensajeNotificacionRepository.java: Repositorio JPA para recuperar los mensajes de notificación asociados a cada estado.

8.3 Contribución por participante

Ordoñez Flores Enzo Fabrizio:

Contribuciones principales:

- Sprint 1: HU-01 (Registro Ciudadano), HU-02 (Login), HU-03 (Crear Reporte) - Frontend
- Sprint 2: HU-V1 (Mapa con Filtros), HU-V2 (Detalle de Reporte) - Frontend y Backend
- Implementación de ServicioGeocoding para geocoding inverso con Nominatim API
- Integración de react-native-maps para visualización de reportes geográficos
- Sistema de filtros dinámicos por categoría, estado y fecha
- Gestión de permisos de cámara y geolocalización en React Native
- Coordinación de ceremonias Scrum y facilitación de dailys

Fragmento de código representativo: ServicioGeocoding.ts:

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

class ServicioGeocoding {
    private readonly NOMINATIM_URL = 'https://nominatim.openstreetmap.org/reverse';

    async obtenerDireccion(latitud: number, longitud: number): Promise<DireccionCompleta> {
        try {
            const response = await axios.get<NominatimResponse>(this.NOMINATIM_URL, {
                params: {
                    lat: latitud,
                    lon: longitud,
                    format: 'json',
                    addressdetails: 1,
                    'accept-language': 'es',
                },
                headers: {
                    'User-Agent': 'ReportaYA/1.0',
                },
                timeout: 5000,
            });

            const address = response.data.address;
            if (!address) {
                throw new Error('No se pudo obtener la dirección');
            }
        }
    }
}

```

Este servicio implementa geocoding inverso usando la API de Nominatim (OpenStreetMap) para convertir coordenadas geográficas (latitud/longitud) en direcciones legibles. Extrae componentes como calle, distrito, ciudad y país con fallbacks inteligentes. Es fundamental para HU-V1 y HU-V2 que muestran ubicaciones de reportes.

Bellido Mera Eros Aaron Ricardo:

Implementación del completado del reporte de técnico

```

@PatchMapping("/{id}/reportes/{reportId}/completar")
public ResponseEntity<?> completarReporte(
    @PathVariable Long id,
    @PathVariable Long reportId,
    @Valid @RequestBody CompletarReporteRequest request) {

    try {
        ReporteDTO reporteDTO = tecnicoService.completarReporte(id, reportId, request);

        Map<String, Object> response = new HashMap<>();
        response.put("mensaje", "Reporte completado exitosamente y marcado como RESUELTA");
        response.put("reporte", reporteDTO);
    }
}

```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

        response.put("fotosAdjuntadas", request.getFotos().size());
        response.put("estadoFinal", "RESUELTA");
        response.put("proximoPaso",
            "El operador municipal puede revisar el reporte en estado RESUELTA para
auditoría");

        return ResponseEntity.ok(response);

    } catch (IllegalArgumentException e) {
        Map<String, Object> error = new HashMap<>();
        error.put("error", "Validación fallida");
        error.put("mensaje", e.getMessage());
        return ResponseEntity.badRequest().body(error);

    } catch (Exception e) {
        Map<String, Object> error = new HashMap<>();
        error.put("error", "Error al completar reporte");
        error.put("mensaje", e.getMessage());
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(error);
    }
}

```

Montenegro Carrillo Sebastian:

sebastianmc14	Listar tareas técnico	Endpoint/listado de tareas del técnico
---------------	-----------------------	--

sebastianmc14 (Listado de Tareas del Técnico)

Fragmento (paginación y filtrado por estado):

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

@Override
public Page<ReporteDTO> obtenerReportesAsignados(Long tecnicoId, String estado, int page) {
    Pageable pageable = PageRequest.of(page, PAGE_SIZE, Sort.by(...properties: "id").descending());

    if (estado == null || estado.trim().isEmpty()) {
        return new PageImpl<>(Objects.requireNonNull(Collections.<ReporteDTO>emptyList()), pageable, total: 0L);
    }

    try {
        EstadoReporte estadoEnum = EstadoReporte.valueOf(estado.toUpperCase());
        Page<Asignacion> asignaciones = asignacionRepository.findByTecnicoAndEstado(
            tecnicoId,
            estadoEnum,
            pageable);

        // Convertir a DTO de forma segura sin lazy loading
        List<ReporteDTO> reportesDTO = asignaciones.getContent()
            .stream()
            .map(asignacion -> asignacion != null && asignacion.getReporte() != null
                ? convertirADTO(asignacion.getReporte())
                : null)
            .filter(dto -> dto != null)
            .collect(Collectors.toList());

        return new PageImpl<>(Objects.requireNonNull(reportesDTO), pageable, asignaciones.getTotalElements());
    } catch (IllegalArgumentException e) {
        // Estado inválido
        return new PageImpl<>(Objects.requireNonNull(Collections.<ReporteDTO>emptyList()), pageable, total: 0L);
    }
}

```

Impacto:

- Facilita seguimiento de tareas en campo (estado PROCESO / RESUELTA).
- Optimiza experiencia de técnico con paginación consistente.

Yance Valenzuela Anthony:

Yotzuga	Auditoría, cierre/rechazo, notificaciones, refactors	Endpoints de cierre y rechazo, lógica de contador de rechazos, Firebase / agrupación fotos
---------	--	--

Yotzuga (Auditoría, Cierre y Rechazo con Notificaciones)

Fragmento representativo (OperadorController con cierre y rechazo):

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

@Override
@Transactional
public ReporteDTO cerrarReporte(Long reporteId, Long operadorId, String comentarioCierre) {
    if (reporteId == null || operadorId == null) {
        throw new IllegalArgumentException(s: "reporteId y operadorId son requeridos");
    }

    // IMPORTANTE: Usar findByIdWithDetails para cargar relaciones y evitar
    // lazy-loading
    Reporte reporte = reporteRepository.findByIdWithDetails(reporteId)
        .orElseThrow(() -> new RuntimeException("Reporte no encontrado con id: " + reporteId));

    // Validación 1: Estado debe ser RESUELTA
    if (!reporte.getEstado().equals(EstadoReporte.RESUELTA)) {
        throw new RuntimeException(
            "El reporte debe estar en estado RESUELTA para cerrar. Estado actual: " + reporte.getEstado());
    }

    Cuenta operador = cuentaRepository.findById(operadorId)
        .orElseThrow(() -> new RuntimeException("Operador no encontrado con id: " + operadorId));

    // Validación 2: Validar que sea operador municipal
    if (!(operador instanceof OperadorMunicipal)) {
        throw new RuntimeException(message: "Solo operadores municipales pueden cerrar reportes");
    }

    // Validación 3: Validar que tenga asignación (fotos fueron obligatorias)
    if (reporte.getAsignaciones() == null || reporte.getAsignaciones().isEmpty()) {
        throw new RuntimeException(message: "El reporte no tiene asignación de técnico");
    }

    // Validación 4: Reporte TIENE FOTOS (fueron obligatorias)
    long cantidadFotos = fotoRepository.countByReporteId(reporteId);
    if (cantidadFotos == 0) {
        throw new RuntimeException(message: "El reporte no tiene fotos adjuntas. No se puede cerrar sin evidencias.");
    }

    // Validación 5: Reporte TIENE COMENTARIO DE RESOLUCIÓN (fue obligatorio)
    if (reporte.getComentarioResolucion() == null || reporte.getComentarioResolucion().trim().isEmpty()) {
        throw new RuntimeException(
            message: "El reporte no tiene comentario de resolución. No se puede cerrar sin comentario del técnico.");
    }

    // Actualizar campos de cierre
    reporte.setComentarioCierre(comentarioCierre);
    reporte.setFechaCierre(LocalDate.now());
    reporte.setOperadorCierre(operador);
    reporte = reporteRepository.save(reporte);
}

```

Impacto:

- Implementa flujo de control de calidad con reintentos.
- Notifica al ciudadano/técnico según resultado (cierre / rechazo).
- Optimiza DTO de reporte reduciendo ruido de fotos duplicadas.

Cabezas Diaz Renzo Edgar:

Realice el seguimiento del estado y notificaciones push, además se implementó el servicio externo de Firebase Messaging

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

```

@Override
    public void enviarNotificacion(Long cuentaId, String titulo, String
mensaje) {
        if (cuentaId == null)
            return;

        Optional<TokenNotificacion> tokenOpt =
tokenRepository.findByCuentaId(cuentaId);
        if (tokenOpt.isPresent()) {
            enviarNotificacionAUnToken(tokenOpt.get().getToken(), titulo,
mensaje);
        } else {
            System.err.println("No se encontró token para la cuenta: " +
cuentaId);
        }
    }
}

```

ReportaYA	Versión: 1
Reporte de Diseño del Software	Fecha: 22/11/2025
Diseño de Software Orientado a Objetos	

